

 **Devin AI Export - PDF Version**

Generated on: 11/4/2025, 10:05:56 PM

Source: Devin AI Platform

## Overview

---

### Relevant source files

- [ README.md](<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/README.md>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/README.md>))
- [ pom.xml](<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml>))
- [ src/main/java/com/difisoft/marketcollector/Application.java](<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/Application.java>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/Application.java>))

## Purpose and Scope

This document provides a high-level introduction to the Market Collector Lotte system, a Spring Boot-based microservice designed to collect, process, and distribute real-time and historical market data from Vietnamese stock exchanges (HOSE, HNX, UPCOM). This page covers the system's architecture, core capabilities, technology stack, and integration points.

For detailed information about specific subsystems, see:

- Application setup and deployment: [Getting Started](#)
- Configuration management details: [Configuration Management](#)
- Real-time data collection implementation: [Real-time Data Collection](#)
- Data structures and models: [Data Models](#)
- Background services and scheduled tasks: [Services & Processing](#)

## System Overview

Market Collector Lotte is a production-grade data collection hub that integrates with multiple external systems to gather comprehensive financial market data. The system operates in two primary modes:

1. **Real-time Collection:** Establishes WebSocket connections to the HTS (Handy Trading System) to receive live market data streams including quotes, bid/ask spreads, and session events
2. **API-based Collection:** Makes REST API calls to Lotte Securities endpoints to retrieve historical data, symbol information, and market metadata

The application serves as a critical integration point in the trading infrastructure, transforming raw market data into standardized formats and distributing it to downstream systems via multiple channels (Kafka, Redis, MongoDB).

Sources: <https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/README.md#L1-L12>

[README.md1-12](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/README.md#L1-L12) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/README.md#L1-L12>)

[src/main/java/com/difisoft/marketcollector/Application.java1-28](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/Application.java#L1-L28) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/Application.java#L1-L28>)  
(<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/Application.java#L1-L28>)

## Core Capabilities

### Market Data Collection

The system collects and processes multiple types of market data:

| Data Type      | Description                             | Source        | Update Frequency       |
|----------------|-----------------------------------------|---------------|------------------------|
| Stock Quotes   | Real-time price, volume, change         | HTS WebSocket | Real-time (sub-second) |
| Index Quotes   | Market index values and performance     | HTS WebSocket | Real-time              |
| Market Depth   | Bid/Ask prices and volumes (order book) | HTS WebSocket | Real-time              |
| Daily Data     | OHLCV historical data                   | Lotte API     | Daily                  |
| Symbol Info    | Security metadata, classifications      | Lotte API     | Daily/On-demand        |
| Session Events | Market open/close, trading halts        | HTS WebSocket | Event-driven           |

Sources: <https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/README.md#L36-L56>

[README.md36-56](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/README.md#L36-L56) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/README.md#L36-L56>)

### Supported Markets and Asset Types

The system handles seven distinct asset classes across three Vietnamese exchanges:

#### Markets:

- **HOSE** (Ho Chi Minh Stock Exchange): Main board, largest market cap
- **HNX** (Hanoi Stock Exchange): Small-cap and growth companies
- **UPCOM**: Unlisted Public Company Market

#### Asset Types:

- **STOCK** : Equity shares (exchange codes: 01, 11, 31)
- **INDEX** : Market indices (exchange codes: 09, 19, 39)

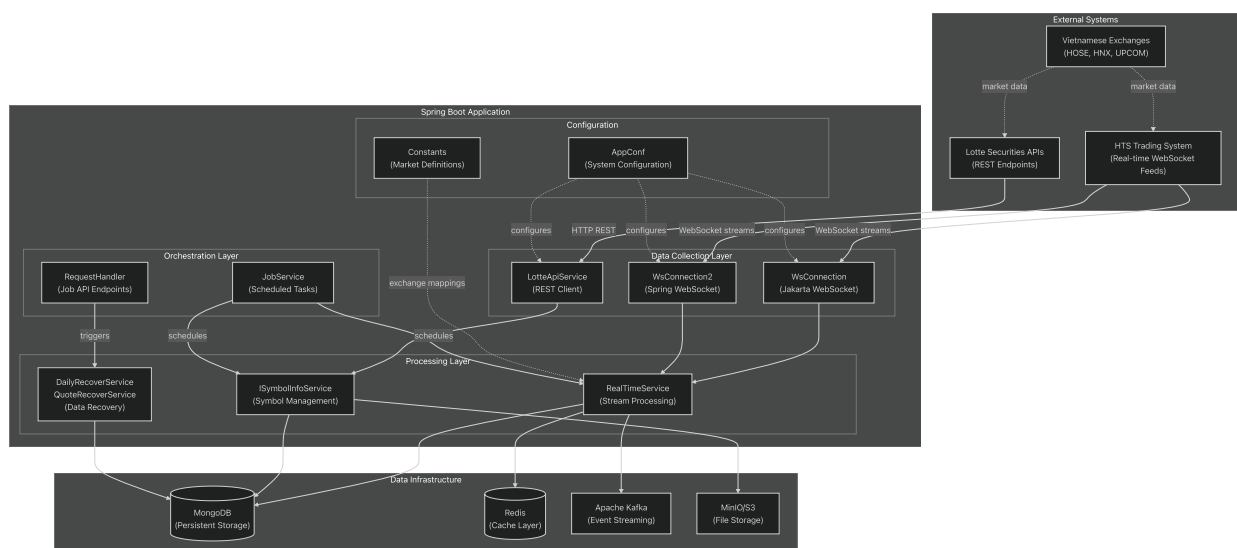
- `FUND` : Investment funds (exchange code: 03)
- `ETF` : Exchange-traded funds (exchange code: 04)
- `BOND` : Debt instruments (exchange code: 12)
- `COVERED_WARRANT` : Derivative instruments (exchange code: 07)
- `FUTURES` : Futures contracts (exchange code: 15)

Sources: <https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/README.md#L43-L56>

[README.md#L43-L56](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/README.md#L43-L56) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/README.md#L43-L56>)

## System Architecture

The following diagram shows the high-level architecture and how code components map to system responsibilities:



### Architecture Layers:

- 1. Data Collection Layer:** Handles all external system integrations
  - `WsConnection` and `WsConnection2` : Two WebSocket implementations for HTS connectivity
  - `LotteApiService` : REST API client for Lotte Securities
- 2. Processing Layer:** Transforms and validates market data
  - `RealTimeService` : Processes real-time WebSocket streams
  - `ISymbolInfoService` : Manages symbol metadata and classifications
  - Recovery Services: Backfill missing or failed data collection
- 3. Orchestration Layer:** Coordinates scheduled tasks and manual operations
  - `JobService` : Manages scheduled jobs (symbol downloads, real-time start/stop)
  - `RequestHandler` : Exposes job-based API for on-demand operations
- 4. Configuration:** Centralized system configuration
  - `AppConf` : Main configuration class with system connections, API settings, feature flags

- **Constants** : Market definitions, exchange codes, type mappings

Sources: <https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/README.md#L14-L33>)

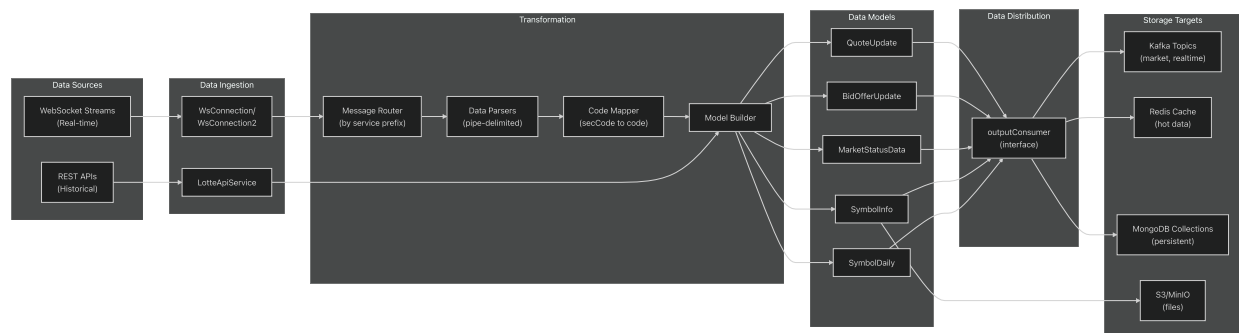
[README.md14-33](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/README.md#L14-L33) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/README.md#L14-L33>)

src/main/java/com/difisoft/marketcollector/Application.java16-21](<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/Application.java#L16-L21>

(<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/Application.java#L16-L21>)

## Data Flow Pipeline

The following diagram illustrates how data flows through the system from collection to distribution:



### Key Processing Steps:

1. **Ingestion**: Data arrives via WebSocket streams (real-time) or REST API calls (historical)
2. **Routing**: WebSocket messages are routed by service prefix (e.g., `auto.gt.*` for stock quotes, `auto.bo.*` for bid/offer)
3. **Parsing**: Pipe-delimited messages are split and parsed into structured fields
4. **Mapping**: Security codes are mapped from HTS format to standardized codes
5. **Model Building**: Parsed data is transformed into typed model objects (`QuoteUpdate`, `SymbolInfo`, etc.)
6. **Distribution**: All data flows through the `outputConsumer` interface for unified distribution
7. **Storage**: Data is persisted to multiple targets based on access patterns (hot: Redis, stream: Kafka, persistent: MongoDB)

Sources: <https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/README.md#L164-L171>)

[README.md164-171](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/README.md#L164-L171) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/README.md#L164-L171>)

## Technology Stack

### Core Framework

The application is built on **Spring Boot 3.1.1** with Java 17, leveraging several Spring modules:

| Component             | Technology                                    | Purpose                        |
|-----------------------|-----------------------------------------------|--------------------------------|
| Application Framework | <code>spring-boot-starter</code>              | Core Spring Boot functionality |
| Data Access           | <code>spring-boot-starter-data-mongodb</code> | MongoDB integration            |
| Caching               | <code>spring-boot-starter-data-redis</code>   | Redis integration              |
| WebSocket             | <code>spring-boot-starter-websocket</code>    | WebSocket client support       |
| Scheduling            | <code>@EnableScheduling</code>                | Scheduled task execution       |
| Async Processing      | <code>@EnableAsync</code>                     | Asynchronous method execution  |

Sources: (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml#L14-L18>)

[pom.xml#L14-18](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml#L14-L18) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml#L14-L18>)

[pom.xml#L24-150](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml#L24-L150)] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml#L24-L150>)

[src/main/java/com/difisoft/marketcollector/Application.java#L14-20](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/Application.java#L14-L20)] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/Application.java#L14-L20>)

## External Dependencies

### Proprietary Libraries:

- `model-common` (1.0.0): Shared data models
- `kafka-common` (1.0.0): Kafka integration utilities
- `market-common-v2` (1.0.0): Market-specific models and repositories
- `job-common` (1.0.0): Job orchestration framework
- `redis-dao` (1.0.0): Redis data access layer
- `hts-connection` (1.0.0): HTS system connectivity

### Third-party Libraries:

- Apache Kafka (2.3.1): Message streaming
- MinIO SDK (8.3.7): Object storage client
- AWS S3 SDK (1.11.311): S3-compatible storage
- Auth0 JWT (4.4.0): Authentication tokens
- Lombok: Code generation

Sources: (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml#L69-L102>)

[pom.xml#L69-102](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml#L69-L102) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml#L69-L102>)

pom.xml#L105-L136](<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml#L105-L136>)  
(<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml#L105-L136>))

## Infrastructure Dependencies

The system requires the following external services:

| Service      | Minimum Version | Purpose            |
|--------------|-----------------|--------------------|
| MongoDB      | 4.4+            | Primary data store |
| Redis        | 6.0+            | Caching layer      |
| Apache Kafka | 2.3+            | Event streaming    |
| MinIO/S3     | -               | File storage       |
| HTS System   | -               | Market data source |

Sources: (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/README.md#L58-L65>)

[README.md#L58-L65](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/README.md#L58-L65) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/README.md#L58-L65>)

## Integration Points

### HTS Trading System Integration

The system establishes WebSocket connections to the HTS system for real-time market data. Two implementations provide redundancy and feature compatibility:

- **WsConnection:** Jakarta WebSocket-based implementation
- **WsConnection2:** Spring WebSocket-based implementation

Both implementations support:

- Automatic connection management with exponential backoff retry (5s to 120s)
- Channel subscription for specific market data types
- Health monitoring and automatic reconnection
- Pipe-delimited message parsing

Sources: (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/README.md#L157-L162>)

[README.md#L157-L162](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/README.md#L157-L162) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/README.md#L157-L162>)

### Lotte Securities API Integration

The `LotteApiService` component integrates with multiple Lotte Securities REST endpoints:

#### Endpoints:

- Market data: Stock quotes, index quotes, ETF prices

- Symbol information: Security names, types, exchange listings
- Historical data: Daily OHLCV data, index performance
- Market depth: Best bid/ask prices and volumes

**Configuration:**

- Base URL: Configured via `AppConf.apiConnection.baseUrl`
- Authentication: API key-based authentication
- Connection settings: Timeouts, retry policies

Sources: <https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/README.md#L146-L162>

[README.md146-162](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/README.md#L146-L162) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/README.md#L146-L162>).

**Data Distribution Targets**

Processed market data is distributed to multiple downstream systems:

**Kafka Topics:**

- `market` : General market data updates
- `realtime` : Real-time quote and bid/offer updates
- `symbolInfoUpdate` : Symbol metadata changes

**Redis Cache:**

- Real-time quotes (sub-millisecond access)
- Market status (open/close states)
- Hot data with TTL expiration

**MongoDB Collections:**

- `SymbolInfo` : Symbol master data
- `SymbolDaily` : Daily OHLCV data
- `IndexDaily` : Index performance data
- `MarketInfo` : Market metadata

**S3/MinIO Storage:**

- Symbol list files
- Market configuration files
- Historical data archives

Sources: <https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/README.md#L21-L23>

[README.md21-23](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/README.md#L21-L23) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/README.md#L21-L23>).

**Operational Characteristics****Market Hours Alignment**

The system respects Vietnamese market trading hours, configured via `AppConf.realtime.workingTime`:

- **Trading Days:** Monday to Friday (weekDays: [2, 3, 4, 5, 6])
- **Trading Hours:** 09:00 - 15:00 (configurable)
- **Auto Start/Stop:** `JobService` automatically starts real-time collection during market hours and stops after close

Sources: [\\_\(https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/README.md#L133-L138\)](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/README.md#L133-L138)

[README.md133-138](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/README.md#L133-L138) [\\_\(https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/README.md#L133-L138\)](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/README.md#L133-L138)

## Connection Management

Robust connection handling ensures continuous data collection:

- **Health Checks:** Periodic WebSocket connection validation
- **Auto-reconnect:** Automatic reconnection on connection failures
- **Exponential Backoff:** Progressive retry delays (5s initial, up to 120s maximum)
- **Error Recovery:** Graceful handling of network interruptions

## Performance Characteristics

- **High Throughput:** Optimized for processing thousands of messages per second
- **Low Latency:** Real-time data processing with minimal buffering
- **Horizontal Scaling:** Stateless design supports multiple instances
- **Native Compilation:** GraalVM native image support for optimal performance

Sources: [\\_\(https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/README.md#L213-L218\)](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/README.md#L213-L218)

[README.md213-218](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/README.md#L213-L218) [\\_\(https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/README.md#L213-L218\)](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/README.md#L213-L218)

## Deployment Model

The application is containerized using Docker with a multi-stage build:

1. **Builder Stage:** Maven 3.9.5 with Temurin 17 for compilation
2. **Production Stage:** Temurin 17 JRE for runtime
3. **Configuration:** Layered configuration system via `entrypoint.sh`
4. **Health Monitoring:** Actuator endpoints for orchestration platforms

The service runs on port 8080 and exposes Spring Boot Actuator endpoints for health checks and monitoring.

Sources: [\\_\(https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/README.md#L103-L110\)](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/README.md#L103-L110)

[README.md103-110](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/README.md#L103-L110) [\\_\(https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/README.md#L103-L110\)](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/README.md#L103-L110)

# Getting Started

## Overview

Relevant source files

- [ README.md](<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/README.md>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/README.md>))
- [ pom.xml](<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml>))

This page provides instructions for setting up, building, and deploying the Market Collector Lotte application. This Spring Boot microservice collects Vietnamese market data from multiple sources including HTS real-time feeds and Lotte Securities APIs.

For detailed information about the core architecture and Spring configuration, see [Core Architecture](#). For comprehensive build configuration details, see [Build Configuration](#). For container orchestration and deployment specifics, see [Containerization & Deployment](#).

## Prerequisites

The Market Collector Lotte application requires the following components:

| Component | Version | Purpose                        |
|-----------|---------|--------------------------------|
| Java JDK  | 17+     | Runtime environment            |
| Maven     | 3.9.5+  | Build tool                     |
| Docker    | Latest  | Containerization               |
| MongoDB   | 4.4+    | Symbol and market data storage |
| Redis     | 6.0+    | Caching and real-time data     |
| Kafka     | 2.3.1+  | Message distribution           |

## External System Access Required:

- HTS Trading System connection for real-time market data
- Lotte Securities API credentials for market information
- MinIO/S3 compatible storage for file operations

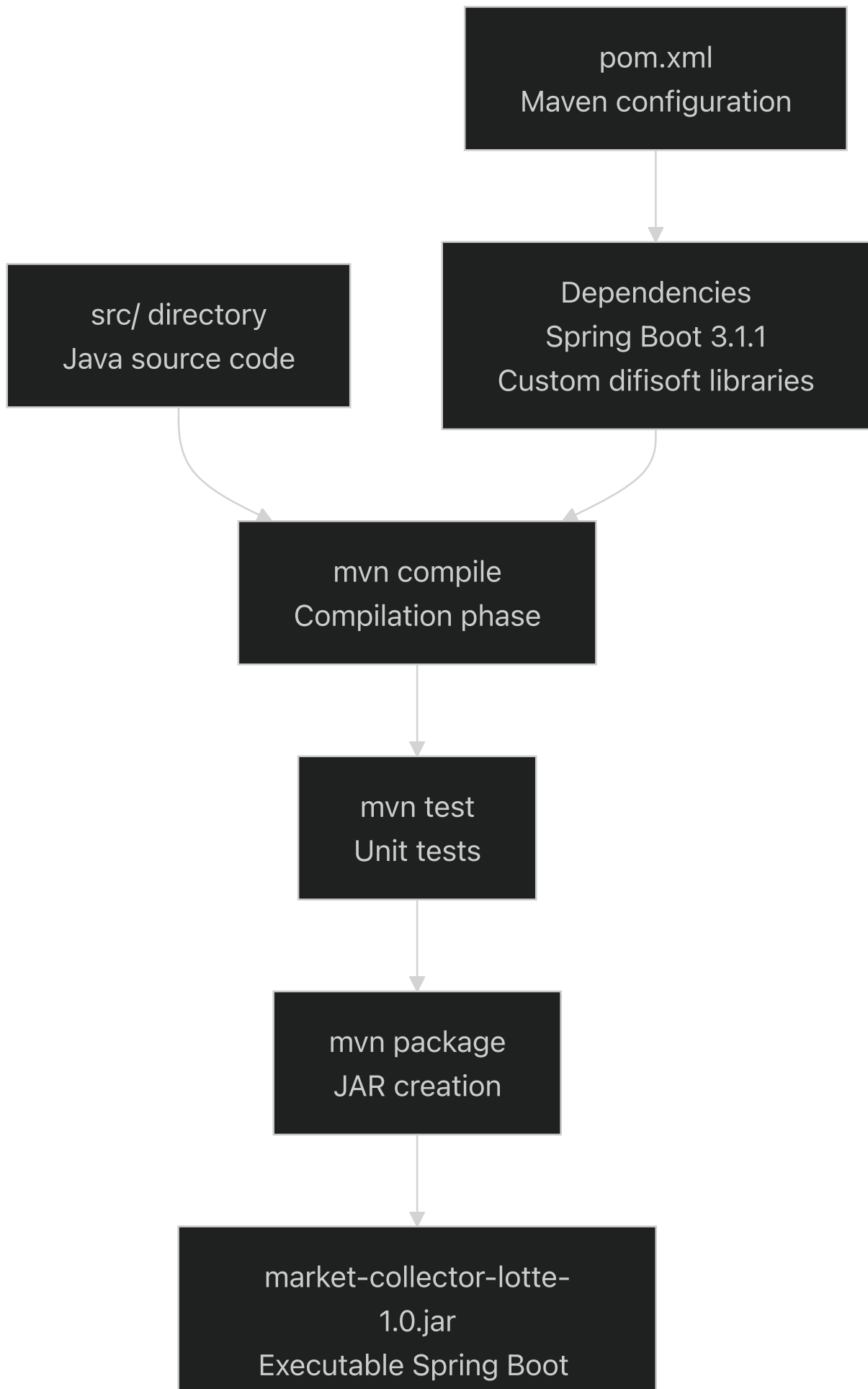
Sources: (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml#L21-L21>)

[pom.xml#L21](#) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml#L21-L21>)

dockerfile2-20](<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/dockerfile#L2-L20>  
(<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/dockerfile#L2-L20>))

## **Build Process**

### **Build Workflow**



## JAR

Sources: [\\_\(\(https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml#L7-L8\)\)](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml#L7-L8)

[pom.xml#L7-8](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml#L7-L8) [\(\(https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml#L7-L8\)\)](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml#L7-L8)

[pom.xml#L153-163](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml#L153-L163) [\(\(https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml#L153-L163\)\)](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml#L153-L163)

### Maven Build Commands

Build the application using Maven with the following commands:

```
mvn dependency:go-offline -B

mvn clean package -DskipTests

mvn clean package
```

The build process creates `target/market-collector-lotte-1.0.jar` as specified in [\\_\(\(https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml#L7-L8\)\)](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml#L7-L8)

[pom.xml#L7-8](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml#L7-L8) [\(\(https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml#L7-L8\)\)](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml#L7-L8)

### Key Dependencies:

- `spring-boot-starter-data-mongodb` for database operations
- `spring-boot-starter-data-redis` for caching
- `kafka-clients` for message distribution
- `hts-connection` for HTS system integration
- `minio` for object storage operations

Sources: [\\_\(\(https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml#L24-L151\)\)](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml#L24-L151)

[pom.xml#L24-151](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml#L24-L151) [\(\(https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml#L24-L151\)\)](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml#L24-L151)

[dockerfile#L10-17](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/dockerfile#L10-L17) [\(\(https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/dockerfile#L10-L17\)\)](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/dockerfile#L10-L17)

### Local Development Setup

#### Environment Configuration

Create local environment configuration following the structure expected by `entrypoint.sh` :

```
mkdir -p /appEnv/services/deployment-resources

touch /appEnv/app_env.sh

touch /appEnv/services/market-collector-lotte.sh
touch /appEnv/services/market-collector-lotte-dev.sh
```

## Running Locally

```
export TRADEX_WORKING_DIR=$(pwd)

java -jar target/market-collector-lotte-1.0.jar \
  --spring.profiles.active=dev \
  --server.port=8080
```

Sources: [\\_\(https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/entrypoint.sh#L35-L35\)](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/entrypoint.sh#L35-L35).

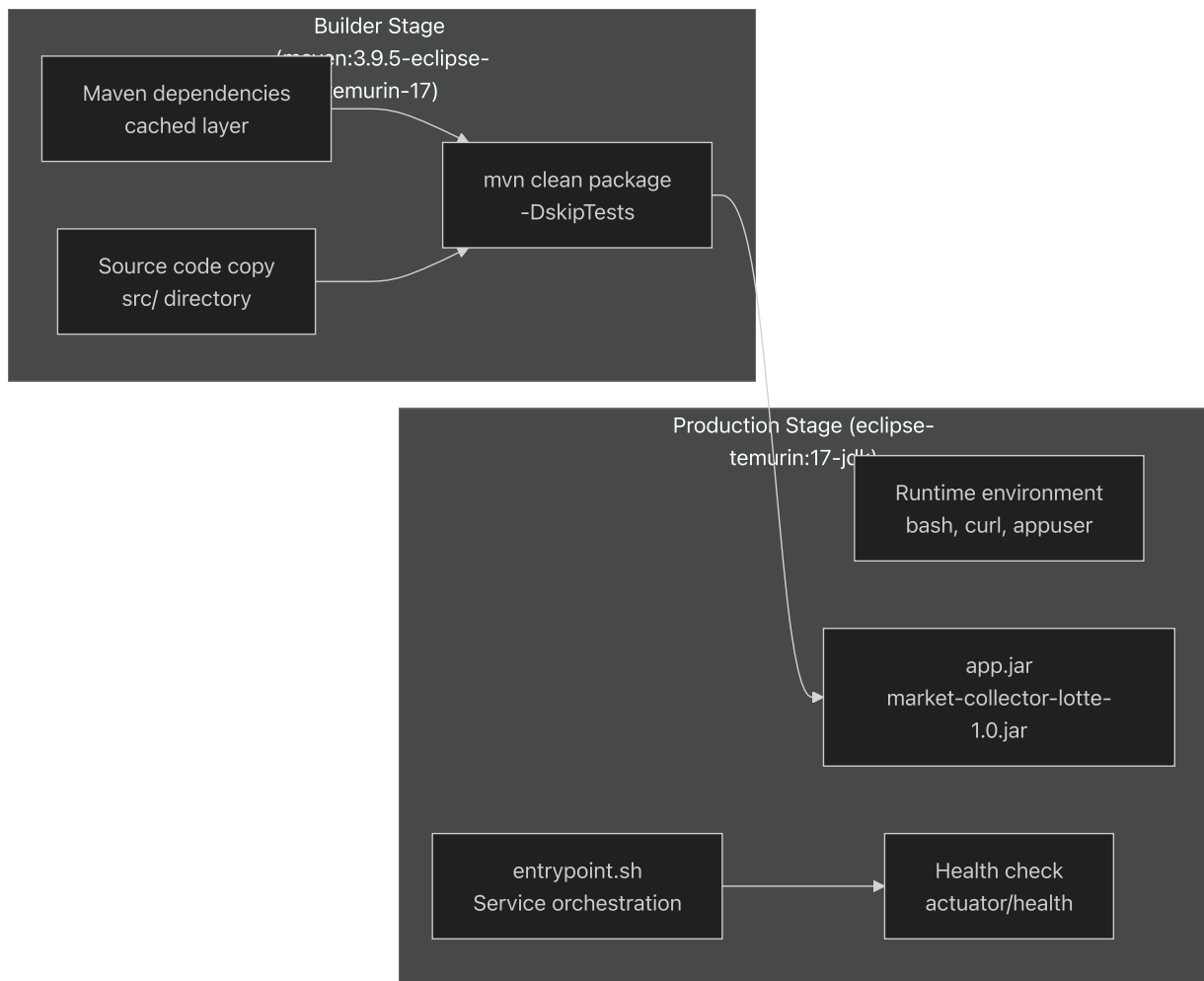
[entrypoint.sh35](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/entrypoint.sh#L35-L35) [\\_\(https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/entrypoint.sh#L35-L35\)](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/entrypoint.sh#L35-L35)[

[dockerfile54](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/dockerfile#L54-L54)](<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/dockerfile#L54-L54>

[\\_\(https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/dockerfile#L54-L54\)](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/dockerfile#L54-L54))

## Docker Deployment

### Container Build Process



Sources: <https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/dockerfile#L1-L62>

[dockerfile1-62](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/dockerfile#L1-L62) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/dockerfile#L1-L62>)

### Building the Container

```
docker build -t market-collector-lotte:1.0 .
```

```
docker run -d \
  --name market-collector-lotte \
  -p 8080:8080 \
  -v /path/to/appEnv:/appEnv \
  market-collector-lotte:1.0 \
  -s market-collector-lotte \
  -e production
```

The container uses <https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/entrypoint.sh>

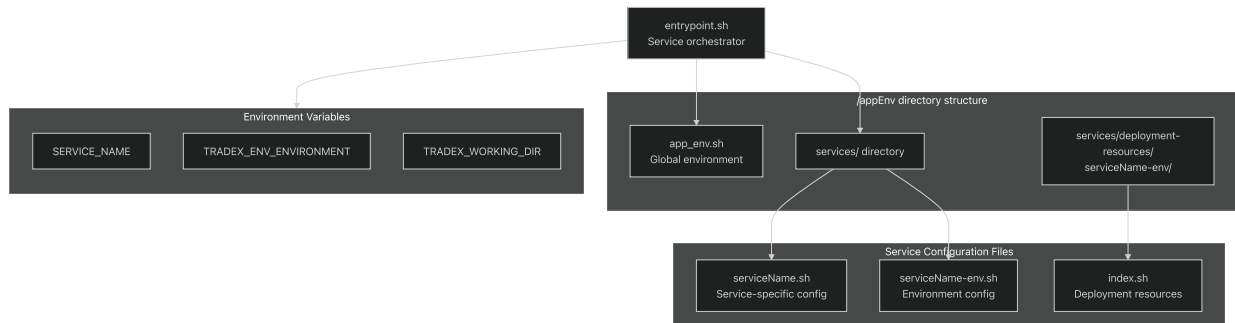
[entrypoint.sh](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/entrypoint.sh) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/entrypoint.sh>) for service orchestration and environment management.

Sources: <https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/dockerfile#L35-L61>

[dockerfile35-61](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/dockerfile#L35-L61) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/dockerfile#L35-L61>).

## Environment Configuration System

### Configuration Hierarchy



Sources: (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/entrypoint.sh#L4-L90>).

[entrypoint.sh4-90](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/entrypoint.sh#L4-L90) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/entrypoint.sh#L4-L90>).

### Configuration Script Execution Order

#### 1. Global Environment:[

entrypoint.sh38-41](<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/entrypoint.sh#L38-L41>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/entrypoint.sh#L38-L41>)) executes `/appEnv/app_env.sh`

#### 2. Service Configuration:[

entrypoint.sh44-48](<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/entrypoint.sh#L44-L48>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/entrypoint.sh#L44-L48>)) executes `services/$serviceName.sh`

#### 3. Environment-Specific:[

entrypoint.sh50-54](<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/entrypoint.sh#L50-L54>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/entrypoint.sh#L50-L54>)) executes `services/$serviceName-$env.sh`

#### 4. Optional Environment:[

entrypoint.sh58-62](<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/entrypoint.sh#L58-L62>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/entrypoint.sh#L58-L62>)) executes `env.sh` if present

#### 5. Deployment Resources:[

entrypoint.sh65-74](<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/entrypoint.sh#L65-L74>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/entrypoint.sh#L65-L74>)) executes deployment-specific scripts

### Required Environment Variables

| Variable                                                                                                                                                                                                                                                                                                                                                                                                                   | Purpose            | Set By |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------|--------|
| <code>SERVICE_NAME</code>                                                                                                                                                                                                                                                                                                                                                                                                  | Service identifier | [      |
| entrypoint.sh84]( <a href="https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/entrypoint.sh#L84-L84">https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/entrypoint.sh#L84-L84</a> ( <a href="https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/entrypoint.sh#L84-L84">https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/entrypoint.sh#L84-L84</a> )) |                    |        |
| <code>TRADEX_ENV_ENVIRONMENT</code>                                                                                                                                                                                                                                                                                                                                                                                        | Environment name   | [      |

entrypoint.sh85-87](<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/entrypoint.sh#L85-L87> (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/entrypoint.sh#L85-L87>)) | | `TRADEX_WORKING_DIR` | Application working directory | [

entrypoint.sh35](<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/entrypoint.sh#L35-L35> (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/entrypoint.sh#L35-L35>)) |

Sources: (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/entrypoint.sh#L7-L87>)

[entrypoint.sh7-87](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/entrypoint.sh#L7-L87) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/entrypoint.sh#L7-L87>)

## Verification and Health Checks

### Application Health Check

The Docker container includes automatic health monitoring:

```
curl -f http://localhost:8080/actuator/health
```

### Service Verification

```
docker logs market-collector-lotte
```

```
docker exec market-collector-lotte env | grep TRADEX
```

```
docker exec market-collector-lotte ps aux | grep java
```

The health check configuration is defined in (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/dockerfile#L57-L58>).

[dockerfile57-58](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/dockerfile#L57-L58) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/dockerfile#L57-L58>) with a 30-second interval and 60-second startup period.

Sources: (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/dockerfile#L57-L58>)

[dockerfile57-58](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/dockerfile#L57-L58) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/dockerfile#L57-L58>)

entrypoint.sh89-90) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/entrypoint.sh#L89-L90>)  
(<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/entrypoint.sh#L89-L90>))

## Build Configuration

Relevant source files

- [  
pom.xml](<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml>  
(<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml>))

This document covers the Maven build configuration for the Market Collector Lotte application, including dependency management, build plugins, and repository configuration. For information about containerization and deployment, see [Containerization & Deployment](#). For development environment setup, see [Development Environment](#).

### Overview

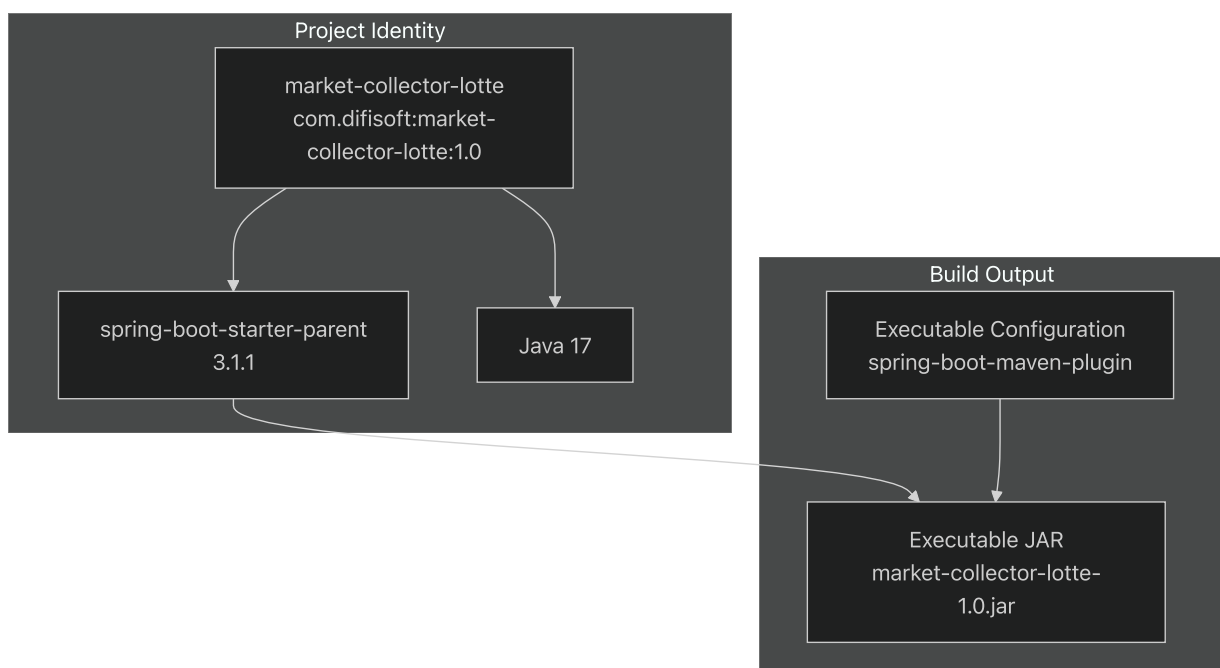
The Market Collector Lotte uses Maven as its build system with a Spring Boot-based configuration. The project is configured as a standalone JAR application that integrates with multiple external systems including HTS trading systems, MongoDB, Redis, Kafka, and Lotte Securities APIs.

Sources: (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml#L1-L185>)

[pom.xml1-185](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml#L1-L185) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml#L1-L185>)

### Project Structure

The Maven project follows standard Spring Boot conventions with custom organizational dependencies from Difisoft.



Sources: <https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml#L6-L22>

[pom.xml#L6-22](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml#L6-L22) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml#L6-L22>)

[pom.xml#L153-163](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml#L153-L163) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml#L153-L163>)  
(<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml#L153-L163>)

## Dependency Categories

The application dependencies are organized into several functional categories that support the market data collection workflow.

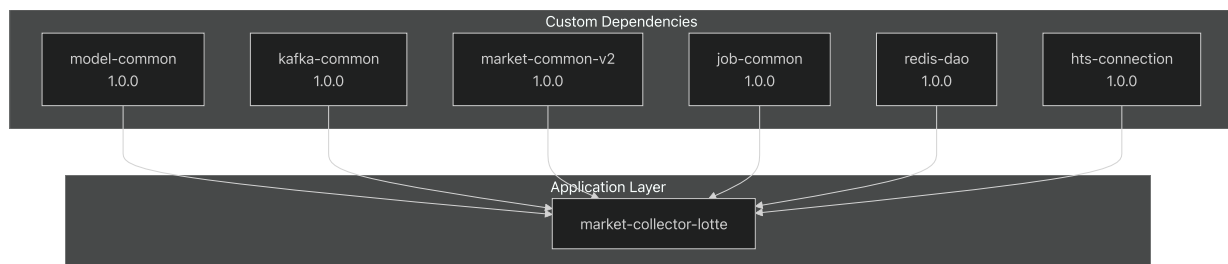
### Core Spring Boot Dependencies

| Dependency                                    | Purpose                        | Version   |
|-----------------------------------------------|--------------------------------|-----------|
| <code>spring-boot-starter</code>              | Core Spring Boot functionality | Inherited |
| <code>spring-boot-starter-data-redis</code>   | Redis integration              | Inherited |
| <code>spring-boot-starter-data-mongodb</code> | MongoDB integration            | Inherited |
| <code>spring-boot-starter-websocket</code>    | WebSocket support              | Inherited |

### External System Integrations

| Dependency                   | Purpose                     | Version  |
|------------------------------|-----------------------------|----------|
| <code>aws-java-sdk-s3</code> | AWS S3/MinIO object storage | 1.11.311 |
| <code>minio</code>           | MinIO client library        | 8.3.7    |
| <code>kafka_2.12</code>      | Kafka messaging broker      | 2.3.1    |
| <code>kafka-clients</code>   | Kafka client API            | 2.3.1    |

### Custom Difisoft Libraries



Sources: <https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml#L69-L102>

[pom.xml#L69-102](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml#L69-L102) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml#L69-L102>)

pom.xml105-114](<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml#L105-L114>)[  
([%5B](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml#L105-L114))

pom.xml117-120](<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml#L117-L120>)[  
([%5B](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml#L117-L120))

### Data Processing Libraries

| Dependency            | Purpose                      | Version   |
|-----------------------|------------------------------|-----------|
| lombok                | Code generation annotations  | Inherited |
| jackson-datatype-joda | Joda time JSON serialization | Inherited |
| commons-io            | File I/O utilities           | 2.6       |
| java-jwt              | JWT token processing         | 4.4.0     |
| reflections           | Runtime reflection utilities | 0.9.11    |
| kotlin-stdlib         | Kotlin standard library      | 1.9.0     |

Sources: (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml#L34-L39>)

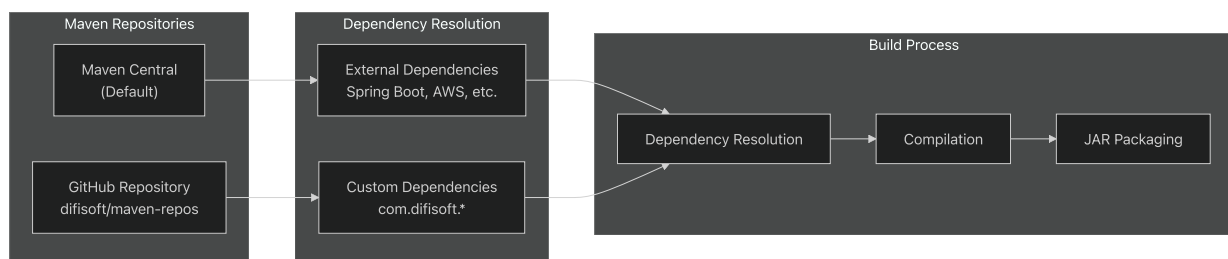
[pom.xml34-39](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml#L34-L39) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml#L34-L39>)

pom.xml46-60](<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml#L46-L60>)[  
([%5B](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml#L46-L60))

pom.xml122-137](<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml#L122-L137>)[  
([%5B](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml#L122-L137))

### Repository Configuration

The build system is configured to access custom Difisoft libraries from a GitHub-hosted Maven repository.



The custom repository configuration includes:

- **URL:** `https://raw.githubusercontent.com/difisoft/maven-repos/main`
- **Snapshots:** Enabled with always update policy
- **Releases:** Enabled with always update policy

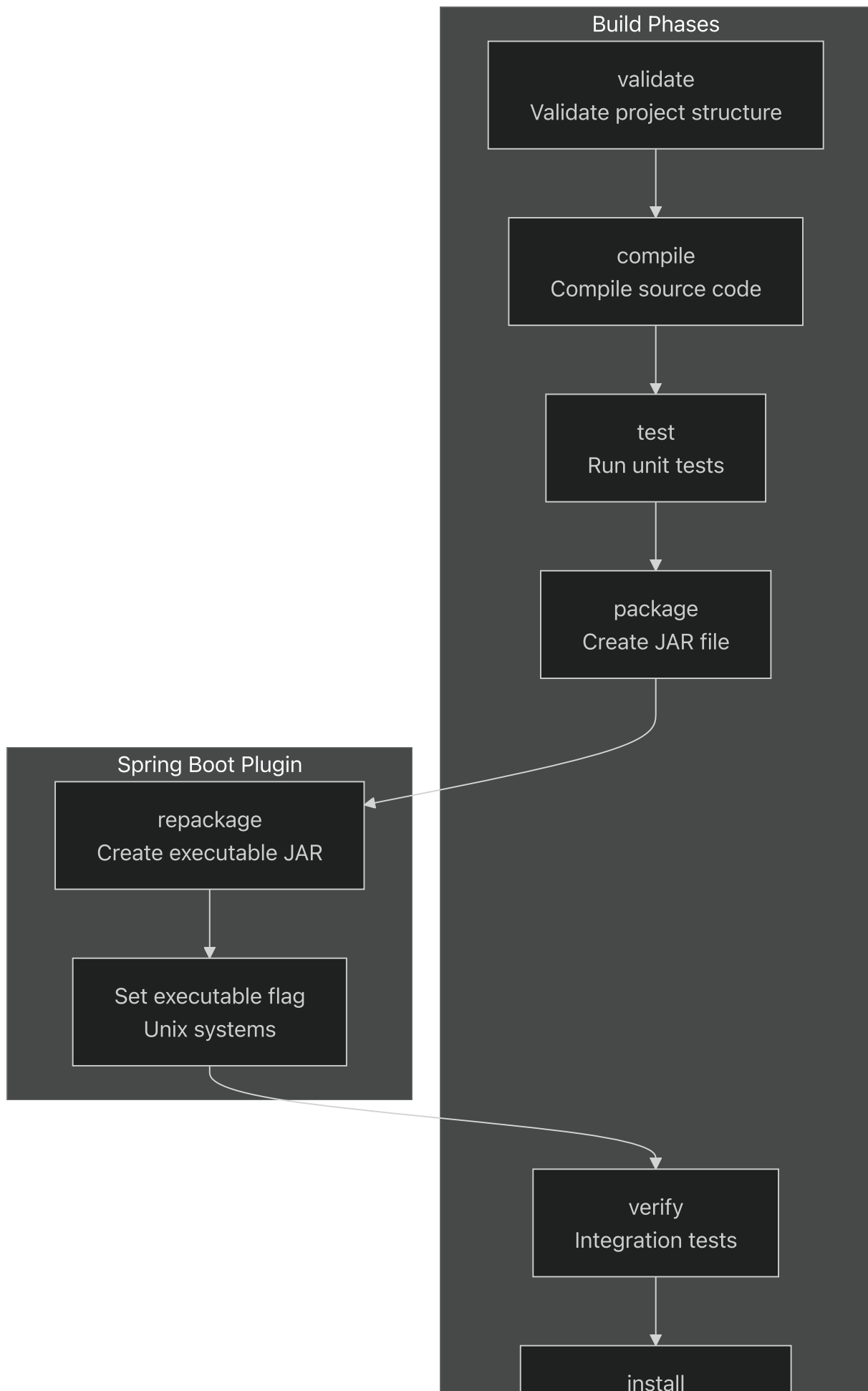
Sources: (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml#L165-L178>)

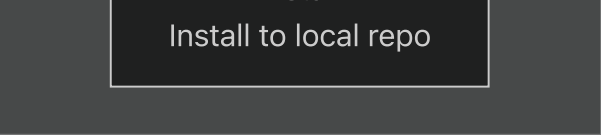
[pom.xml#L165-L178](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml#L165-L178) (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml#L165-L178).

## **Build Process**

The Spring Boot Maven plugin is configured to generate an executable JAR file that can be run directly or deployed in containers.

## **Build Configuration**




 A dark gray rectangular button with a white border and the text "Install to local repo" in white.

### Plugin Configuration

The `spring-boot-maven-plugin` is configured with:

- **Executable:** `true` - Enables direct execution on Unix systems
- **Repackaging:** Automatic fat JAR creation with embedded dependencies

Sources: <https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml#L155-L161>

[pom.xml#L155-L161](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml#L155-L161) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml#L155-L161>)

### Dependency Version Management

The project leverages Spring Boot's dependency management to maintain consistent versions across Spring ecosystem components while explicitly defining versions for external libraries that require specific compatibility.

#### Version Strategy

| Component Type         | Version Management        | Rationale                  |
|------------------------|---------------------------|----------------------------|
| Spring Boot Components | Inherited from parent BOM | Ensures compatibility      |
| Kafka Libraries        | Explicitly set to 2.3.1   | HTS system compatibility   |
| AWS SDK                | Fixed at 1.11.311         | Stable MinIO integration   |
| Custom Libraries       | All at 1.0.0              | Internal version alignment |

Sources: <https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml#L14-L18>

[pom.xml#L14-18](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml#L14-L18) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml#L14-L18>)

[pom.xml#L105-114](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml#L105-L114) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml#L105-L114>)  
(<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml#L105-L114>)%5B

[pom.xml#L63-66](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml#L63-L66) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml#L63-L66>)  
(<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml#L63-L66>)

## Containerization & Deployment

Relevant source files

- [[dockerfile](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/dockerfile)] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/dockerfile>)  
(<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/dockerfile>)
- [

```
entrypoint.sh](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/entrypoint.sh  
.(https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/entrypoint.sh))
```

This page covers the Docker containerization setup and deployment orchestration for the Market Collector Lotte system. It details the multi-stage build process, configuration script hierarchy, and runtime service orchestration through the entrypoint system.

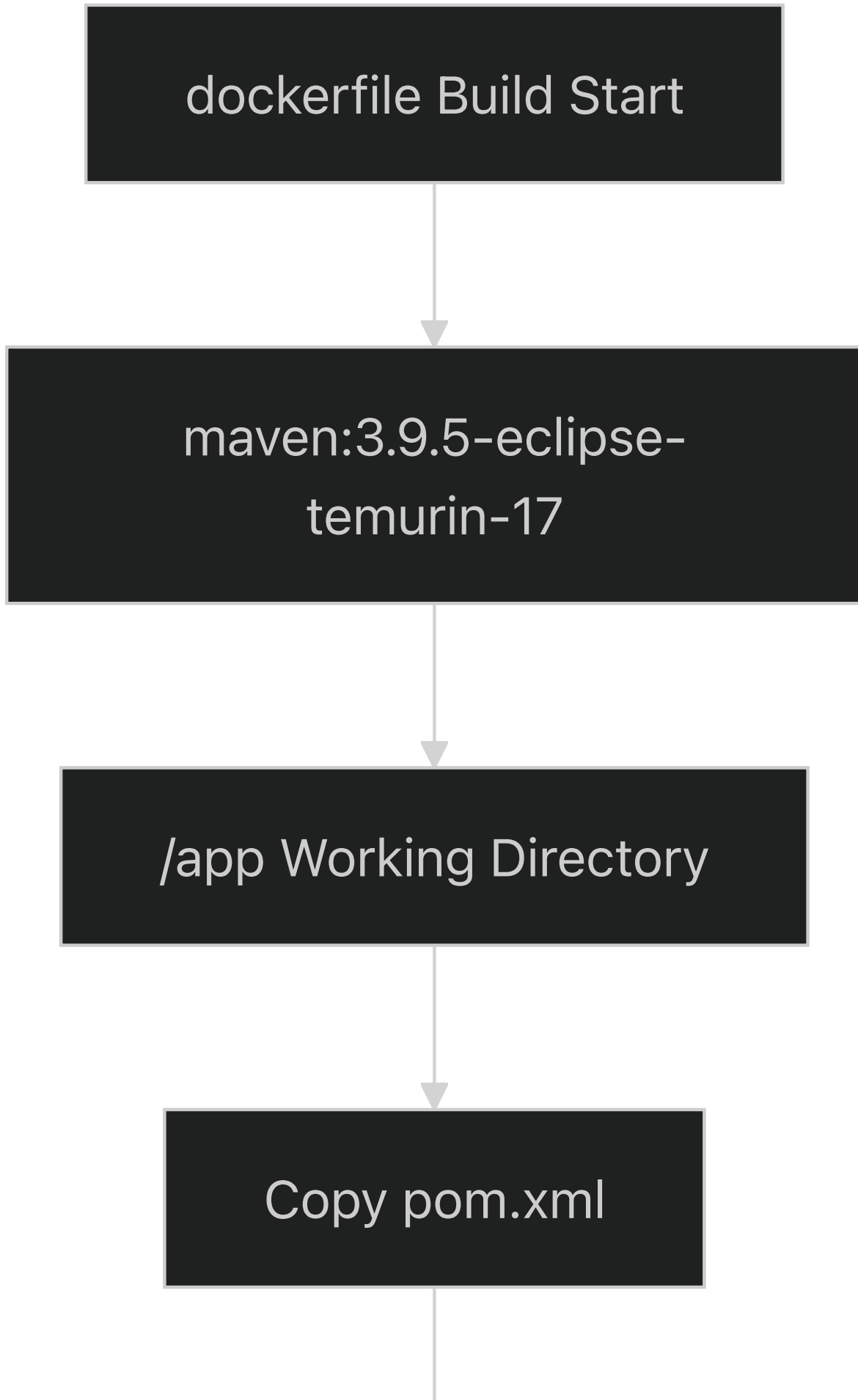
For information about Maven build configuration and dependencies, see [Build Configuration](#). For development environment setup, see [Development Environment](#).

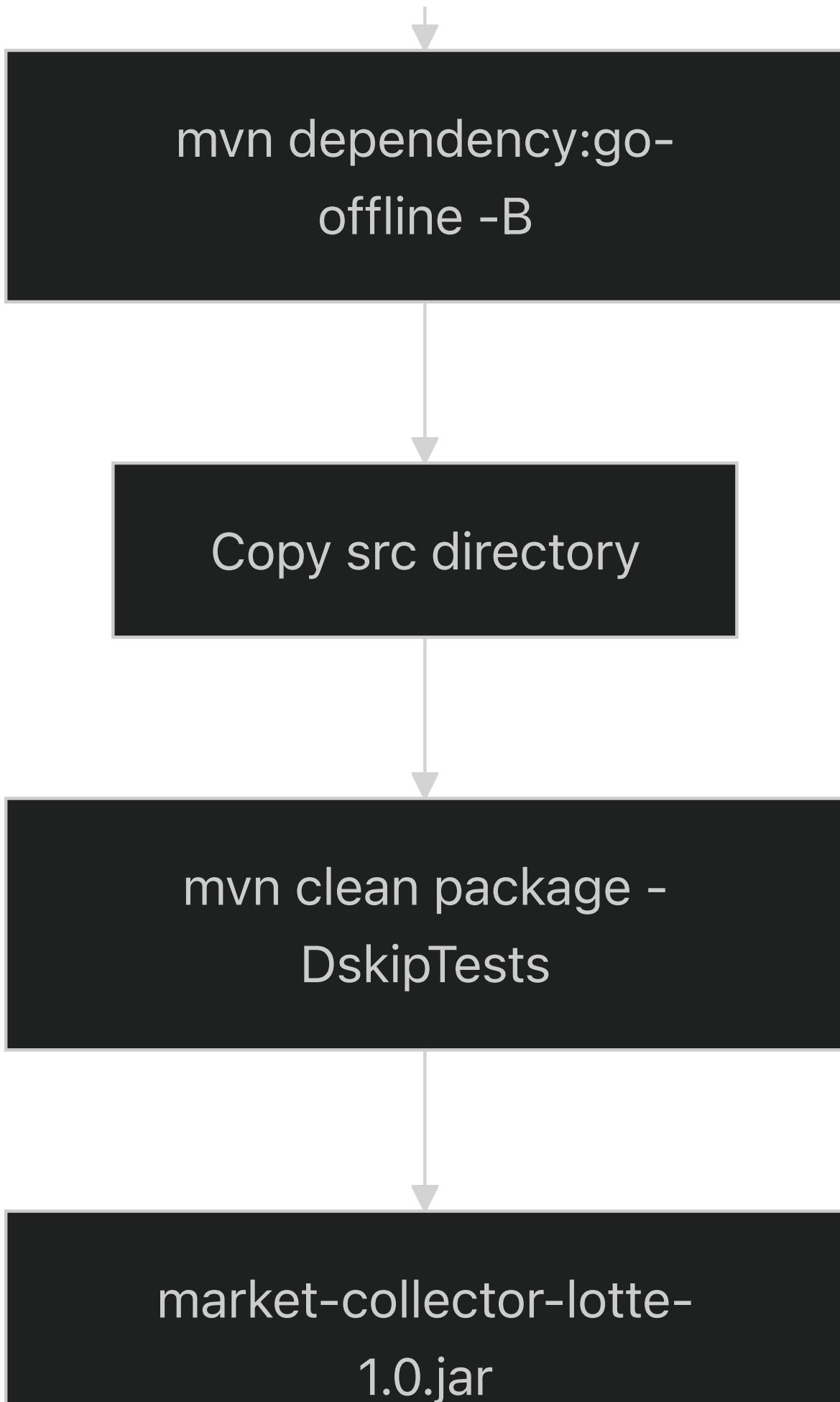
## Docker Multi-Stage Build Process

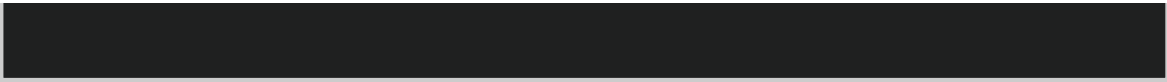
The Market Collector Lotte application uses a multi-stage Docker build to optimize image size and build efficiency. The build process consists of two distinct stages: a Maven-based builder stage and a production runtime stage.

### Builder Stage

The first stage uses `maven:3.9.5-eclipse-temurin-17` as the base image and handles compilation and packaging:







The build process is optimized for Docker layer caching by copying `pom.xml` first and downloading dependencies before copying source code\_(<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/dockerfile#L8-L11>).

[dockerfile8-11](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/dockerfile#L8-L11) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/dockerfile#L8-L11>) This ensures that dependency downloads are cached unless the Maven configuration changes.

Sources:\_(<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/dockerfile#L1-L17>).

[dockerfile1-17](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/dockerfile#L1-L17) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/dockerfile#L1-L17>).

### Production Stage

The production stage creates a secure runtime environment using `eclipse-temurin:17-jdk` :

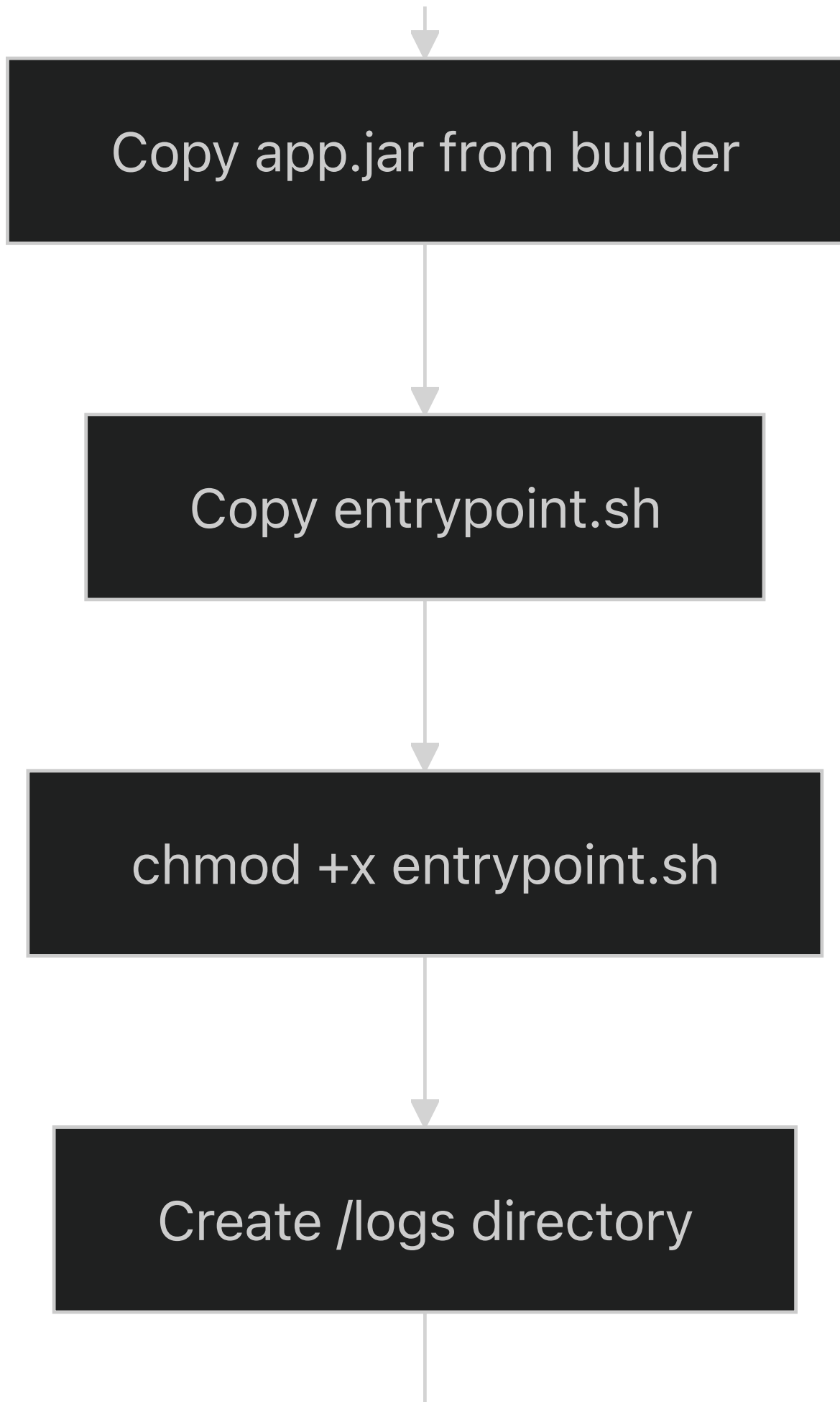
eclipse-temurin:17-jdk

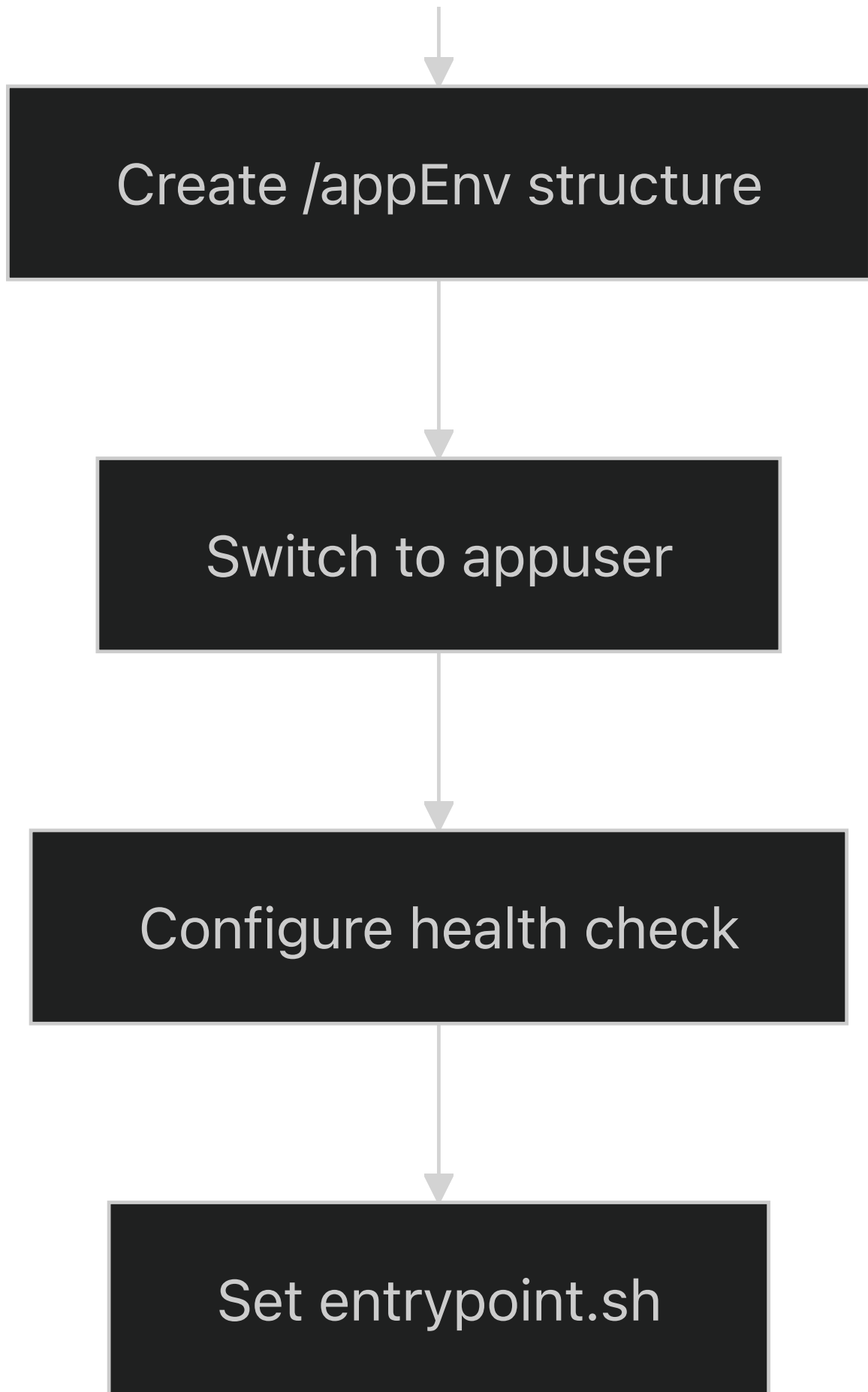
```
graph TD; A[eclipse-temurin:17-jdk] --> B[Install bash + curl]; B --> C[Create appuser]; C --> D[/app Working Directory];
```

Install bash + curl

Create appuser

/app Working Directory





The production stage implements several security best practices:

- Creates a dedicated `appuser` instead of running as root[

dockerfile29](<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/dockerfile#L29-L29>)  
(<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/dockerfile#L29-L29>))

- Sets appropriate ownership for application directories[

dockerfile44-48](<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/dockerfile#L44-L48>)  
(<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/dockerfile#L44-L48>))

- Exposes only the necessary port 8080[

dockerfile54](<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/dockerfile#L54-L54>)  
(<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/dockerfile#L54-L54>))

Sources: (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/dockerfile#L19-L62>).

[dockerfile19-62](#) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/dockerfile#L19-L62>).

## Entrypoint Orchestration System

The `entrypoint.sh` script provides a sophisticated configuration management system that loads environment-specific settings and service configurations in a hierarchical manner.

### Command Line Interface

The entrypoint script accepts command line arguments to specify the service name and environment:

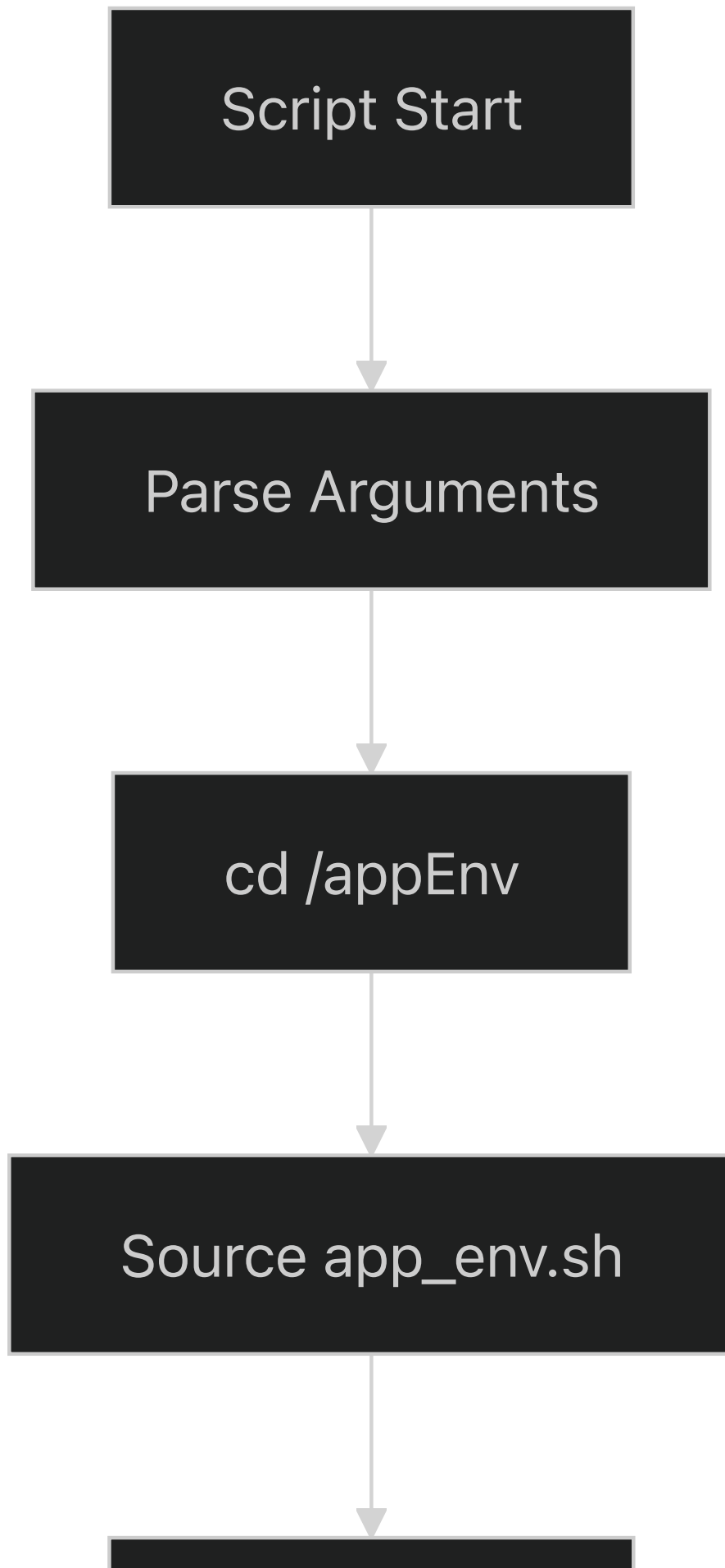
| Parameter                  | Short Form      | Description                                   |
|----------------------------|-----------------|-----------------------------------------------|
| <code>--serviceName</code> | <code>-s</code> | Name of the service being deployed            |
| <code>--env</code>         | <code>-e</code> | Target environment (dev, staging, prod, etc.) |

Sources: (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/entrypoint.sh#L7-L30>).

[entrypoint.sh7-30](#) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/entrypoint.sh#L7-L30>).

### Configuration Loading Hierarchy

The entrypoint script loads configuration files in a specific order to allow for environment and service-specific overrides:



```
cd services
```

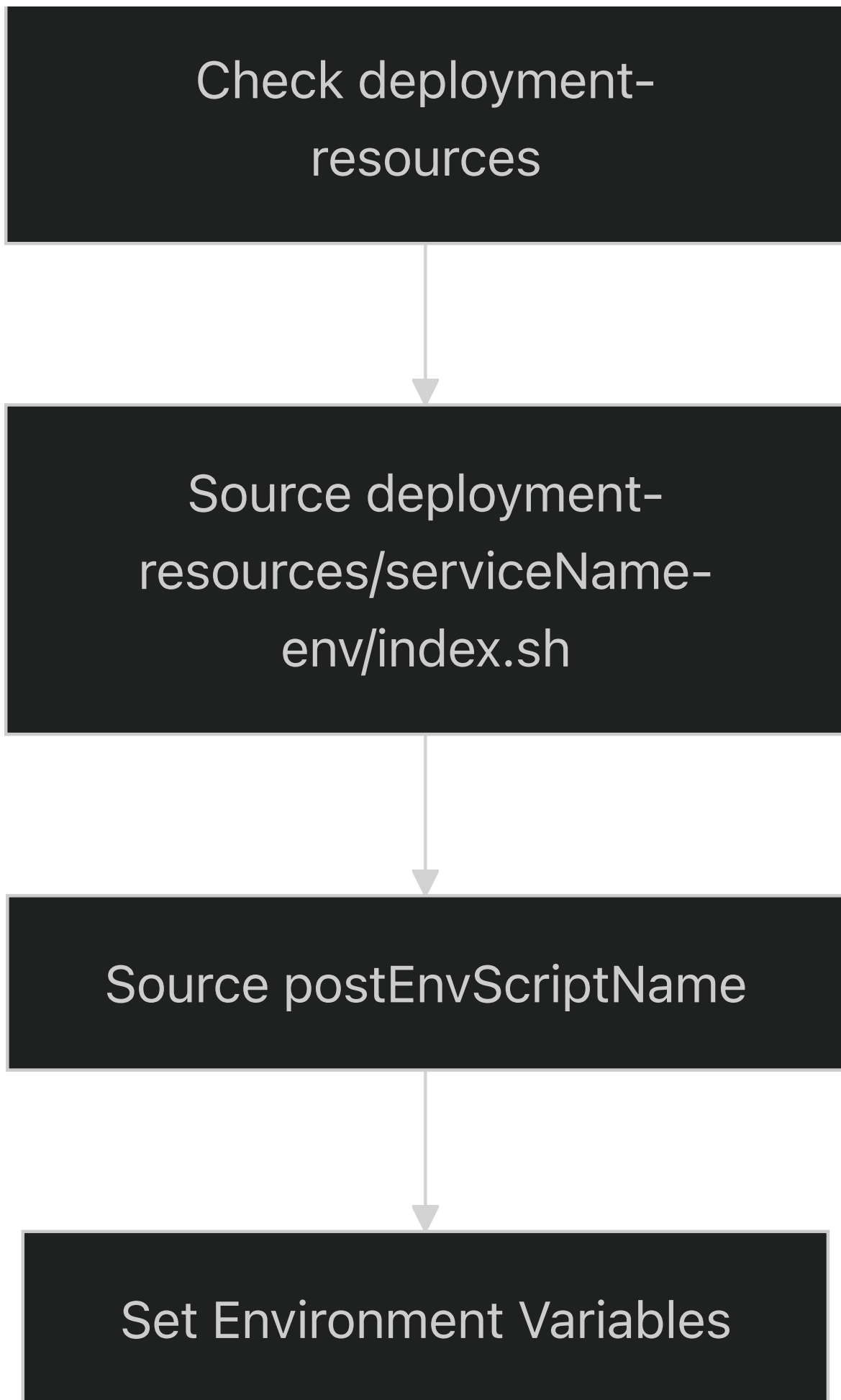
```
graph TD; A[cd services] --> B[Source serviceName.sh]; B --> C[Source serviceName-env.sh]; C --> D[cd .. && Source env.sh]; D --> E[ ];
```

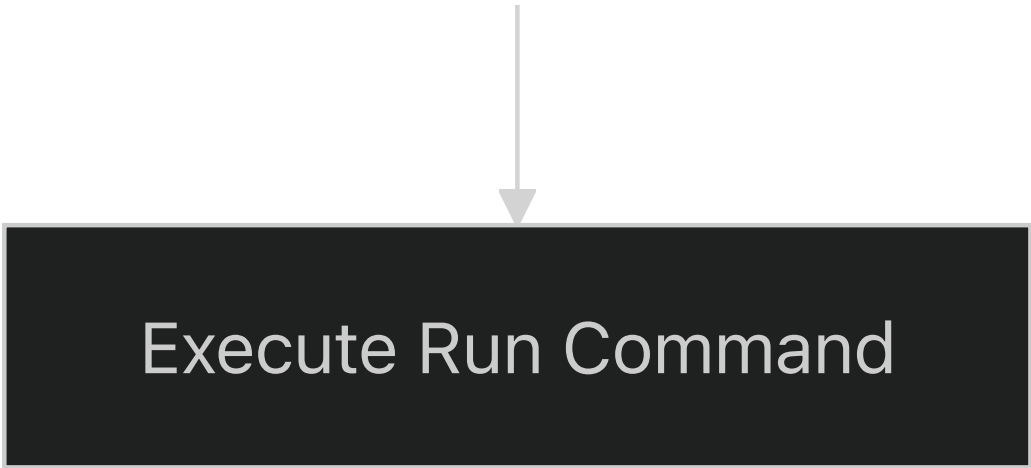
```
Source serviceName.sh
```

```
Source serviceName-  
env.sh
```

```
cd .. && Source env.sh
```

---





# Execute Run Command

The configuration hierarchy allows for flexible deployment scenarios:

1. **Global Configuration:** `/appEnv/app_env.sh` - System-wide environment settings[

entrypoint.sh38-41](<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/entrypoint.sh#L38-L41> (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/entrypoint.sh#L38-L41>))

2. **Service Configuration:** `/appEnv/services/$serviceName.sh` - Service-specific settings[

entrypoint.sh44-48](<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/entrypoint.sh#L44-L48> (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/entrypoint.sh#L44-L48>))

3. **Environment Configuration:** `/appEnv/services/$serviceName-$env.sh` - Environment-specific overrides[

entrypoint.sh50-54](<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/entrypoint.sh#L50-L54> (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/entrypoint.sh#L50-L54>))

4. **Deployment Resources:** `/appEnv/services/deployment-resources/$serviceName-$env/index.sh` - Deployment-specific resources[

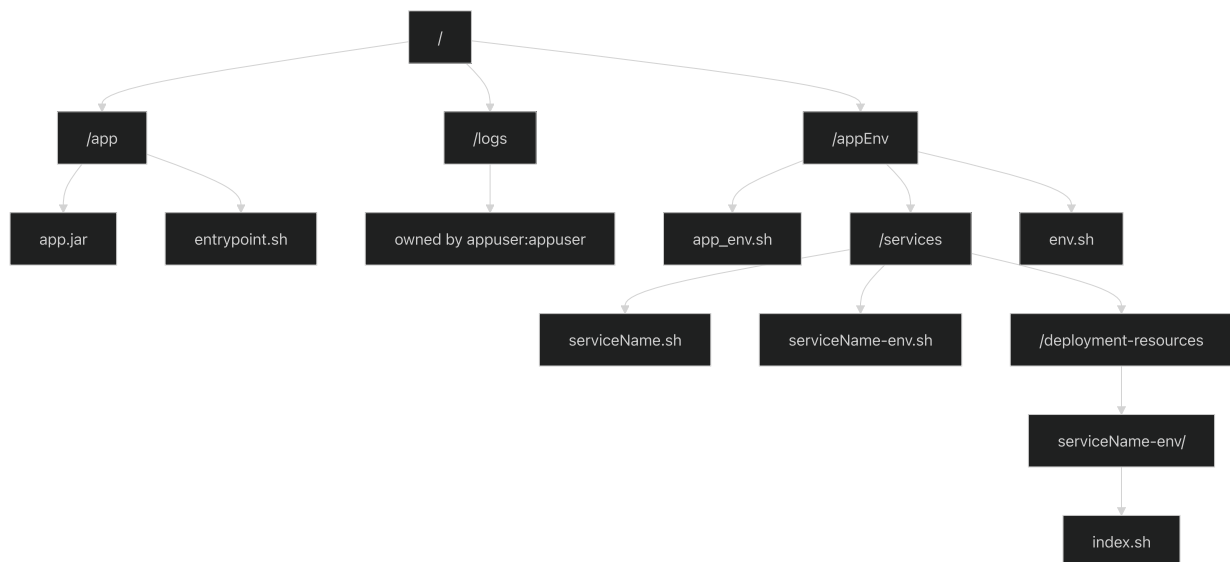
entrypoint.sh65-74](<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/entrypoint.sh#L65-L74> (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/entrypoint.sh#L65-L74>))

Sources: (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/entrypoint.sh#L32-L90>).

[entrypoint.sh32-90](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/entrypoint.sh#L32-L90) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/entrypoint.sh#L32-L90>).

## Directory Structure and Security

The containerization setup creates a well-organized directory structure with appropriate security boundaries:



## Security Configuration

The Docker setup implements security best practices:

- **Non-root execution:** Application runs as `appuser` [dockerfile29-51](<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/dockerfile#L29-L51> (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/dockerfile#L29-L51>))
- **Minimal packages:** Only installs necessary runtime dependencies [dockerfile23-26](<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/dockerfile#L23-L26> (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/dockerfile#L23-L26>))
- **Health monitoring:** Configured health check endpoint [dockerfile57-58](<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/dockerfile#L57-L58> (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/dockerfile#L57-L58>))
- **Working directory isolation:** Application files contained in `/app` [dockerfile32](<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/dockerfile#L32-L32> (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/dockerfile#L32-L32>))

Sources: (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/dockerfile#L23-L62>).

[dockerfile23-62](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/dockerfile#L23-L62) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/dockerfile#L23-L62>).

## Environment Variable Management

The entrypoint script sets several key environment variables that are used throughout the application:

| Variable                                                                                                                                                                                                                                                                                                                                                                                                                   | Source                | Purpose                    |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------|----------------------------|
| TRADEX_WORKING_DIR                                                                                                                                                                                                                                                                                                                                                                                                         | Script calculated     | Current working directory[ |
| entrypoint.sh35]( <a href="https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/entrypoint.sh#L35-L35">https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/entrypoint.sh#L35-L35</a> ( <a href="https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/entrypoint.sh#L35-L35">https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/entrypoint.sh#L35-L35</a> )) |                       |                            |
| SERVICE_NAME                                                                                                                                                                                                                                                                                                                                                                                                               | Command line argument | Service identifier[        |

entrypoint.sh84](<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/entrypoint.sh#L84-L84> (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/entrypoint.sh#L84-L84>)) | | TRADEX\_ENV\_ENVIRONMENT | Command line argument | Environment identifier[

entrypoint.sh85-87](<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/entrypoint.sh#L85-L87> (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/entrypoint.sh#L85-L87>)) |

These environment variables are consumed by the Spring Boot application configuration system described in [Configuration Management](#).

Sources: (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/entrypoint.sh#L35-L87>).

[entrypoint.sh35-87](#) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/entrypoint.sh#L35-L87>).

## Health Check Configuration

The Docker container includes a comprehensive health check configuration that monitors application status:

```
HEALTHCHECK --interval=30s --timeout=10s --start-period=60s --retries=3 \
  CMD curl -f http://localhost:8080/actuator/health
  | | exit 1 |
|---| ---|
```

The health check configuration:

- **\*\*Interval\*\***: Checks every 30 seconds during normal operation
- **\*\*Timeout\*\***: Each check times out after 10 seconds
- **\*\*Start Period\*\***: Allows 60 seconds for application startup before first check
- **\*\*Retries\*\***: Considers container unhealthy after 3 consecutive failures
- **\*\*Endpoint\*\***: Uses Spring Boot Actuator health endpoint

Sources:[] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/dockerfile#L57-58>)

[dockerfile57-58] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/dockerfile#L57-58>)

---

### ### Development Environment

Relevant source files

- [ [.gitignore](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/.gitignore)] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/.gitignore>)
- [ [README.md](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/README.md)] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/README.md>)

This document covers the development environment setup for the Market Collector Lotte system.

For information about build configuration and Maven dependencies, see [Build Configuration].

### #### Development Prerequisites

The Market Collector Lotte application requires specific development tools and runtime dependencies.

### ##### Required Development Tools

| Tool                | Version          | Purpose                                    | Notes                                                                                                                                                                                                                                                                                                               |
|---------------------|------------------|--------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>**Java JDK**</b> | 17+              | Runtime and compilation                    | Spring Boot 3.1.1 requires Java 17 minimum                                                                                                                                                                                                                                                                          |
| <b>**Maven**</b>    | 3.6+             | Build automation and dependency management | Configured in[ <a href="https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml">pom.xml</a> ] ( <a href="https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml">https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/pom.xml</a> )                        |
| <b>**Docker**</b>   | Latest stable    | Container runtime for local services       | Required for MongoDB                                                                                                                                                                                                                                                                                                |
| <b>**Git**</b>      | 2.x+             | Version control                            | Configured exclusions in[ <a href="https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/.gitignore">.gitignore</a> ] ( <a href="https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/.gitignore">https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/.gitignore</a> ) |
| <b>**GaalVM**</b>   | 22.3+ (Optional) | Native image compilation                   | For <code>-Pnative</code> build profile                                                                                                                                                                                                                                                                             |

### ##### External System Dependencies

| System                    | Version        | Purpose                                        | Configuration                           |
|---------------------------|----------------|------------------------------------------------|-----------------------------------------|
| ***MongoDB**              | 4.4+           | Persistent data storage                        | Connection string in `application.yaml` |
| ***Redis**                | 6.0+           | Cache layer and real-time data                 | Host/port in `application.yaml`         |
| ***Apache Kafka**         | 2.3+           | Event streaming and message distribution       | Broker URLs in `ap                      |
| ***MinIO/S3**             | Compatible API | File storage for symbol lists and market files | Credent                                 |
| ***HTS System**           | N/A            | Real-time market data feed source              | Requires `HTS_USER` and `HTS_           |
| ***Lotte Securities API** | N/A            | Historical data and symbol information         | Base URL and                            |

! [Chart 1] (market-collector-lotte-images/market-collector-lotte-section4-1.svg)

**\*\*Sources:\*\*** [] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/README.r>

[README.md58-66] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/README>)

README.md25-33] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/README.md>)

The project supports multiple IDEs (IntelliJ IDEA, VS Code, Eclipse) and maintains clean ve

The[] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/.gitignore>)

[.gitignore] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/.gitignore>)

```
| Pattern | Category | Purpose |
|---|---|---|
| `.vscode/**` | IDE Configuration | VS Code workspace settings and extensions |
| `.idea/**` | IDE Configuration | IntelliJ IDEA project configuration |
| `*.iml` | IDE Configuration | IntelliJ IDEA module definition files |
| `.project`, `.classpath`, `.factorypath` | IDE Configuration | Eclipse project metadata |
| `.settings/**` | IDE Configuration | Eclipse settings directory |
| `target/**` | Build Artifacts | Maven build output directory |
| `*.log*` | Runtime Artifacts | Application log files |
| `*.tbl` | Data Files | Temporary table data files |
| `master*` | Data Files | Master data files (production market data) |
| `index-mapping-code*.json` | Generated Files | HTS exchange code mappings generated at
```

**Sources:** <https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/.gitignore>

[.gitignore1-12] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/.gitignore>)

! [Chart 2] (market-collector-lotte-images/market-collector-lotte-section4-2.svg)

**\*\*Sources:\*\*** [] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/.gitignore>)

```
[.gitignore1-12] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/.gitignore1-12)

##### IDE-Specific Setup

##### IntelliJ IDEA

-   **Project Type:** Maven-based Spring Boot application
-   **SDK:** Java 17 or higher
-   **Excluded Files:**

    .gitignore2] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/.gitignore2)

    .gitignore12] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/.gitignore12)
-   **Run Configuration:** Run `Application.java` with VM options for environment variables
-   **Spring Support:** Enable Spring Boot DevTools for hot reload

##### Visual Studio Code

-   **Extensions Recommended:**
    -   Java Extension Pack
    -   Spring Boot Extension Pack
    -   Docker Extension
-   **Excluded Files:**

    .gitignore1] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/.gitignore1)
-   **Launch Configuration:** Configure `launch.json` for debugging with environment variables
-   **Settings:** Workspace settings stored locally, not in version control

##### Eclipse

-   **Project Type:** Maven project with Spring facets
-   **Excluded Files:**

    .gitignore7-9] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/.gitignore7-9)
-   **Workspace:** Import as existing Maven project
-   **Spring Tools:** Install Spring Tools 4 for enhanced Spring Boot support

#### Project Structure and Codebase Organization

The project follows standard Spring Boot conventions with a clear package structure that scales well.

##### Source Code Organization

The Java source code is organized under `src/main/java/com/difisoft/marketcollector/`:

| Package | Purpose | Key Classes |
|---|---|---|
| `configurations/` | Spring configuration classes | `AppConf`, `BeanConf` |
| `constants/` | Market constants and enumerations | `Constants`, exchange mappings |
| `consumers/` | Kafka message consumers | Consumer implementations |
| `job/` | Scheduled job services | `JobService` |
| `model/` | Data models and entities | `SymbolInfo`, `QuoteUpdate`, DTOs |
| `repositories/` | Data access layer | MongoDB repository interfaces |
| `services/` | Business logic services | `RealTimeService`, `LotteApiService` |
```

```

| `socket/` | Socket connection handlers | Connection management |
| `utils/` | Utility classes | Helper functions and utilities |
| `ws/` | WebSocket services | `WsConnection`, `WsConnection2` |

#### Application Entry Point

![[Chart 3](market-collector-lotte-images/market-collector-lotte-section4-3.svg)]

**Sources:**[] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/README.r
[README.md175-189](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/REA

#### Build Artifacts and Exclusions

Maven generates various artifacts during the build process, all of which are excluded from

![[Chart 4](market-collector-lotte-images/market-collector-lotte-section4-4.svg)]

**Sources:**[] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/.gitigno
[.gitignore6](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/.gitigno
README.md94-101](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/README

#### Environment Configuration

The application requires environment-specific configuration through environment variables a

#### Required Environment Variables

The following environment variables must be set before running the application:

| Variable | Purpose | Example Value |
| --- | --- | --- |
| `TECHX_ENV` | Deployment environment identifier | `development`, `production` |
| `TRADEX_ENV_DOMAIN` | Service domain namespace | `tradex` |
| `TRADEX_ENV_NODE_ID` | Node identifier in cluster | `node1` |
| `TRADEX_ENV_INSTANCE_ID` | Instance identifier | `0` |
| `HTS_USER` | HTS trading system username | `your_hts_username` |
| `HTS_PASSWORD` | HTS trading system password | `your_hts_password` |

**Example Configuration:**

```

```

export TECHX_ENV=development
export TRADEX_ENV_DOMAIN=tradex
export TRADEX_ENV_NODE_ID=node1
export TRADEX_ENV_INSTANCE_ID=0
export HTS_USER=your_hts_username
export HTS_PASSWORD=your_hts_password

```

```

**Sources:**[] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/README.r
[README.md76-84] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/README

##### Application Configuration File

The `src/main/resources/application.yaml` file contains connection strings and operational

| Configuration Section | Key Settings | Purpose |
|---| --- | --- |
| `app.apiConnection` | `baseUrl`, `apiKey` | Lotte Securities API endpoints and authentic
| `app.marketConf.fileConfig.minio` | `baseUrl`, `accessKey`, `privateKey` | MinIO/S3 file
| `app.realtime.workingTime` | `from`, `to`, `weekDays` | Market operating hours (09:00-1
| `spring.data.mongodb` | `uri` | MongoDB connection string |
| `spring.redis` | `host`, `port` | Redis cache server connection |
| `spring.kafka` | `bootstrap-servers` | Kafka broker URLs |

**Sources:**[] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/README.r
[README.md86-93] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/README
README.md118-138] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/READ

##### Configuration Loading Architecture

![[Chart 5] (market-collector-lotte-images/market-collector-lotte-section4-5.svg)]

**Sources:**[] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/README.r
[README.md112-145] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/REA

#### Development Environment Validation

To verify the development environment is properly configured, developers should ensure:

1. **IDE Exclusions Active:** IDE-specific directories are not tracked by Git
2. **Build Artifacts Excluded:** Maven `target/` directory is ignored
3. **Runtime Data Excluded:** Log files and data tables are not committed
4. **External Dependencies Available:** Required services (MongoDB, Redis, Kafka) are acc

The `.gitignore` configuration ensures that only source code and configuration templates are

**Sources:**[] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/.gitignore
[.gitignore1-12] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/.gitig

---

## Core Architecture

### Overview

Relevant source files

```

```

- [
    src/main/java/com/difisoft/marketcollector/Application.java] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/Application.java)
- [
    src/main/java/com/difisoft/marketcollector/configurations/AppConf.java] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java)
- [
    src/main/java/com/difisoft/marketcollector/configurations/BeanConf.java] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/BeanConf.java)

```

#### #### Purpose and Scope

This document describes the foundational architecture of the Market Collector Lotte application.

For detailed information about specific architectural subsystems, see:

- Application initialization and component scanning: [Application Bootstrap] (#application-bootstrap)
- Configuration properties and system settings: [Configuration Management] (#configuration-management)
- Bean factory and conditional service instantiation: [Bean Configuration & Service Factory] (#bean-configuration-and-service-factory)
- Real-time data collection components: [Real-time Data Collection] (#real-time-data-collection)

\* \* \*

#### #### Spring Boot Application Foundation

The application is built on Spring Boot 2.3.x and leverages multiple Spring framework modules.

#### ##### Application Entry Point

The `Application` class serves as the main entry point and configuration hub for the entire application.

![[Chart 1](market-collector-lotte-images/market-collector-lotte-section5-1.svg)]

**Sources:** [1] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/Application.java#L16-21>)

[src/main/java/com/difisoft/marketcollector/Application.java#L16-21] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/Application.java#L16-21>)

The `Application` class uses the following Spring annotations to configure the application:

| Annotation                     | Purpose                                                                      | Configuration Target                                                      |
|--------------------------------|------------------------------------------------------------------------------|---------------------------------------------------------------------------|
| @SpringBootApplication         | Enables auto-configuration, component scanning, and configuration management | Application                                                               |
| @EnableConfigurationProperties | Binds YAML properties to Java configuration classes                          | AppConf, BeanConf                                                         |
| @EnableMongoRepositories       | Enables Spring Data MongoDB repositories                                     | Base package from src/main/java/com/difisoft/marketcollector/repositories |
| @EnableScheduling              | Enables scheduled task execution with @Scheduled annotations                 | Real-time Data Collection                                                 |
| @EnableAsync                   | Enables asynchronous method execution with @Async annotations                | Async tasks                                                               |
| @Import                        | Explicitly imports additional configuration beans                            | RedisDao, MarketRedis                                                     |

**Sources:** [2] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/Application.java#L16-21>)

[src/main/java/com/difisoft/marketcollector/Application.java#L16-21] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/Application.java#L16-21>)

#### ##### Static Application Context Reference

The application maintains a static reference to the `ConfigurableApplicationContext`[] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/Application.java#L23>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/Application.java#L23-26>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/Application.java#L23-26>)

**\*\*\***

### #### Configuration Architecture

The configuration system is the backbone of the application's flexibility, enabling the same

### ##### Configuration Hierarchy

![[Chart 2](market-collector-lotte-images/market-collector-lotte-section5-2.svg)]

**\*\*Configuration Domain Mapping\*\***

**\*\*Sources:\*\*** [] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java#L1-258>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java#L1-258>)

### ##### AppConf Structure

The `AppConf` class is annotated with `@ConfigurationProperties(prefix = "app")`[] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java#L16>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java#L16>)

| Configuration Domain  | Class                 | Purpose                                              | Key Properties       |
|-----------------------|-----------------------|------------------------------------------------------|----------------------|
| System Connections    | `SystemConf`          | Connection details for external systems (HTS, Lotte) |                      |
| Real-time Processing  | `RealtimeConf`        | WebSocket configuration and account management       |                      |
| API Integration       | `ApiConnection`       | REST API endpoint URLs and configuration             | `baseUrl`            |
| Kafka Topics          | `Topics`              | Kafka topic names for message routing                | `market`, `realtime` |
| Accounts              | `RealtimeAccountConf` | Trading account credentials and settings             | `username`           |
| WebSocket Connections | `WebSocketConf`       | WebSocket connection parameters                      | `name`, `url`        |

**\*\*Sources:\*\*** [] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java#L61-68>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java#L61-68>)

[src/main/java/com/difisoft/marketcollector/configurations/AppConf.java#L103-120](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java#L103-120)) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java#L103-120>)

[src/main/java/com/difisoft/marketcollector/configurations/AppConf.java#L237-253](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java#L237-253)) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java#L237-253>)

[src/main/java/com/difisoft/marketcollector/configurations/AppConf.java#L71-83](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java#L71-83)) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java#L71-83>)

### ##### Feature Flags

The configuration system includes multiple boolean feature flags that control application

![Chart 3](market-collector-lotte-images/market-collector-lotte-section5-3.svg)

**Sources:** [ ] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java#L32-40>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java#L32>)

The most critical feature flag is `isUsingApi` [ ] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java#L32>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java#L32>)

\* \* \*

### Dependency Injection and Bean Management

The application uses a custom bean factory pattern implemented in `BeanConf` to provide configuration for the application.

### Bean Factory Pattern

![Chart 4](market-collector-lotte-images/market-collector-lotte-section5-4.svg)

**Sources:** [ ] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/BeanConf.java#L1-91>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/BeanConf.java#L1-91>)

### Conditional Service Instantiation

The `symbolInfoService()` method in `BeanConf` demonstrates the conditional instantiation of the `SymbolInfoService` interface.

- Factory Method Definition:** The method is annotated with `@Bean` and returns the `ISymbolInfoService` interface.

[src/main/java/com/difisoft/marketcollector/configurations/BeanConf.java#L38] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/BeanConf.java#L38>)

- Dependency Injection:** The method accepts 15 dependencies as constructor parameters, including `AppConf`, `LotteApiSymbolInfoService`, and `HtsSymbolInfoService`.

[src/main/java/com/difisoft/marketcollector/configurations/BeanConf.java#L38-55] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/BeanConf.java#L38-55>)

- Conditional Logic:** The method checks `appConf.isUsingApi()` to determine which implementation to instantiate.

[src/main/java/com/difisoft/marketcollector/configurations/BeanConf.java#L56] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/BeanConf.java#L56>)

- Implementation Selection:**
  - If `true`: Instantiates `LotteApiSymbolInfoService` with API-specific dependencies.

[src/main/java/com/difisoft/marketcollector/configurations/BeanConf.java#L57-70] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/BeanConf.java#L57-70>)

  - If `false`: Instantiates `HtsSymbolInfoService` with HTS-specific dependencies.

[src/main/java/com/difisoft/marketcollector/configurations/BeanConf.java#L72-88] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/BeanConf.java#L72-88>)

**Sources:** [ ] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/BeanConf.java#L37-90>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/BeanConf.java#L37-90>)

### Implementation Comparison

| Aspect                 | LotteApiSymbolInfoService                    | HtsSymbolInfoService                                              |
|------------------------|----------------------------------------------|-------------------------------------------------------------------|
| Data Source            | Lotte Securities REST APIs                   | HTS socket connections                                            |
| Dependencies           | Requires `lotteApiService`                   | Requires `downloadSymbolListService`, `downloadSymbolListService` |
| Use Case               | Production deployment with stable API access | Development/testing with direct market data                       |
| Constructor Parameters | 10 dependencies                              | 14 dependencies                                                   |

**Sources:** (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/BeanConf.java57-88) (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/BeanConf.java57-88)

#### ##### ObjectMapper Configuration

The `BeanConf` also provides a customized `ObjectMapper` bean for JSON serialization/deserialization.

![[Chart 5](market-collector-lotte-images/market-collector-lotte-section5-5.svg)]

**Sources:** (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/BeanConf.java23-30) (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/BeanConf.java23-30)

The `ObjectMapper` is configured to:

- Ignore unknown properties during deserialization (src/main/java/com/difisoft/marketcollector/configurations/BeanConf.java26) (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/BeanConf.java26)
- Include default views (src/main/java/com/difisoft/marketcollector/configurations/BeanConf.java27) (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/BeanConf.java27)
- Exclude null values from serialization (src/main/java/com/difisoft/marketcollector/configurations/BeanConf.java28) (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/BeanConf.java28)

\* \* \*

#### ##### Component Organization

The application follows a layered architecture pattern where components are organized by responsibility.

![[Chart 6](market-collector-lotte-images/market-collector-lotte-section5-6.svg)]

**Sources:** (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/Application.java1-29) (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java1-258) (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/BeanConf.java1-91) (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/BeanConf.java1-91)

#### ##### Layer Responsibilities

| Layer | Responsibility | Key Components |
|-------|----------------|----------------|
| ---   | ---            | ---            |

```

| **Presentation** | Handle external requests and trigger jobs | `RequestHandler`, Kafka
| **Service** | Business logic and orchestration | `JobService`, `ISymbolInfoService`, `Re
| **Integration** | External system communication | `LotteApiService`, `WsConnection`, `Ws
| **Data Access** | Data persistence and retrieval | Spring Data repositories, `MarketRed
| **Configuration** | System configuration and bean management | `AppConf`, `BeanConf` |

```

```

* * *

```

#### #### System Configuration Methods

The `AppConf` class provides utility methods for constructing connection information objects.

##### ##### LoginData Construction

The `get(Account accountDownload)` method[] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java#L207-213>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java#L207-213>)

[src/main/java/com/difisoft/marketcollector/configurations/AppConf.java#L207-213] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java#L207-213>)

1. Retrieving the `SystemConf` for the account's system
2. Creating a `LoginData` instance with host, ports, and credentials
3. Setting additional IP and media type properties

**\*\*Sources:\*\***[] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java#L207-213>)

[src/main/java/com/difisoft/marketcollector/configurations/AppConf.java#L207-213] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java#L207-213>)

##### ##### DataConnectionInfo Construction

The `get(RealtimeAccountConf account)` method[] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java#L216-234>)

[src/main/java/com/difisoft/marketcollector/configurations/AppConf.java#L216-234] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java#L216-234>)

1. Retrieving the `SystemConf` for the account's system
2. Creating a `DataConnectionInfo` instance with connection details
3. Setting account-specific fields like branch codes, personal ID, agency number, and department

**\*\*Sources:\*\***[] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java#L216-234>)

[src/main/java/com/difisoft/marketcollector/configurations/AppConf.java#L216-234] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java#L216-234>)

These methods serve as factories that combine system-level configuration (from `sysConf` map)

```

* * *

```

#### #### Configuration Data Classes

The `AppConf` class defines several nested configuration data classes that organize related

##### ##### SystemConf

Maps system names to connection details[] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java#L61-68>)

[src/main/java/com/difisoft/marketcollector/configurations/AppConf.java#L61-68] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java#L61-68>)

- ``host``: Primary host address
- ``backupHosts``: List of backup host addresses for failover
- ``loginPort``: Port for authentication
- ``dataPort``: Port for data streaming
- ``marketPort``: Port for market data
- ``secCode``: Security code for system access

#### ##### RealtimeAccountConf

Extends ``Account`` with real-time specific configuration[] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java#L148-191>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java#L148-191>)

[src/main/java/com/difisoft/marketcollector/configurations/AppConf.java#L148-191] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java#L148-191>)

- Account credentials and system identification
- ``codeType``: Type of symbols (STOCK, INDEX, etc.)
- ``exchange``: Market exchange (HOSE, HNX, UPCOM)
- ``topics``: List of WebSocket topics to subscribe to
- ``topicMapping``: Custom topic name mappings
- ``onlyCodes``: Filter for specific symbol codes
- ``noOfThread``: Thread pool size for processing
- ``multipleConnections``: Configuration for splitting connections
- Branch codes, personal ID, agency/department numbers for HTS authentication

#### ##### WebSocketConf

Configuration for Spring WebSocket connections[] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java#L123-131>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java#L123-131>)

[src/main/java/com/difisoft/marketcollector/configurations/AppConf.java#L123-131] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java#L123-131>)

- ``name``: Connection identifier
- ``url``: WebSocket endpoint URL
- ``noOfThread``: Thread pool size
- ``codeType``: Symbol type filter
- ``exchange``: Market exchange filter
- ``channels``: List of channels to subscribe to
- ``codeMapping``: Symbol code transformation mappings

**\*\*Sources:\*\*** [] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java#L61-191>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java#L61-191>)

[src/main/java/com/difisoft/marketcollector/configurations/AppConf.java#L61-191] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java#L61-191>)

\* \* \*

#### #### Summary

The core architecture of Market Collector Lotte is built on Spring Boot with a configuration

1. **\*\*Centralized Configuration\*\***: The ``AppConf`` class serves as a single source of truth for all configuration.
2. **\*\*Conditional Dependency Injection\*\***: The ``BeanConf`` factory pattern enables runtime selection of beans based on environment.
3. **\*\*Layered Architecture\*\***: Components are organized into presentation, service, and integration layers.
4. **\*\*Multi-Environment Support\*\***: Configuration can be overridden through Spring profiles.

The architecture emphasizes:

- **\*\*Flexibility\*\***: Feature flags and conditional beans enable different operational modes
- **\*\*Modularity\*\***: Clear separation of concerns across layers
- **\*\*Maintainability\*\***: Configuration-driven design reduces code changes for environment-specific requirements
- **\*\*Testability\*\***: Interface-based dependency injection enables easy mocking and testing

For implementation details of specific architectural components, refer to the child pages of this section.

### ### Application Bootstrap

Relevant source files

- [src/main/java/com/difisoft/marketcollector/Application.java] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/Application.java>)

This document covers the `Application` class entry point, Spring Boot initialization sequence, and configuration details.

For detailed configuration property mapping, see page 3.2. For conditional bean instantiation logic, see page 3.3.

### #### Application Class

The `Application` class at [src/main/java/com/difisoft/marketcollector/Application.java] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/Application.java>)

[src/main/java/com/difisoft/marketcollector/Application.java22-28] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/Application.java#L22-28>)

The class uses multiple Spring annotations to configure framework features:

| Annotation                     | Configuration Target                              | Effect                                                                    |
|--------------------------------|---------------------------------------------------|---------------------------------------------------------------------------|
| @SpringBootApplication         | Package `com.difisoft.marketcollector`            | Auto-configuration, component scanning, and enabling Spring Boot features |
| @EnableConfigurationProperties | `AppConf.class`                                   | Binds external properties to `AppConf` class                              |
| @EnableMongoRepositories       | `basePackageClasses = SymbolInfoRepository.class` | Scans for and registers MongoDB repositories                              |
| @EnableScheduling              | N/A                                               | Activates `@Scheduled` annotation processing for `JobService`             |
| @EnableAsync                   | N/A                                               | Enables `@Async` annotation for asynchronous method execution             |
| @Import                        | `RedisDao.class, MarketRedisDao.class`            | Explicitly imports Redis data access layer                                |

Sources: [src/main/java/com/difisoft/marketcollector/Application.java] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/Application.java>)

[src/main/java/com/difisoft/marketcollector/Application.java16-21] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/Application.java#L16-21>)

### #### Bootstrap Initialization Sequence

#### \*\*Spring Boot Startup Flow\*\*

![[Chart 1](market-collector-lotte-images/market-collector-lotte-section6-1.svg)]

The initialization sequence follows standard Spring Boot lifecycle. Component scanning discovers beans, configuration properties are loaded, and the application context is initialized.

Sources: [src/main/java/com/difisoft/marketcollector/Application.java] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/Application.java>)

[src/main/java/com/difisoft/marketcollector/Application.java25-27] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/Application.java#L25-27>)

### #### Component Scanning and Bean Registration

## **\*\*Annotation-Driven Bean Discovery\*\***

![[Chart 2](market-collector-lotte-images/market-collector-lotte-section6-2.svg)]

The `@SpringBootApplication` annotation combines `@Configuration`, `@EnableAutoConfiguration`

Sources:[] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java>

[src/main/java/com/difisoft/marketcollector/Application.java#L16-21] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/Application.java#L16-21>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/Application.java#L16-21>)

### **#### Configuration Property Binding**

The `@EnableConfigurationProperties(AppConf.class)` annotation triggers Spring's configurat

## **\*\*Property Binding Flow\*\***

![[Chart 3](market-collector-lotte-images/market-collector-lotte-section6-3.svg)]

The `AppConf` instance becomes available as a Spring bean and can be injected into other co

Sources:[] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java>

[src/main/java/com/difisoft/marketcollector/Application.java#L17] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/Application.java#L17>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/Application.java#L17>)

### **#### MongoDB Repository Configuration**

The `@EnableMongoRepositories(basePackageClasses = SymbolInfoRepository.class)` annotation

## **\*\*Repository Scanning\*\***

![[Chart 4](market-collector-lotte-images/market-collector-lotte-section6-4.svg)]

Sources:[] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java>

[src/main/java/com/difisoft/marketcollector/Application.java#L18] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/Application.java#L18>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/Application.java#L18>)

### **#### Redis Data Access Layer**

The `@Import({RedisDao.class, MarketRedisDao.class})` annotation explicitly registers Redis

| Class          | Purpose                                                                |
|----------------|------------------------------------------------------------------------|
| RedisDao       | Generic Redis operations (get, set, delete)                            |
| MarketRedisDao | Market-specific Redis operations (quote caching, market depth storage) |

Sources:[] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java>

[src/main/java/com/difisoft/marketcollector/Application.java#L21] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/Application.java#L21>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/Application.java#L21>)

### **#### Static Application Context Reference**

The `Application` class stores the `ConfigurableApplicationContext` in a static field:

```
public static ConfigurableApplicationContext applicationContext;
```

This static reference allows non-Spring managed code to access the application context and

```
applicationContext = new SpringApplication(Application.class).run(args);
```

## **\*\*Context Access Pattern\*\***

![[Chart 5](market-collector-lotte-images/market-collector-lotte-section6-5.svg)]

This pattern is used when dependency injection is not available, such as in utility classes.

Sources:[] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java

[src/main/java/com/difisoft/marketcollector/Application.java23-27] (https://github.com/nhsv-

### **#### Initialization Dependencies**

The application bootstrap process requires several key dependencies to be available:

| Dependency Type  | Components           | Purpose                          |
|------------------|----------------------|----------------------------------|
| Configuration    | `AppConf`            | Central configuration management |
| Database         | MongoDB repositories | Market data persistence          |
| Caching          | Redis DAOs           | High-performance data caching    |
| Scheduling       | Spring Scheduler     | Automated data collection tasks  |
| Async Processing | Spring Async         | Non-blocking operations          |

Sources:[] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java

[src/main/java/com/difisoft/marketcollector/Application.java3-11] (https://github.com/nhsv-t

---

### **### Configuration Management**

Relevant source files

- [src/main/java/com/difisoft/marketcollector/configurations/AppConf.java] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java)

### **#### Purpose and Scope**

This document covers the comprehensive configuration management system implemented in the M

For information about Spring bean configuration and service instantiation, see [Bean Config

### **#### Configuration Architecture Overview**

The configuration system is centered around the `AppConf` class, which serves as the root c

![[Chart 1](market-collector-lotte-images/market-collector-lotte-section7-1.svg)]

**\*\*Sources:\*\***[] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main

[src/main/java/com/difisoft/marketcollector/configurations/AppConf.java16-33] (https://githu

### **#### Core Configuration Components**

#### ##### AppConf - Central Configuration Hub

The `AppConf` class serves as the primary configuration container, marked with `@Configurat

| Property      | Type    | Purpose                                                |
|---------------|---------|--------------------------------------------------------|
| ---           | ---     | ---                                                    |
| `domain`      | String  | Application domain identifier                          |
| `clusterId`   | String  | Cluster identification for distributed deployment      |
| `serviceName` | String  | Service name for identification and logging            |
| `nodeId`      | String  | Unique node identifier                                 |
| `kafkaUrl`    | String  | Kafka broker connection URLs                           |
| `isUsingApi`  | boolean | Toggle between API-based and HTS-based implementations |

**Sources:** <https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java#L19-32> (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java#L19-32>)

#### ##### SystemConf - External System Connections

The `SystemConf` class manages connection parameters for external trading systems, support:

![[Chart 2](market-collector-lotte-images/market-collector-lotte-section7-2.svg)]

**Sources:** <https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java#L61-68> (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java#L61-68>) <https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java#L207-234> (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java#L207-234>)

#### ##### Topics - Kafka Message Routing

The `Topics` configuration class defines Kafka topic names for different message types, ena

| Topic Property            | Purpose                           |
|---------------------------|-----------------------------------|
| ---                       | ---                               |
| `market`                  | Market data distribution          |
| `marketInit`              | Market initialization messages    |
| `fix`                     | FIX protocol messages             |
| `requestResponseListener` | Request-response pattern handling |
| `realtime`                | Real-time data streaming          |
| `symbolInfoUpdate`        | Symbol information updates        |
| `quoteRecover`            | Quote recovery operations         |

**Sources:** <https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java#L71-83> (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java#L71-83>)

#### #### Real-time Data Configuration

##### ##### RealtimeConf - Real-time Collection Settings

The `RealtimeConf` class manages comprehensive settings for real-time data collection, inc

![Chart 3](market-collector-lotte-images/market-collector-lotte-section7-3.svg)

**Sources:** [\[src/main/java/com/difisoft/marketcollector/configurations/AppConf.java103-120\]](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java#L103-120) ([https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java103-120](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java#L103-120)) ([https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java103-120](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java#L103-120))

#### ##### RealtimeAccountConf - Account-Specific Configuration

The `RealtimeAccountConf` class extends the base `Account` class with real-time specific settings.

| Property            | Type                | Purpose                              |
|---------------------|---------------------|--------------------------------------|
| name                | String              | Account identifier name              |
| codeType            | SymbolTypeEnum      | Symbol type filter                   |
| exchange            | MarketTypeEnum      | Target market exchange               |
| topicMapping        | Map<String, String> | Topic routing mappings               |
| onlyCodes           | Set                 | Specific symbol codes to monitor     |
| multipleConnections | List                | Additional connection configurations |
| resubscribeAfterMs  | Integer             | Resubscription interval              |

**Sources:** [\[src/main/java/com/difisoft/marketcollector/configurations/AppConf.java148-165\]](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java#L148-165) ([https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java148-165](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java#L148-165)) ([https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java148-165](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java#L148-165))

#### #### API Configuration Management

##### ##### ApiConnection - External API Integration

The `ApiConnection` class centralizes all external API endpoint configurations for Lotte Securities.

![Chart 4](market-collector-lotte-images/market-collector-lotte-section7-4.svg)

**Sources:** [\[src/main/java/com/difisoft/marketcollector/configurations/AppConf.java237-253\]](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java#L237-253) ([https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java237-253](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java#L237-253)) ([https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java237-253](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java#L237-253))

#### #### Feature Flags and System Settings

The `AppConf` class includes multiple boolean configuration properties that control various application features.

| Feature Flag                | Default | Purpose                                    |
|-----------------------------|---------|--------------------------------------------|
| enableMultipleInstance      | false   | Support for multiple application instances |
| enableQuery                 | true    | Enable query functionality                 |
| enableInitMarket            | \-      | Enable market initialization               |
| enableUsingRedisForRealtime | true    | Use Redis for real-time data caching       |
| enableIgnoreQuote           | true    | Ignore specific quote types                |
| enableBond                  | false   | Enable bond instrument processing          |
| enableStoreConnectionInfo   | true    | Store connection information               |
| logRealtimePacket           | false   | Log real-time packet details               |

**Sources:** [\[src/main/java/com/difisoft/marketcollector/configurations/AppConf.java34-40\]](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java#L34-40) ([https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java34-40](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java#L34-40)) ([https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java34-40](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java#L34-40))

src/main/java/com/difisoft/marketcollector/configurations/AppConf.java31] (<https://github.com/nhsv-tradex/market-collector-lotte-images/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java31>)

### Configuration Initialization and Usage

#### Connection Object Factory Methods

The `AppConf` class provides factory methods to create connection objects from configuration properties.

! [Chart 5] (market-collector-lotte-images/market-collector-lotte-section7-5.svg)

The `get(Account accountDownload)` method creates `LoginData` objects by combining account information and download data.

**Sources:** [1] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java207-234>) (<https://github.com/nhsv-tradex/market-collector-lotte-images/blob/1df4d191/market-collector-lotte-section7-5.svg>)

[src/main/java/com/difisoft/marketcollector/configurations/AppConf.java207-234] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java207-234>)

[src/main/java/com/difisoft/marketcollector/configurations/AppConf.java203-205] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java203-205>) (<https://github.com/nhsv-tradex/market-collector-lotte-images/blob/1df4d191/market-collector-lotte-section7-5.svg>)

#### Configuration Binding

The configuration system uses Spring Boot's `@ConfigurationProperties` annotation to automatically bind configuration properties to the `AppConf` class.

**Sources:** [1] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java16>) (<https://github.com/nhsv-tradex/market-collector-lotte-images/blob/1df4d191/market-collector-lotte-section7-5.svg>)

[src/main/java/com/difisoft/marketcollector/configurations/AppConf.java16] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/AppConf.java16>)

---

### Bean Configuration & Service Factory

Relevant source files

- [src/main/java/com/difisoft/marketcollector/configurations/BeanConf.java] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/BeanConf.java>)

This document covers the Spring bean configuration system and service factory pattern implementation.

For information about the overall configuration management system, see [Configuration Management Overview].

### Configuration Overview

The `BeanConf` class serves as the central Spring configuration component that defines application beans and their dependencies.

#### Bean Configuration Structure

! [Chart 1] (market-collector-lotte-images/market-collector-lotte-section8-1.svg)

**Sources:** [1] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/BeanConf.java21-90>) (<https://github.com/nhsv-tradex/market-collector-lotte-images/blob/1df4d191/market-collector-lotte-section8-1.svg>)

[src/main/java/com/difisoft/marketcollector/configurations/BeanConf.java21-90] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/configurations/BeanConf.java21-90>)

### Core Bean Definitions

#### ##### ObjectMapper Configuration

The `objectMapper` bean provides a centrally configured Jackson `ObjectMapper` for JSON ser

| Configuration Property       | Value      | Purpose                                        |
|------------------------------|------------|------------------------------------------------|
| ---   ---   ---              |            |                                                |
| `FAIL_ON_UNKNOWN_PROPERTIES` | `false`    | Ignore unknown JSON properties during deserial |
| `DEFAULT_VIEW_INCLUSION`     | `true`     | Include all fields in JSON views by default    |
| `SerializationInclusion`     | `NON_NULL` | Exclude null values from JSON output           |

**Sources:** <https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main>

[src/main/java/com/difisoft/marketcollector/configurations/BeanConf.java23-30] (<https://gitl>

#### ##### FileService Configuration

The `fileService` bean is instantiated using the file configuration from `AppConf` through

**Sources:** <https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main>

[src/main/java/com/difisoft/marketcollector/configurations/BeanConf.java32-35] (<https://gitl>

#### #### Service Factory Pattern

#### ##### Conditional Service Instantiation

The `symbolInfoService` bean implements a factory pattern that conditionally creates differ

! [Chart 2] (<market-collector-lotte-images/market-collector-lotte-section8-2.svg>)

**Sources:** <https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main>

[src/main/java/com/difisoft/marketcollector/configurations/BeanConf.java37-89] (<https://gitl>

#### ##### Implementation Selection Strategy

The factory method uses the following decision logic:

- **API Mode** (`isUsingApi() == true`): Creates `LotteApiSymbolInfoService` instance
- **HTS Mode** (`isUsingApi() == false`): Creates `HtsSymbolInfoService` instance

Each implementation receives a different set of dependencies appropriate for its data colle

#### #### Dependency Injection Architecture

#### ##### Shared Dependencies

Both service implementations receive common dependencies:

| Dependency       | Type                   | Purpose                   |
|------------------|------------------------|---------------------------|
| ---   ---   ---  |                        |                           |
| `symbolInfoRepo` | `SymbolInfoRepository` | Symbol data persistence   |
| `appConf`        | `AppConf`              | Application configuration |

```

| `requestSender` | `RequestSender` | Request handling |
| `marketRedisDao` | `MarketRedisDao` | Redis caching operations |
| `indexStockListRepository` | `IndexStockListRepository` | Index stock data |
| `objectMapper` | `ObjectMapper` | JSON processing |
| `symbolInfoRollerRepository` | `SymbolInfoRollerRepository` | Symbol rollover data |
| `symbolDailyRepository` | `SymbolDailyRepository` | Daily symbol data |
| `fileService` | `FileService` | File operations |
| `coordinatorService` | `CoordinatorService` | Service coordination |
| `holidayService` | `HolidayService` | Holiday calendar |

**Sources:**[] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main

[src/main/java/com/difisoft/marketcollector/configurations/BeanConf.java38-54] (https://gitl

##### Implementation-Specific Dependencies

##### LotteApiSymbolInfoService Dependencies

- Excludes HTS-specific services
- Includes `lotteApiService` for API integration

**Sources:**[] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main

[src/main/java/com/difisoft/marketcollector/configurations/BeanConf.java57-70] (https://gitl

##### HtsSymbolInfoService Dependencies

- Includes `downloadSymbolListService` for symbol list management
- Includes `downloadInfoService` for data download operations
- Includes `cacheService` for caching functionality
- Includes `downloadAccountService` for account data
- Includes `realTimeDataListenerService` for real-time data processing

**Sources:**[] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main

[src/main/java/com/difisoft/marketcollector/configurations/BeanConf.java72-88] (https://gitl

#### Configuration Integration

The bean configuration integrates with the broader application configuration system through

1. **AppConf Integration**: Uses `AppConf` for file service configuration and implementat
2. **Repository Integration**: Injects all required repository beans for data access
3. **Service Integration**: Coordinates with other application services through dependency

The factory pattern enables runtime flexibility while maintaining compile-time type safety

**Sources:**[] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main

[src/main/java/com/difisoft/marketcollector/configurations/BeanConf.java1-91] (https://githu

---

## Real-time Data Collection

```

### ### Overview

#### Relevant source files

- [src/main/java/com/difisoft/marketcollector/ws/WsConnection.java](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection.java) (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection.java)
- [src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java) (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java)

### #### Purpose and Scope

This document provides a comprehensive overview of the real-time market data collection system.

For detailed information about specific subsystems, see:

- WebSocket connection lifecycle and channel management: [WebSocket Architecture] (#websocket-architecture)
- Message parsing and handler implementations: [Message Processing Pipeline] (#message-processing-pipeline)
- Reconnection strategies and error recovery: [Connection Management & Recovery] (#connection-management-&-recovery)
- Output interfaces and downstream distribution: [Data Output & Distribution] (#4.4)

For configuration of the WebSocket connections, see [Configuration Management] (#configuration-management).

\* \* \*

### #### System Architecture

The real-time data collection system consists of two parallel WebSocket implementations that collect data from different sources.

#### ##### High-Level Component Structure

![[Chart 1](market-collector-lotte-images/market-collector-lotte-section9-1.svg)]

**Sources:** [src/main/java/com/difisoft/marketcollector/ws/WsConnection.java] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection.java)

[src/main/java/com/difisoft/marketcollector/ws/WsConnection.java] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection.java)

[src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java)

\* \* \*

### #### Dual WebSocket Implementation

The system provides two alternative WebSocket implementations that offer the same functionality.

#### ##### Implementation Comparison

| Aspect            | WsConnection                                                                                                                  | WsConnection2                                                                                                            |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| API               | Jakarta WebSocket ( <code>jakarta.websocket.*</code> )                                                                        | Spring WebSocket ( <code>org.springframework.websocket.*</code> )                                                        |
| Annotation Style  | <code>@ClientEndpoint</code> , <code>@OnOpen</code> , <code>@OnClose</code> , <code>@OnMessage</code> , <code>@OnError</code> | <code>ServerEndpoint</code> , <code>OnOpen</code> , <code>OnClose</code> , <code>OnMessage</code> , <code>OnError</code> |
| Connection Method | <code>ContainerProvider.getWebSocketContainer().connectToServer()</code>                                                      | <code>WebSocketClient.connectToServer()</code>                                                                           |

```

| **Message Sending** | `session.getAsyncRemote().sendText()` | `session.sendMessage(new`
| **Ping/Pong Handling** | Automatic (by container) | Manual via `PingMessage` / `PongMes`
| **Time Format Parsing** | Lotte format (`Hmms`) with custom formatter | Standard format
| **Redis Integration** | Yes (via `MarketRedisDao`) | No |
| **Symbol Cache** | Yes (via `CacheService`) | No (uses code mapping only) |

```

**\*\*Sources:\*\*** [] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection.java#L36-75>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java#L36-66>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection3.java#L36-66>)

#### ##### Connection Initialization

Both implementations follow a similar startup sequence:

! [Chart 2] (market-collector-lotte-images/market-collector-lotte-section9-2.svg)

**\*\*Sources:\*\*** [] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection.java#L77-99>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java#L68-115>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection3.java#L68-115>)

\* \* \*

#### ##### Message Flow and Routing

The system receives pipe-delimited messages from HTS and routes them to specialized handlers.

#### ##### Message Format and Service Prefixes

All HTS messages follow a pipe-delimited format: `service|field1|field2|...`. The `service`

| Service Prefix      | Handler Method         | Output Model       | Description                       |
|---------------------|------------------------|--------------------|-----------------------------------|
| `auto.idxqt.*`      | `handleIndexQuote()`   | `QuoteUpdate`      | Index quotes (VNINDEX, HNX-INDEX) |
| `auto.qt.*`         | `handleStockQuote()`   | `QuoteUpdate`      | Stock/ETF/Fund quotes             |
| `auto.bo.*`         | `handleStockBidAsk()`  | `BidOfferUpdate`   | Bid/offer depth data (10 levels)  |
| `auto.tickerNews.*` | `handleSessionEvent()` | `MarketStatusData` | Market session event              |

**\*\*Sources:\*\*** [] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection.java#L154-171>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java#L188-219>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection3.java#L188-219>)

#### ##### Message Processing Flow

! [Chart 3] (market-collector-lotte-images/market-collector-lotte-section9-3.svg)

**\*\*Sources:\*\*** [] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection.java#L154-171>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java#L188-219>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection3.java#L188-219>)

```

src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java203-219] (https://github.com/
* * *

#### Channel Subscription Model

The system subscribes to specific channels on the HTS WebSocket to receive targeted data st

#### Channel Patterns

| Code Type | Channel Pattern | Example | Purpose |
|---|---|---|---|
| `INDEX` | `sub/pro.pub.auto.idxqt./{codes}` | `sub/pro.pub.auto.idxqt./VNINDEX` | HNXINI
| `MARKET-STATUS` | `sub/pro.pub.auto.tickerNews.*/` | `sub/pro.pub.auto.tickerNews.*/` |
| Stock/Other | `{template}{codes}` | `sub/pro.pub.auto.qt./AAA` | FPT |

#### Code Mapping Strategy

Both implementations maintain a `codeMapping` map to translate HTS security codes to standa

! [Chart 4] (market-collector-lotte-images/market-collector-lotte-section9-4.svg)

**Sources:** [] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main

[src/main/java/com/difisoft/marketcollector/ws/WsConnection.java78-89] (https://github.com/

src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java69-106] (https://github.com/

* * *

#### Data Transformation

Each handler method parses the pipe-delimited fields and populates the corresponding output

#### Common Transformation Operations

| Operation | Method | Description |
|---|---|---|
| **Timestamp Parsing** | `LocalTime.parse(parts[n], formatter)` | Converts HTS time stri
| **Timezone Conversion** | `.atZone(VIETNAM_ID).withZoneSameInstant(UTC_ID)` | Converts
| **Code Mapping** | `codeMapping.getOrDefault(code, code)` | Translates HTS codes to star
| **Numeric Parsing** | `getDouble()`, `getLong()`, `getInteger()` | Safe parsing with de
| **Precision Rounding** | `BigDecimal.setScale(4, HALF_EVEN)` | Rounds to 4 decimal place

#### Example: Stock Quote Transformation

The `handleStockQuote()` method demonstrates the comprehensive field mapping:

! [Chart 5] (market-collector-lotte-images/market-collector-lotte-section9-5.svg)

**Sources:** [] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main

[src/main/java/com/difisoft/marketcollector/ws/WsConnection.java182-226] (https://github.com/

```

src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java237-272] (<https://github.com>)

#### #### Bid/Offer Depth Processing

The `handleStockBidAsk()` method extracts 10 levels of market depth data:

**\*\*Field Layout (WsConnection):\*\***

- Parts 13-73: Bid/offer levels (6 fields per level × 10 levels)
  - `parts[13+i*6]`: Bid price level i
  - `parts[13+i*6+2]`: Bid volume level i
  - `parts[13+i*6+3]`: Offer price level i
  - `parts[13+i*6+5]`: Offer volume level i
- Parts 73-74: Total bid/offer volumes
- Parts 83-85: Total bid/offer counts

**\*\*Session Mapping (WsConnection only):\*\***

The `parts[4]` control code is mapped to session types based on market:

| Control Code | HOSE Session   | HNX Session    |
|--------------|----------------|----------------|
| 'P'          | 'ATO'          | 'LO'           |
| 'O'          | 'LO'           | 'LO'           |
| 'R'          | 'LO'           | '-'            |
| 'I'          | 'INTERMISSION' | '-'            |
| '2'          | '-'            | 'INTERMISSION' |
| 'A'          | 'ATC'          | 'ATC'          |
| 'C'          | 'PLO'          | 'PLO'          |
| 'K', 'G'     | 'CLOSED'       | '-'            |
| '13', '97'   | '-'            | 'CLOSED'       |

**\*\*Sources:\*\*** [<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main>]

[src/main/java/com/difisoft/marketcollector/ws/WsConnection.java228-277] (<https://github.com>)

src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java274-293] (<https://github.com>)

\* \* \*

#### #### Configuration Dependencies

The WebSocket connections are configured through `AppConf.WebSocketConf` and `AppConf.ApiCo`

#### #### WebSocketConf Properties

| Property                   | Type                                     | Description                                         |
|----------------------------|------------------------------------------|-----------------------------------------------------|
| <code>'name'</code>        | <code>'String'</code>                    | Connection identifier for logging                   |
| <code>'url'</code>         | <code>'String'</code>                    | WebSocket endpoint URL                              |
| <code>'codeType'</code>    | <code>'String'</code>                    | Data type: "INDEX", "MARKET-STATUS", or stock types |
| <code>'channels'</code>    | <code>'List&lt;String&gt;'</code>        | Channel subscription templates                      |
| <code>'codeMapping'</code> | <code>'Map&lt;String, String&gt;'</code> | Additional code translations (WsConnection2)        |

#### ##### Required Dependencies

Each WebSocket implementation requires specific dependencies injected via constructor:

**\*\*WsConnection:\*\***

- ``Map<String, String> statusMap``: Market status code translations
- ``AppConf.WebSocketConf config``: Connection configuration
- ``AppConf.ApiConnection apiConfig``: API configuration
- ``ObjectMapper objectMapper``: JSON serialization
- ``Consumer<Object> outputConsumer``: Output handler
- ``MarketRedisDao marketRedisDao``: Redis access for symbol info
- ``CacheService cacheService``: In-memory symbol cache
- ``Set<String> codes``: Symbol codes to subscribe to

**\*\*WsConnection2:\*\***

- Same as WsConnection, except:
  - No ``marketRedisDao`` parameter
  - No ``cacheService`` parameter
  - Uses API calls instead for symbol lookup

**\*\*Sources:\*\*** [[https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection.java#L56-75)

[src/main/java/com/difisoft/marketcollector/ws/WsConnection.java#L56-75](#)] ([https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java#L51-66)

[src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java#L51-66](#)] ([https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java#L51-66)

\* \* \*

#### ##### Error Handling and Resilience

Both implementations include robust error handling mechanisms to ensure continuous operation.

#### ##### Connection Lifecycle Callbacks

! [Chart 6] ([market-collector-lotte-images/market-collector-lotte-section9-6.svg](https://github.com/nhsv-tradex/market-collector-lotte-images/blob/main/market-collector-lotte-section9-6.svg))

#### ##### Exponential Backoff Strategy

Both implementations use identical exponential backoff logic when connection attempts fail.

$\text{timeout} = \text{index} \times 2000 + 5000$   $\text{timeout} = \min(\text{timeout}, 120000)$

Attempt 0: 5 seconds Attempt 1: 7 seconds Attempt 2: 9 seconds Attempt 3: 11 seconds ... Attempt 58+: 120 seconds (max)

This is implemented in the `startWebsocket(long index)` method with recursive retry logic:

**Sources:** [https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection.java#L101-123] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java#L117-139) (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection3.java#L117-139)

### #### Safe Data Parsing

All numeric conversions use defensive parsing methods that return default values on parse errors:

| Method                                  | Return Type          | Default           | Example Usage         |
|-----------------------------------------|----------------------|-------------------|-----------------------|
| <code>getDouble(value, default)</code>  | <code>Double</code>  | <code>null</code> | Parse prices, rates   |
| <code>getLong(value, default)</code>    | <code>Long</code>    | <code>null</code> | Parse volumes, counts |
| <code>getInteger(value, default)</code> | <code>Integer</code> | <code>null</code> | Parse integer counts  |

These methods catch `Exception` and return the default value if parsing fails, preventing runtime errors.

**Sources:** [https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection.java#L314-346] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java#L329-360) (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection3.java#L329-360)

\* \* \*

### #### Output Interface

All processed data models are published through a single unified interface: `Consumer<Object>`

### #### Consumer Contract

The `outputConsumer` accepts three types of objects:

- QuoteUpdate:** Stock and index quote data
- BidOfferUpdate:** Market depth (bid/offer) data
- MarketStatusData:** Market session status events

Each handler method calls `outputConsumer.accept(model)` after building the appropriate data model.

- Publishing to Kafka topics
- Caching in Redis
- Persisting to MongoDB
- Any other downstream processing

For details on the consumer implementation and distribution strategies, see [Data Output & Distribution Strategies]

**Sources:** [https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection.java#L46-311] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java#L46-311) (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection3.java#L46-311)

```

src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java43-326] (https://github.com/

* * *

#### Summary

The real-time data collection system provides fault-tolerant WebSocket connectivity to HTS

Key design principles:

- **Dual implementation support**: Choose Jakarta or Spring WebSocket based on deployment
- **Unified message processing**: Identical handler logic regardless of WebSocket library
- **Defensive parsing**: All conversions handle errors gracefully
- **Automatic recovery**: Exponential backoff with configurable retry limits
- **Decoupled output**: Single consumer interface for all data types

### WebSocket Architecture

Relevant source files

- [
    src/main/java/com/difisoft/marketcollector/ws/WsConnection.java] (https://github.com/nhs
- [
    src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java] (https://github.com/nhs

#### Purpose and Scope

This document describes the WebSocket connection layer responsible for establishing and man

For message parsing and routing logic, see [Message Processing Pipeline](#message-processin

* * *

#### Dual WebSocket Implementation

The system implements two separate WebSocket client technologies to provide operational flex

| Implementation | Technology Stack | Client Endpoint Annotation | Primary Use Case |
|---|---|---|---|
| `WsConnection` | Jakarta WebSocket API (JSR-356) | `@ClientEndpoint` | Index quotes, sto
| `WsConnection2` | Spring WebSocket Framework | `WebSocketHandler` interface | Alternativ

##### Implementation Comparison

![[Chart 1](market-collector-lotte-images/market-collector-lotte-section10-1.svg)]

**Sources:** [] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main

[src/main/java/com/difisoft/marketcollector/ws/WsConnection.java36-37] (https://github.com/

src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java36] (https://github.com/nhsv

```

### #### Key Architectural Differences

#### \*\*WsConnection (Jakarta API)\*\*

- Uses annotation-based lifecycle callbacks: `@OnOpen`, `@OnMessage`, `@OnClose`, `@OnError`  
 src/main/java/com/difisoft/marketcollector/ws/WsConnection.java125-176] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection.java#L125-176>)
- Obtains `WebSocketContainer` via `ContainerProvider.getWebSocketContainer()`  
 src/main/java/com/difisoft/marketcollector/ws/WsConnection.java109] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection.java#L109>)
- Connects with `container.connectToServer(this, URI.create(config.getUrl()))`  
 src/main/java/com/difisoft/marketcollector/ws/WsConnection.java111] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection.java#L111>)
- Uses `session.getAsyncRemote().sendText(it)` for message transmission  
 src/main/java/com/difisoft/marketcollector/ws/WsConnection.java131] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection.java#L131>)

#### \*\*WsConnection2 (Spring Framework)\*\*

- Implements `WebSocketHandler` interface with explicit method overrides  
 src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java36] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java#L36>)
- Uses `StandardWebSocketClient` and `client.execute(this, config.getUrl()).join()`  
 src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java125-126] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java#L125-126>)
- Sends messages via `session.sendMessage(new TextMessage(it))`  
 src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java178] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java#L178>)
- Handles `PingMessage` and `PongMessage` explicitly  
 src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java190-195] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java#L190-195>)

\*\*Sources:\*\*[] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection.java#L36-75>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java#L36-66>)

\* \* \*

### #### Connection Lifecycle

Both implementations follow a consistent connection lifecycle with initialization, channel

### #### Lifecycle State Machine

![Chart 2] ([market-collector-lotte-images/market-collector-lotte-section10-2.svg](https://github.com/nhsv-tradex/market-collector-lotte-images/blob/1df4d191/market-collector-lotte-section10-2.svg))

\*\*Sources:\*\*[] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection.java#L77-99>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java#L68-115>)

#### ##### Initialization Phase

The `start()` method prepares channel subscriptions and initiates the WebSocket connection.

! [Chart 3] (market-collector-lotte-images/market-collector-lotte-section10-3.svg)

**Sources:** [] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main>

[src/main/java/com/difisoft/marketcollector/ws/WsConnection.java77-99] (<https://github.com/>

src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java68-115] (<https://github.com/>

#### ##### Connection Establishment

The `startWebsocket(long index)` method implements connection logic with exponential backoff.

**WsConnection Implementation:**

1. Close existing session if present [WsConnection.java:102-108]
2. Obtain WebSocketContainer from ContainerProvider [WsConnection.java:109]
3. Connect to URI from config [WsConnection.java:111]
4. On failure: calculate timeout = min(index \* 2000 + 5000, 120000) [WsConnection.java:113-114]
5. Sleep and retry with incremented index [WsConnection.java:117-118]

**WsConnection2 Implementation:**

1. Close existing session if present [WsConnection2.java:118-123]
2. Create StandardWebSocketClient [WsConnection2.java:125]
3. Execute connection with .join() to block [WsConnection2.java:126]
4. On failure: same exponential backoff logic [WsConnection2.java:129-137]

**Sources:** [] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main>

[src/main/java/com/difisoft/marketcollector/ws/WsConnection.java101-123] (<https://github.com/>

src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java117-139] (<https://github.com/>

#### ##### Post-Connection Subscription

After successful connection, both implementations subscribe to configured channels:

**WsConnection:** Immediate subscription in `@OnOpen` callback[] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection.java125-133>) (<https://github.com/>

[src/main/java/com/difisoft/marketcollector/ws/WsConnection.java125-133] (<https://github.com/>

#### @OnOpen

```
public void onOpen(Session userSession) {
    session = userSession;
    channels.forEach(it -> {
```

```

        session.getAsyncRemote().sendText(it);
    });
}

```

```

**WsConnection2:** Delayed subscription via `Timer` (500ms delay) in `afterConnectionEstablished`
[src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java:170-185] (https://github.com/difisoft/marketcollector/blob/master/src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java#L170-L185)

```

```

public void afterConnectionEstablished(WebSocketSession session) {
    Timer timer = new Timer();
    timer.schedule(new TimerTask() {
        public void run() {
            channels.forEach(it -> {
                session.sendMessage(new TextMessage(it));
            });
        }
    }, 500);
}

```

```

**Sources:**[] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/
[src/main/java/com/difisoft/marketcollector/ws/WsConnection.java125-133] (https://github.com/
src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java170-185] (https://github.com/

```

```

* * *

```

#### #### Channel Subscription Model

The system uses a text-based subscription protocol where channel names follow specific patterns.

#### ##### Channel Format Specification

| Code Type     | Channel Format                       | Example                           | Purpose |
|---------------|--------------------------------------|-----------------------------------|---------|
| INDEX         | `sub/pro.pub.auto.idxqt./{secCodes}` | `sub/pro.pub.auto.idxqt./VN30`    | VNINDEX |
| MARKET-STATUS | `sub/pro.pub.auto.tickerNews.*/*`    | `sub/pro.pub.auto.tickerNews.*/*` |         |
| Stock quotes  | `{channel_prefix}{codes}`            | `sub/pro.pub.auto.qt./FPT`        | VNM\`   |
| Bid/Ask       | `{channel_prefix}{codes}`            | `sub/pro.pub.auto.bo./FPT`        | VNM\`   |

```

**Sources:**[] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/
[src/main/java/com/difisoft/marketcollector/ws/WsConnection.java78-96] (https://github.com/
src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java69-113] (https://github.com/

```

#### ##### Index Subscription Flow

Index subscription requires special handling to map exchange codes to standard symbol codes.

! [Chart 4] (market-collector-lotte-images/market-collector-lotte-section10-4.svg)

```

**Sources:**[] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/
[src/main/java/com/difisoft/marketcollector/ws/WsConnection.java78-90] (https://github.com/
src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java69-106] (https://github.com/

```

#### ##### Stock and Bid/Ask Subscription

For stock quotes and bid/ask data, channels are constructed from configuration:

```

**WsConnection**[] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/
[src/main/java/com/difisoft/marketcollector/ws/WsConnection.java94-96] (https://github.com/

```

```

config.getChannels().forEach(it -> {
    channels.add(it + String.join("|", codes));
});

```

Where `config.getChannels()` returns channel prefixes like:

- `sub/pro.pub.auto.qt./`` for stock quotes
- `sub/pro.pub.auto.bo./`` for bid/ask spreads

```

**Sources:**[] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/
[src/main/java/com/difisoft/marketcollector/ws/WsConnection.java94-96] (https://github.com/
src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java110-112] (https://github.com/

* * *

```

#### #### Code Mapping System

Both implementations maintain a `codeMapping` dictionary to translate HTS security codes (

#### ##### Code Mapping Architecture

! [Chart 5] (market-collector-lotte-images/market-collector-lotte-section10-5.svg)

```

**Sources:**[] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/
[src/main/java/com/difisoft/marketcollector/ws/WsConnection.java52] (https://github.com/nhsv
src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java47] (https://github.com/nhsv

```

#### ##### Population Strategies

```

**WsConnection (Redis-based):**

```

```

// [WsConnection.java:86-89]
symbolInfos.map(item -> {
    codeMapping.put(item.getSecCode(), item.getCode());
    return item.getSecCode();
})

```

```

**WsConnection2 (API-based):**

```

```

// [WsConnection2.java:103-105]
indexList.forEach(item -> {
    codeMapping.put(item.getCode(),
        config.getCodeMapping().getOrDefault(item.getSymbol(), item.getSymbol()));
});

```

```

**Sources:**[] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/

[src/main/java/com/difisoft/marketcollector/ws/WsConnection.java86-89] (https://github.com/

src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java103-105] (https://github.com

##### Usage in Message Handlers

All message handlers use `codeMapping.getOrDefault(parts[3], parts[3])` to normalize secur:

| Handler Method | Usage Location | Purpose |
|---|---|---|
| `handleStockQuote()` | [ |
| WsConnection.java187] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/
| `handleStockBidAsk()` | [ |

WsConnection.java233] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/
| Map bid/ask security code |
|---|

WsConnection.java284] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/
| Map index security code |
|---|

WsConnection.java308] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/

**Sources:**[] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/

[src/main/java/com/difisoft/marketcollector/ws/WsConnection.java187-308] (https://github.com

src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java242-323] (https://github.com

* * *

##### Reconnection Strategy

Both implementations use identical exponential backoff logic for automatic reconnection aft

##### Reconnection Trigger Points

![Chart 6] (market-collector-lotte-images/market-collector-lotte-section10-6.svg)

**Sources:**[] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/

[src/main/java/com/difisoft/marketcollector/ws/WsConnection.java141-176] (https://github.com

src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java222-230] (https://github.com

##### Backoff Calculation

The timeout calculation implements a capped exponential backoff:

| Attempt | Index | Formula | Calculated Timeout | Actual Timeout (capped at 120s) |

```

```
|--- | --- | --- | --- | --- |  
| 1st | 0 | `0 * 2000 + 5000` | 5,000 ms | 5,000 ms |  
| 2nd | 1 | `1 * 2000 + 5000` | 7,000 ms | 7,000 ms |  
| 3rd | 2 | `2 * 2000 + 5000` | 9,000 ms | 9,000 ms |  
| 10th | 9 | `9 * 2000 + 5000` | 23,000 ms | 23,000 ms |  
| 58th | 57 | `57 * 2000 + 5000` | 119,000 ms | 119,000 ms |  
| 59th+ | 58+ | `58 * 2000 + 5000` | 121,000+ ms | **120,000 ms** |
```

```
**Implementation:**
```

```
// [WsConnection.java:113-114]
```

```
long timeout = index * 2000 + 5000;
```

```
timeout = Math.min(timeout, 120000);
```

```

**Sources:**[] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/

[src/main/java/com/difisoft/marketcollector/ws/WsConnection.java113-114] (https://github.com/

src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java129-130] (https://github.com/

* * *

#### Configuration Structure

WebSocket connections are configured via `AppConf.WebSocketConf` objects passed during cons

##### Configuration Class Hierarchy

![[Chart 7] (market-collector-lotte-images/market-collector-lotte-section10-7.svg)]

**Sources:**[] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/

[src/main/java/com/difisoft/marketcollector/ws/WsConnection.java43-44] (https://github.com/

src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java40-41] (https://github.com/

##### Constructor Dependency Injection

Both implementations receive identical constructor parameters:

| Parameter | Type | Purpose |
|---|---|---|
| `statusMap` | `Map<String, String>` | Maps status codes to session types |
| `config` | `AppConf.WebSocketConf` | WebSocket connection configuration |
| `apiConfig` | `AppConf.ApiConnection` | REST API configuration for auxiliary calls |
| `objectMapper` | `ObjectMapper` | JSON serialization/deserialization |
| `outputConsumer` | `Consumer<Object>` | Callback for processed messages |
| `codes` | `Set<String>` | Security codes to subscribe to |

**WsConnection-specific:**

- `marketRedisDao` - Redis access for symbol information[

src/main/java/com/difisoft/marketcollector/ws/WsConnection.java62] (https://github.com/

- `cacheService` - In-memory cache for symbol lookup[

src/main/java/com/difisoft/marketcollector/ws/WsConnection.java63] (https://github.com/

**Sources:**[] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/

[src/main/java/com/difisoft/marketcollector/ws/WsConnection.java56-75] (https://github.com/

src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java51-66] (https://github.com/

##### Configuration-Driven Channel Construction

The `codeType` field determines subscription strategy:

```

! [Chart 8] (market-collector-lotte-images/market-collector-lotte-section10-8.svg)

**Sources:** [ (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection.java78-96) (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java69-113) (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection3.java69-113) ]

---

### Message Processing Pipeline

Relevant source files

- [ (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection.java) (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java) (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection3.java) ]

#### Purpose and Scope

This document describes the message processing pipeline that transforms raw WebSocket messages into a structured format for the HTS Trading System. The pipeline includes message reception, routing, and transformation.

For information about WebSocket connection establishment and subscription management, see the [WebSocket Connection Management](#) document.

\* \* \*

#### Message Flow Overview

The message processing pipeline consists of three main stages: reception, routing, and transformation. The flow is illustrated in the following diagram:

! [Chart 1] (market-collector-lotte-images/market-collector-lotte-section11-1.svg)

**Sources:**

- [ (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection.java154-171) (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java188-219) (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection3.java188-219) ]

\* \* \*

#### Message Reception and Initial Processing

##### Message Format

Incoming messages from the HTS Trading System arrive as pipe-delimited strings where each field is separated by a pipe character (|). The message structure is as follows:

Example message structures:

```
-    **Index Quote**: `auto.idxqt.service
  | field1 | timestamp | code | ... | fieldN` |
|---|---|---|---|---|
  | -    **Bid/Ask**: `auto.bo.service | field1 | timestamp | code | ... | fieldN` |
  | -    **Session Event**: `auto.tickerNews.service | field1 | timestamp | code | ... | fieldN` |
```

#### #### Message Reception Points

Two WebSocket implementations handle incoming messages:

```
  | Implementation | Entry Point | Framework | Location |
|---|---|---|---|
  | `WsConnection` | `onMessage(String data)` | Jakarta WebSocket | [ |
  | WsConnection.java154] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/ |
  | `WsConnection2` | `handleMessage(WebSocketSession, WebSocketMessage)` | Spring WebSocket |
```

WsConnection2.java188] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/

Both implementations follow identical message processing logic after extracting the string

#### \*\*Sources:\*\*

```
- [
    src/main/java/com/difisoft/marketcollector/ws/WsConnection.java148-171] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/
- [
    src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java188-219] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/
```

\* \* \*

#### #### Message Routing

##### ##### Router Logic

The router examines the service prefix in `parts[0]` and dispatches to the appropriate handler.

! [Chart 2] (market-collector-lotte-images/market-collector-lotte-section11-2.svg)

##### ##### Routing Implementation

The routing logic in both `WsConnection` and `WsConnection2` uses identical service prefix

```
if (service.startsWith("auto.idxqt")) { this.handleIndexQuote(parts); } else if (service.startsWith("auto.qt")) {
this.handleStockQuote(parts); } else if (service.startsWith("auto.bo")) { this.handleStockBidAsk(parts); } else if
(service.startsWith("auto.tickerNews")) { this.handleSessionEvent(parts); }
```

```

**Sources:**

- [
    src/main/java/com/difisoft/marketcollector/ws/WsConnection.java154-171] (https://github.com/difisoft/market-collector-lotte-images/blob/master/src/main/java/com/difisoft/marketcollector/ws/WsConnection.java#L154-L171)
- [
    src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java203-219] (https://github.com/difisoft/market-collector-lotte-images/blob/master/src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java#L203-L219)

* * *

#### Message Handlers

##### handleIndexQuote

Processes market index data (VNINDEX, HNX-INDEX, UPCOM-INDEX, etc.) from messages with service prefix `auto.idx`.

**Field Mapping:**

| Field Index | Description | Data Type | Transformation |
|---|---|---|---|
| parts\[2\] | Timestamp | String → ZonedDateTime | Parse with `timeFormatter` or `lotteTimeFormatter` |
| parts\[3\] | Index code | String | Map via `codeMapping` |
| parts\[5\] | Open value | String → Double | `getDouble()` |
| parts\[7\] | High value | String → Double | `getDouble()` |
| parts\[9\] | Low value | String → Double | `getDouble()` |
| parts\[11\] | Last value | String → Double | `getDouble()` |
| parts\[13\] | Change | String → Double | `getDouble()` |
| parts\[14\] | Rate | String → Double | `getDouble()` |
| parts\[15\] | Trading volume | String → Long | `getLong()` |
| parts\[16\] | Trading value | String → Double | `getDouble()` |
| parts\[17\] | Matching volume | String → Long | `getLong()` |
| parts\[18-22\] | Market statistics | String → Integer | `getInteger()` (ceiling/up/unchanged)

**Output:** Creates a `QuoteUpdate` object with `type` set to `SymbolTypeEnum.INDEX`.

**Sources:**

- [
    src/main/java/com/difisoft/marketcollector/ws/WsConnection.java279-301] (https://github.com/difisoft/market-collector-lotte-images/blob/master/src/main/java/com/difisoft/marketcollector/ws/WsConnection.java#L279-L301)
- [
    src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java295-316] (https://github.com/difisoft/market-collector-lotte-images/blob/master/src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java#L295-L316)

* * *

##### handleStockQuote

Processes individual stock quote data from messages with service prefix `auto.qt`.

! [Chart 3] (market-collector-lotte-images/market-collector-lotte-section11-3.svg)

```

```

**Special Processing:**

- **Matched By Field** (parts\[24\]): Converted from numeric codes to string values

  - `"83"` → `"ASK"`
  - `"66"` → `"BID"`
  - Other values → `null`
- **Conditional Fields**: Total bid/offer volumes (parts\[33-34\]) are only parsed if the

**Output:** Creates a `QuoteUpdate` object with `type` populated from the corresponding `Sy

**Sources:**

- [
  src/main/java/com/difisoft/marketcollector/ws/WsConnection.java182-226] (https://github.
- [
  src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java237-272] (https://github

* * *

##### handleStockBidAsk

Processes market depth (bid/ask) data from messages with service prefix `auto.bo`. This han

! [Chart 4] (market-collector-lotte-images/market-collector-lotte-section11-4.svg)

**Order Book Construction:**

The handler parses 10 price levels starting at field index 13, advancing by 6 fields per le

```

Level 1: parts[13-18] → Bid Price, Bid Vol, Offer Price, Offer Vol Level 2: parts[19-24] → Bid Price, Bid Vol, Offer Price, Offer Vol ... Level 10: parts[67-72] → Bid Price, Bid Vol, Offer Price, Offer Vol

**\*\*Session Type Mapping (WsConnection only):\*\***

The control code in parts\[4\] is mapped differently for HOSE vs. HNX/UPCOM exchanges:

|  | Control Code | HOSE Session | HNX/UPCOM Session |
|--|--------------|--------------|-------------------|
|  | P            | ATO          | LO                |
|  | O            | LO           | LO                |
|  | R            | LO           | \-                |
|  | 2            | \-           | INTERMISSION      |
|  | I            | INTERMISSION | \-                |
|  | A            | ATC          | ATC               |
|  | C            | PLO          | PLO               |
|  | K, G         | CLOSED       | \-                |
|  | 13, 97       | \-           | CLOSED            |

**\*\*Cleanup Logic:\*\***

After parsing all levels, the handler removes trailing empty items by iterating in reverse

[WsConnection.java265-271] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4c>)

**\*\*Output:\*\*** Creates a `BidOfferUpdate` object containing the list of `BidOfferItem` objects

**\*\*Sources:\*\***

```
- [
    src/main/java/com/difisoft/marketcollector/ws/WsConnection.java228-277] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4c)
- [
    src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java274-293] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4c)
```

\* \* \*

##### handleSessionEvent

Processes market session status events from messages with service prefix `auto.tickerNews`.

**\*\*Field Mapping:\*\***

|  | Field Index | Description        | Transformation            |
|--|-------------|--------------------|---------------------------|
|  | parts\[2\]  | Timestamp          | Parse with time formatter |
|  | parts\[3\]  | Market/Symbol code | Map via `codeMapping`     |
|  | parts\[9\]  | Event title        | Raw string                |

**\*\*Status Parsing:\*\***

The event title is passed to `MarketStatusData.parseStatus(statusMap)` which uses the `stat

**\*\*Output:\*\*** Creates a `MarketStatusData` object containing the timestamp, code, title, and

```

**Sources:**

- [
    src/main/java/com/difisoft/marketcollector/ws/WsConnection.java303-312] (https://github.com
- [
    src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java318-327] (https://github.com

* * *

#### Data Transformation Utilities

#### Timestamp Conversion

Both implementations convert timestamps from Vietnam timezone (Asia/Ho_Chi_Minh) to UTC :

| Format | Pattern | Usage | Example |
|---|---|---|---|
| `timeFormatter` | HHmmss | WsConnection2 | 093045 → 09:30:45 |
| `lotteTimeFormatter` | Hmmss | WsConnection | 93045 → 09:30:45 |

The conversion process:

1. Parse time string to `LocalTime`
2. Combine with current `LocalDate`
3. Convert to `ZonedDateTime` in Vietnam timezone
4. Convert to UTC (when needed)
5. Format as HHmmss string

**Sources:**

- [
    src/main/java/com/difisoft/marketcollector/ws/WsConnection.java39-40] (https://github.com
- [
    src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java37] (https://github.com,

* * *

#### Code Mapping

The `codeMapping` dictionary translates HTS-specific security codes to standardized codes :

- **For INDEX connections**: Populated from Redis `SymbolInfo` cache or Lotte API index
- **For STOCK connections**: Uses codes directly from subscription list

The mapping is applied consistently across all handlers:

```

```
String code = parts[3]; code = codeMapping.getOrDefault(code, code);
```

If a code is not found in the mapping, the original code is used.

**\*\*Sources:\*\***

- [
  - src/main/java/com/difisoft/marketcollector/ws/WsConnection.java52-89] (<https://github.com>)
- [
  - src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java47-105] (<https://github.com>)

\* \* \*

#### ##### Type Conversion Functions

Three utility functions handle safe conversion from string values with default fallback:

![Chart 5] (market-collector-lotte-images/market-collector-lotte-section11-5.svg)

**\*\*Key Differences Between Implementations:\*\***

| Function     | WsConnection                                          | WsConnection2            |
|--------------|-------------------------------------------------------|--------------------------|
| getDouble()  | Rounds to 4 decimal places using `HALF_EVEN` rounding | No rounding applied      |
| getLong()    | Standard Long parsing                                 | Standard Long parsing    |
| getInteger() | Standard Integer parsing                              | Standard Integer parsing |

**\*\*Sources:\*\***

- [
  - src/main/java/com/difisoft/marketcollector/ws/WsConnection.java314-346] (<https://github.com>)
- [
  - src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java329-360] (<https://github.com>)
- [
  - src/main/java/com/difisoft/marketcollector/ws/WsConnection.java38] (<https://github.com>)

\* \* \*

#### ##### Output Distribution

#### ##### Output Consumer Interface

All processed data models are passed to a single `Consumer<Object>` interface called `outputConsumer`.

![Chart 6] (market-collector-lotte-images/market-collector-lotte-section11-6.svg)

The `outputConsumer` is provided during construction of `WsConnection` or `WsConnection2` as follows:

```
outputConsumer.accept(quoteUpdate); outputConsumer.accept(bidAsk);
outputConsumer.accept(marketStatusData);
```

**\*\*Output Locations in Code:\*\***

```

    | Handler | WsConnection | WsConnection2 |
|---|---|---|
    | `handleStockQuote()` | [ |
    | Line 225] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/Line 225)

Line 271] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/Line 271) |
    | `handleStockBidAsk()` | [

Line 276] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/Line 276) |

Line 292] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/Line 292) |
    | `handleIndexQuote()` | [

Line 300] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/Line 300) |

Line 315] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/Line 315) |
    | `handleSessionEvent()` | [

Line 311] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/Line 311) |

Line 326] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/Line 326) |

```

**\*\*Sources:\*\***

```

- [

    src/main/java/com/difisoft/marketcollector/ws/WsConnection.java46-311] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection.java#L46-L311)

- [

    src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java43-326] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java#L43-L326)

```

\* \* \*

**#### Processing Pipeline Summary**

The complete message processing pipeline can be summarized as follows:

1. **\*\*Reception\*\***: WebSocket message arrives as pipe-delimited string
2. **\*\*Validation\*\***: JSON messages (ping/pong) are filtered out
3. **\*\*Splitting\*\***: Message is split by `|` delimiter into parts array
4. **\*\*Routing\*\***: Service prefix (parts[0]) determines handler method
5. **\*\*Parsing\*\***: Handler extracts and parses relevant fields
6. **\*\*Transformation\*\***: String values converted to appropriate types (Double, Long, Integer)
7. **\*\*Code Mapping\*\***: Security codes mapped from HTS format to standardized format
8. **\*\*Enrichment\*\***: Additional data (like SymbolType) added from cache
9. **\*\*Model Construction\*\***: Typed data model created (QuoteUpdate, BidOfferUpdate, MarketStatusUpdate)
10. **\*\*Output\*\***: Model passed to `outputConsumer` for distribution

This pipeline handles four distinct message types, each with its own field structure and business logic.

**\*\*Sources:\*\***

```

- [
    src/main/java/com/difisoft/marketcollector/ws/WsConnection.java154-312] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection.java154-312)
- [
    src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java188-327] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java188-327)

---

### Connection Management & Recovery

Relevant source files

- [
    src/main/java/com/difisoft/marketcollector/ws/WsConnection.java] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection.java)
- [
    src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java)

#### Purpose & Scope

This document details the connection management and automatic recovery mechanisms implemented in the market collector system.

For information about the WebSocket connection initialization and channel subscription mechanism, see the [WebSocket Connection Initialization and Channel Subscription Mechanism] (src/main/java/com/difisoft/marketcollector/ws/WsConnection.java) and [WebSocket Connection Initialization and Channel Subscription Mechanism] (src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java).

* * *

#### Connection Lifecycle Overview

The system maintains persistent WebSocket connections to the HTS trading system. Both implementations follow a similar lifecycle:

1. [Chart 1] (market-collector-lotte-images/market-collector-lotte-section12-1.svg)

**WsConnection Session Management:**

- Session stored in `session` field[
    src/main/java/com/difisoft/marketcollector/ws/WsConnection.java53] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection.java53)
- Jakarta WebSocket `Session` object lifecycle
- Managed through `@OnOpen`, `@OnClose`, `@OnError` callbacks

**WsConnection2 Session Management:**

- Session stored in `session` field[
    src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java48] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java48)
- Spring `WebSocketSession` object lifecycle
- Managed through `WebSocketHandler` interface methods

**Sources:** [
    src/main/java/com/difisoft/marketcollector/ws/WsConnection.java1-348] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection.java1-348)
    [src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java1-327] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java1-327)

```

```

src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java-361] (https://github.com/

* * *

#### Automatic Reconnection Strategy

Both implementations provide automatic reconnection when connections fail or are terminated

| Trigger Scenario | WsConnection Handler | WsConnection2 Handler |
|---|---|---|
| Initial connection failure | `startWebsocket()` exception handler | `startWebsocket()` e
| Connection closed unexpectedly | `onClose()` callback | Not implemented - logs only |
| Transport/protocol error | `onError()` logs only | `handleTransportError()` |

##### Reconnection Entry Points

![[Chart 2](market-collector-lotte-images/market-collector-lotte-section12-2.svg)]

**WsConnection Close Handler:**

```

Hii

```

The `onClose()` method immediately triggers reconnection by calling `startWebsocket(0)`, re

**WsConnection2 Error Handler:**

```

Hii

```

The `handleTransportError()` method triggers reconnection by calling `startWebSocket(0)`.

**Sources:**[] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main
[src/main/java/com/difisoft/marketcollector/ws/WsConnection.java#L101-123] (https://github.com
src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java#L117-139] (https://github.com

* * *

#### Exponential Backoff Algorithm

Both implementations use identical exponential backoff logic to avoid overwhelming the serv

##### Backoff Calculation

![[Chart 3](market-collector-lotte-images/market-collector-lotte-section12-3.svg)]

| Retry Attempt | Calculation | Timeout (ms) | Timeout (seconds) |
|---|---|---|---|
| 0 (initial) | 0 * 2000 + 5000 | 5000 | 5s |
| 1 | 1 * 2000 + 5000 | 7000 | 7s |
| 2 | 2 * 2000 + 5000 | 9000 | 9s |
| 3 | 3 * 2000 + 5000 | 11000 | 11s |
| ... | ... | ... | ... |
| 10 | 10 * 2000 + 5000 | 25000 | 25s |
| 20 | 20 * 2000 + 5000 | 45000 | 45s |
| 50 | 50 * 2000 + 5000 | 105000 | 105s |
| 58+ | 58 * 2000 + 5000 | 120000 (capped) | 120s (max) |

##### Implementation Details

**WsConnection Implementation:**

```

Hii

```

**WsConnection2 Implementation:**

```

Hii

## Key characteristics:

- **Initial delay:** 5 seconds (provides immediate retry after transient failures)
- **Increment:** 2 seconds per retry (gradual backoff)
- **Maximum delay:** 120 seconds (prevents indefinite growth)
- **No maximum attempts:** System retries indefinitely until successful

**Sources:** [] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection.java#L13-114>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java#L129-130>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection3.java#L131-142>)

\* \* \*

## #### Error Handling Mechanisms

## ##### WsConnection Error Handling

![Chart 4] (<market-collector-lotte-images/market-collector-lotte-section12-4.svg>)

**Connection Failure Handling:**

- Catches `DeploymentException` and `IOException` [<src/main/java/com/difisoft/marketcollector/ws/WsConnection.java#L112>] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection.java#L112>)
- Logs error with connection name, URL, and scheduled retry timeout [<src/main/java/com/difisoft/marketcollector/ws/WsConnection.java#L115>] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection.java#L115>)
- Applies exponential backoff before retry [<src/main/java/com/difisoft/marketcollector/ws/WsConnection.java#L116-121>] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection.java#L116-121>)
- Handles `InterruptedException` by preserving interrupt status [<src/main/java/com/difisoft/marketcollector/ws/WsConnection.java#L119-121>] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection.java#L119-121>)

**Close Event Handling:**

- Receives `CloseReason` parameter with close code and phrase [<src/main/java/com/difisoft/marketcollector/ws/WsConnection.java#L142>] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection.java#L142>)
- Logs connection name and close reason [<src/main/java/com/difisoft/marketcollector/ws/WsConnection.java#L143>] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection.java#L143>)
- Sets session to null [<src/main/java/com/difisoft/marketcollector/ws/WsConnection.java#L144>] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection.java#L144>)
- Immediately triggers reconnection with index 0 [<src/main/java/com/difisoft/marketcollector/ws/WsConnection.java#L145>] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/ws/WsConnection.java#L145>)

**Error Callback:**

```

-   Receives `Throwable` parameter[

    src/main/java/com/difisoft/marketcollector/ws/WsConnection.java174] (https://github.com,
-   Logs error with connection name[

    src/main/java/com/difisoft/marketcollector/ws/WsConnection.java175] (https://github.com,
-   **Does not trigger automatic reconnection** (relies on `onClose` to follow)

**Sources:**[] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main

[src/main/java/com/difisoft/marketcollector/ws/WsConnection.java101-123] (https://github.com,

src/main/java/com/difisoft/marketcollector/ws/WsConnection.java141-146] (https://github.com,

src/main/java/com/difisoft/marketcollector/ws/WsConnection.java173-176] (https://github.com,

* * *

##### WsConnection2 Error Handling

![[Chart 5] (market-collector-lotte-images/market-collector-lotte-section12-5.svg)

**Connection Failure Handling:**

-   Catches `IOException` from `WebSocketClient.execute()` [

    src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java128] (https://github.com,
-   Logs error with connection name, URL, and scheduled retry timeout[

    src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java131] (https://github.com,
-   Applies exponential backoff before retry[

    src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java132-137] (https://github.com,
-   Handles `InterruptedException` by preserving interrupt status[

    src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java135-137] (https://github.com,

**Transport Error Handling:**

-   Receives `Throwable` exception parameter[

    src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java222] (https://github.com,
-   Logs error with "retrying..." message[

    src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java223] (https://github.com,
-   Immediately triggers reconnection with index 0[

    src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java224] (https://github.com,

**Connection Closed Handling:**

-   Receives `CloseStatus` parameter with close code and reason[

    src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java228] (https://github.com,

```

```

- Logs closure as warning[

    src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java229] (https://github.com
- **Does not trigger automatic reconnection** (application must explicitly restart)

**Sources:**[] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main

[src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java117-139] (https://github.co

src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java222-225] (https://github.com

src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java228-230] (https://github.com

* * *

#### Session State Management

Both implementations manage WebSocket session state to track active connections and ensure

##### Session Lifecycle

![[Chart 6] (market-collector-lotte-images/market-collector-lotte-section12-6.svg)

##### Session Cleanup

**WsConnection Session Cleanup:**

- Before reconnection, closes existing session if not null[

    src/main/java/com/difisoft/marketcollector/ws/WsConnection.java102-108] (https://github.com
- Sets session to null in `onClose()`[

    src/main/java/com/difisoft/marketcollector/ws/WsConnection.java144] (https://github.com,
- Catches and logs `IOException` during close[

    src/main/java/com/difisoft/marketcollector/ws/WsConnection.java105-106] (https://github.com

**WsConnection2 Session Cleanup:**

- Before reconnection, closes existing session if not null[

    src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java118-124] (https://github.com
- Uses try-with-resources for automatic session cleanup[

    src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java126] (https://github.com
- Catches and logs `IOException` during close[

    src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java121-123] (https://github.com

**Sources:**[] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main

[src/main/java/com/difisoft/marketcollector/ws/WsConnection.java102-108] (https://github.com

src/main/java/com/difisoft/marketcollector/ws/WsConnection.java144] (https://github.com/nhsv

```

```

src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java118-124] (https://github.com
* * *

#### Channel Subscription Management

After successful connection establishment, both implementations send subscription messages

##### WsConnection Subscription (Immediate)

! [Chart 7] (market-collector-lotte-images/market-collector-lotte-section12-7.svg)

The `@OnOpen` callback sends subscription messages immediately upon connection[] (https://g:

[src/main/java/com/difisoft/marketcollector/ws/WsConnection.java129-132] (https://github.com

**Sources:**[] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main

[src/main/java/com/difisoft/marketcollector/ws/WsConnection.java125-133] (https://github.com

* * *

##### WsConnection2 Subscription (Delayed)

! [Chart 8] (market-collector-lotte-images/market-collector-lotte-section12-8.svg)

The `afterConnectionEstablished()` method schedules subscription messages with a 500ms dela

[src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java171-184] (https://github.co

**Sources:**[] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main

[src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java170-185] (https://github.co

* * *

#### Implementation Comparison

| Feature | WsConnection | WsConnection2 |
|---|---|---|
| **WebSocket API** | Jakarta WebSocket (`@ClientEndpoint`) | Spring WebSocket (`WebSocket
| **Connection Method** | `ContainerProvider.getWebSocketContainer().connectToServer()` |
| **Session Type** | `jakarta.websocket.Session` | `org.springframework.web.socket.WebSoc
| **Lifecycle Callbacks** | Annotations: `@OnOpen`, `@OnClose`, `@OnMessage`, `@OnError`
| **Subscription Timing** | Immediate (in `@OnOpen`) | Delayed 500ms (via `Timer`) |
| **Reconnect on Close** | Yes (automatic in `onClose()`) | No (logs only) |
| **Reconnect on Error** | No (`onError()` logs only) | Yes (automatic in `handleTransport
| **Backoff Algorithm** | Identical exponential backoff | Identical exponential backoff |
| **Session Cleanup** | Manual null assignment | Try-with-resources |

##### Key Behavioral Differences

**Reconnection Triggers:**

```

```

- `WsConnection` automatically reconnects on clean close events
- `WsConnection2` automatically reconnects only on transport errors
- Both reconnect on initial connection failures with exponential backoff

**Subscription Delivery:**

- `WsConnection` sends subscriptions synchronously in connection callback
- `WsConnection2` delays subscriptions by 500ms to allow server initialization

**Error Callback Behavior:**

- `WsConnection` `@OnError` does not trigger reconnection (expects `@OnClose` to follow)
- `WsConnection2` `handleTransportError()` immediately triggers reconnection

**Sources:**[] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/
[src/main/java/com/difisoft/marketcollector/ws/WsConnection.java#L348] (https://github.com/
src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java#L361] (https://github.com/

* * *

#### Configuration Dependencies

Connection management behavior is configured through `AppConf.WebSocketConf` objects passed

#### Required Configuration Fields

| Field | Type | Purpose | Usage |
|---|---|---|---|
| `name` | String | Connection identifier | Logging and error messages |
| `url` | String | WebSocket endpoint URL | Connection target[ |
| WsConnection111] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/WsC
WsConnection2126] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/WsC

| `codeType` | String | Market data type filter | Determines channel subscription logic |
|---|---|---|---|
| `codeMapping` | Map<String, String> | Code translation map | Maps server codes to client

**Configuration Source:**

- Both classes receive `AppConf.WebSocketConf` config` in constructor[

src/main/java/com/difisoft/marketcollector/ws/WsConnection.java#L58] (https://github.com/
src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java#L53] (https://github.com/
- Configuration loaded from `AppConf` central configuration (see [Configuration Managemer

**Sources:**[] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/
[src/main/java/com/difisoft/marketcollector/ws/WsConnection.java#L43-75] (https://github.com/

```

```

src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java40-66] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/WsConnection2.java)

---

### Data Output & Distribution

Relevant source files

- [
    src/main/java/com/difisoft/marketcollector/ws/WsConnection.java] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/WsConnection.java)
- [
    src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/WsConnection2.java)

#### Purpose and Scope

This document describes how processed market data is output from the WebSocket connection. For details on how downstream systems (Kafka, Redis, MongoDB) consume and store this data, see the [Data Output & Distribution] section.

* * *

#### Output Consumer Abstraction

Both WebSocket implementations ([WsConnection] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/WsConnection.java) and [WsConnection2] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/WsConnection2.java)) implement the [OutputConsumer] interface.

#### Consumer Interface Pattern

![[Chart 1] (market-collector-lotte-images/market-collector-lotte-section13-1.svg)]

**Sources:**

- [
    src/main/java/com/difisoft/marketcollector/ws/WsConnection.java46] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/WsConnection.java#L46)
- [
    src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java43] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/WsConnection2.java#L43)

#### Constructor Injection

The `outputConsumer` is injected during WebSocket connection construction, enabling runtime dependency injection.

| WebSocket Class | Consumer Field | Constructor Parameter | Invocation Sites |
|---|---|---|---|
| `WsConnection` | Line 46 | Line 61 | Lines 225, 276, 300, 311 |
| `WsConnection2` | Line 43 | Line 56 | Lines 271, 292, 315, 326 |

```

The consumer is declared as a final field to ensure immutability:

```
private final Consumer outputConsumer;
```

**\*\*Sources:\*\***

- [
  - src/main/java/com/difisoft/marketcollector/ws/WsConnection.java46] (<https://github.com/>)
- [
  - src/main/java/com/difisoft/marketcollector/ws/WsConnection.java56-75] (<https://github.com/>)
- [
  - src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java43] (<https://github.com/>)
- [
  - src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java51-66] (<https://github.com/>)

\* \* \*

#### #### Data Models Output

The WebSocket handlers produce three primary data model types, each corresponding to different

#### ##### Output Data Model Categories

![Chart 2] (market-collector-lotte-images/market-collector-lotte-section13-2.svg)

**\*\*Sources:\*\***

- [
  - src/main/java/com/difisoft/marketcollector/ws/WsConnection.java182-226] (<https://github.com/>)
- [
  - src/main/java/com/difisoft/marketcollector/ws/WsConnection.java228-277] (<https://github.com/>)
- [
  - src/main/java/com/difisoft/marketcollector/ws/WsConnection.java279-301] (<https://github.com/>)
- [
  - src/main/java/com/difisoft/marketcollector/ws/WsConnection.java303-312] (<https://github.com/>)

#### ##### QuoteUpdate Model

The `QuoteUpdate` model represents both stock and index price quotes. It contains real-time

#### ##### Stock Quote Fields

- **\*\*Timestamp fields\*\*:** `time`, `highTime`, `lowTime`
- **\*\*Price data\*\*:** `open`, `high`, `low`, `last`, `change`, `rate`
- **\*\*Volume data\*\*:** `tradingVolume`, `matchingVolume`, `totalBidVolume`, `totalOfferVolume`
- **\*\*Turnover data\*\*:** `tradingValue`, `turnoverRate`, `averagePrice`, `referencePrice`
- **\*\*Foreign trading\*\*:** `foreignerBuyVolume`, `foreignerSellVolume`, `foreignerCurrentRoom`
- **\*\*Metadata\*\*:** `code`, `type`, `matchedBy`

## ##### Index Quote Fields

- **\*\*Timestamp\*\***: `time`
- **\*\*OHLC prices\*\***: `open`, `high`, `low`, `last`, `change`, `rate`
- **\*\*Volume/value\*\***: `tradingVolume`, `tradingValue`, `matchingVolume`
- **\*\*Market breadth\*\***: `ceilingCount`, `upCount`, `unchangedCount`, `downCount`, `floorCount`
- **\*\*Metadata\*\***: `code`, `type` (set to `SymbolTypeEnum.INDEX`)

**\*\*Sources\*\***

- [
  - src/main/java/com/difisoft/marketcollector/ws/WsConnection.java182-226] (<https://github.com/difisoft/marketcollector/blob/master/src/main/java/com/difisoft/marketcollector/ws/WsConnection.java#L182-L226>)
- [
  - src/main/java/com/difisoft/marketcollector/ws/WsConnection.java279-301] (<https://github.com/difisoft/marketcollector/blob/master/src/main/java/com/difisoft/marketcollector/ws/WsConnection.java#L279-L301>)
- [
  - src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java237-272] (<https://github.com/difisoft/marketcollector/blob/master/src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java#L237-L272>)
- [
  - src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java295-316] (<https://github.com/difisoft/marketcollector/blob/master/src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java#L295-L316>)

## ##### BidOfferUpdate Model

The `BidOfferUpdate` model represents order book depth data (level 2 market data), showing

## ##### Fields

- **\*\*Timestamp\*\***: `time`
- **\*\*Symbol\*\***: `code`
- **\*\*Session indicator\*\***: `session` (mapped from control codes)
- **\*\*Expected price\*\***: `expectedPrice`
- **\*\*Order book\*\***: `bidOfferList` (list of `BidOfferItem`)
- **\*\*Totals\*\***: `totalBidVolume`, `totalOfferVolume`, `totalBidCount`, `totalOfferCount`

## ##### BidOfferItem Structure

Each item in `bidOfferList` contains:

- `bidPrice`, `bidVolume`
- `offerPrice`, `offerVolume`

The list is constructed from pipe-delimited message fields starting at position 13, with 6

**\*\*Sources\*\***

- [
  - src/main/java/com/difisoft/marketcollector/ws/WsConnection.java228-277] (<https://github.com/difisoft/marketcollector/blob/master/src/main/java/com/difisoft/marketcollector/ws/WsConnection.java#L228-L277>)
- [
  - src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java274-293] (<https://github.com/difisoft/marketcollector/blob/master/src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java#L274-L293>)

#### #### MarketStatusData Model

The `MarketStatusData` model represents market session events and status changes.

#### ##### Fields

- **\*\*Timestamp\*\***: `time`
- **\*\*Symbol\*\***: `code`
- **\*\*Event title\*\***: `title` (from message position 9)
- **\*\*Parsed status\*\***: Extracted via `parseStatus(statusMap)` method

**\*\*Sources\*\***

- [
  - src/main/java/com/difisoft/marketcollector/ws/WsConnection.java303-312] (<https://github.com/difisoft/market-collector-lotte-images/blob/master/src/main/java/com/difisoft/marketcollector/ws/WsConnection.java#L303-L312>)
- [
  - src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java318-327] (<https://github.com/difisoft/market-collector-lotte-images/blob/master/src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java#L318-L327>)

\* \* \*

#### #### Output Processing Flow

The following sequence diagram illustrates the complete flow from message reception to data

#### ##### Stock Quote Output Sequence

![Chart 3](market-collector-lotte-images/market-collector-lotte-section13-3.svg)

**\*\*Sources\*\***

- [
  - src/main/java/com/difisoft/marketcollector/ws/WsConnection.java182-226] (<https://github.com/difisoft/market-collector-lotte-images/blob/master/src/main/java/com/difisoft/marketcollector/ws/WsConnection.java#L182-L226>)
- [
  - src/main/java/com/difisoft/marketcollector/ws/WsConnection.java314-346] (<https://github.com/difisoft/market-collector-lotte-images/blob/master/src/main/java/com/difisoft/marketcollector/ws/WsConnection.java#L314-L346>)

#### ##### Handler Invocation Points

Each message handler invokes the output consumer exactly once after model construction and

| Handler Method         | Output Line | Model Type         | WsConnection | WsConnection2 |
|------------------------|-------------|--------------------|--------------|---------------|
| `handleStockQuote()`   | 225 / 271   | `QuoteUpdate`      | Line 225     | Line 271      |
| `handleStockBidAsk()`  | 276 / 292   | `BidOfferUpdate`   | Line 276     | Line 292      |
| `handleIndexQuote()`   | 300 / 315   | `QuoteUpdate`      | Line 300     | Line 315      |
| `handleSessionEvent()` | 311 / 326   | `MarketStatusData` | Line 311     | Line 326      |

**\*\*Sources\*\***

- [

```

src/main/java/com/difisoft/marketcollector/ws/WsConnection.java225] (https://github.com,
- [

src/main/java/com/difisoft/marketcollector/ws/WsConnection.java276] (https://github.com,
- [

src/main/java/com/difisoft/marketcollector/ws/WsConnection.java300] (https://github.com,
- [

src/main/java/com/difisoft/marketcollector/ws/WsConnection.java311] (https://github.com,
- [

src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java271] (https://github.com,
- [

src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java292] (https://github.com,
- [

src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java315] (https://github.com,
- [

src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java326] (https://github.com,

* * *

#### Code Mapping Integration

Both WebSocket implementations maintain a `codeMapping` structure to translate HTS security

##### Code Mapping Process

![Chart 4] (market-collector-lotte-images/market-collector-lotte-section13-4.svg)

**Sources:**

- [

src/main/java/com/difisoft/marketcollector/ws/WsConnection.java52] (https://github.com,
- [

src/main/java/com/difisoft/marketcollector/ws/WsConnection.java77-98] (https://github.com,
- [

src/main/java/com/difisoft/marketcollector/ws/WsConnection.java187] (https://github.com,
- [

src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java47] (https://github.com,
- [

src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java68-114] (https://github,
- [

src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java242] (https://github.com,

```

```
##### Mapping Application
```

```
In every handler, the raw code from the HTS message is translated using:
```

```
code = codeMapping.getOrDefault(code, code);
```

This pattern ensures that if a mapping exists, the standardized code is used; otherwise, the

```

**Sources:**

- [
    src/main/java/com/difisoft/marketcollector/ws/WsConnection.java187] (https://github.com,
- [
    src/main/java/com/difisoft/marketcollector/ws/WsConnection.java233] (https://github.com,
- [
    src/main/java/com/difisoft/marketcollector/ws/WsConnection.java284] (https://github.com,
- [
    src/main/java/com/difisoft/marketcollector/ws/WsConnection.java308] (https://github.com,

* * *

#### Session Type Mapping (BidOfferUpdate Only)

The `handleStockBidAsk()` method in `WsConnection` includes logic to map control codes to s

##### Session Type Mapping Logic

![[Chart 5](market-collector-lotte-images/market-collector-lotte-section13-5.svg)]

##### Session Type Codes

| Control Code | HOSE Session | HNX/UPCOM Session | SessionType Enum |
|---|---|---|---|
| P | ATO | LO | `SessionType.ATO` / `SessionType.LO` |
| O, R | LO | LO | `SessionType.LO` |
| I | INTERMISSION | \- | `SessionType.INTERMISSION` |
| 2 | \- | INTERMISSION | `SessionType.INTERMISSION` |
| A | ATC | ATC | `SessionType.ATC` |
| C | PLO | PLO | `SessionType.PLO` |
| K, G | CLOSED | \- | `SessionType.CLOSED` |
| 13, 97 | \- | CLOSED | `SessionType.CLOSED` |

**Note:** This session mapping is only present in `WsConnection` ([](https://github.com/nhs
[lines 236-254] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/lines 2

**Sources:**

- [
    src/main/java/com/difisoft/marketcollector/ws/WsConnection.java236-254] (https://github

* * *

#### Data Enrichment

```

Before outputting data models, handlers enrich them with metadata retrieved from the `Cache`

##### Symbol Type Enrichment (WsConnection Only)

In `WsConnection.handleStockQuote()`, the symbol type is retrieved from cached symbol info:

```
SymbolInfo symbolInfo = cacheService.getMapSymbolInfo().get(code);  
// ... field population ...  
quoteUpdate.setType(symbolInfo.getType());
```

This ensures that `QuoteUpdate` models carry the correct `SymbolTypeEnum` (STOCK, FUND, ETI

**\*\*Index quotes\*\*** explicitly set the type:

```
quoteUpdate.setType(SymbolTypeEnum.INDEX);
```

**\*\*Sources:\*\***

- [ [src/main/java/com/difisoft/marketcollector/ws/WsConnection.java188-191](https://github.com/difisoft/marketcollector/blob/main/src/main/java/com/difisoft/marketcollector/ws/WsConnection.java#L188-191)] (<https://github.com/difisoft/marketcollector/blob/main/src/main/java/com/difisoft/marketcollector/ws/WsConnection.java#L188-191>)
- [ [src/main/java/com/difisoft/marketcollector/ws/WsConnection.java224](https://github.com/difisoft/marketcollector/blob/main/src/main/java/com/difisoft/marketcollector/ws/WsConnection.java#L224)] (<https://github.com/difisoft/marketcollector/blob/main/src/main/java/com/difisoft/marketcollector/ws/WsConnection.java#L224>)
- [ [src/main/java/com/difisoft/marketcollector/ws/WsConnection.java299](https://github.com/difisoft/marketcollector/blob/main/src/main/java/com/difisoft/marketcollector/ws/WsConnection.java#L299)] (<https://github.com/difisoft/marketcollector/blob/main/src/main/java/com/difisoft/marketcollector/ws/WsConnection.java#L299>)

#### ##### Differences Between Implementations

|  | Feature                | WsConnection        | WsConnection2 |
|--|------------------------|---------------------|---------------|
|  | Symbol type enrichment | Yes (line 224)      | No            |
|  | Session type mapping   | Yes (lines 236-254) | No            |
|  | Symbol info lookup     | Via `CacheService`  | Not used      |
|  | Matched-by mapping     | Yes (line 213)      | No            |
|  | Foreign room fields    | Yes (lines 216-217) | No            |

`WsConnection` provides more complete data enrichment, while `WsConnection2` focuses on raw data.

**\*\*Sources:\*\***

- [ [src/main/java/com/difisoft/marketcollector/ws/WsConnection.java182-226](https://github.com/difisoft/marketcollector/blob/main/src/main/java/com/difisoft/marketcollector/ws/WsConnection.java#L182-226)] (<https://github.com/difisoft/marketcollector/blob/main/src/main/java/com/difisoft/marketcollector/ws/WsConnection.java#L182-226>)
- [ [src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java237-272](https://github.com/difisoft/marketcollector/blob/main/src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java#L237-272)] (<https://github.com/difisoft/marketcollector/blob/main/src/main/java/com/difisoft/marketcollector/ws/WsConnection2.java#L237-272>)

\* \* \*

#### ##### Output Volume and Frequency

The volume of data output depends on market activity and the number of symbols subscribed to.

#### ##### Expected Data Rates

|  | Data Type                | Update Frequency      | Symbols     | Estimated Rate   |
|--|--------------------------|-----------------------|-------------|------------------|
|  | `QuoteUpdate` (Stock)    | Per trade execution   | 1000-2000   | 100-1000 msg/sec |
|  | `BidOfferUpdate` (Stock) | Per order book change | 1000-2000   | 500-5000 msg/sec |
|  | `QuoteUpdate` (Index)    | Per index calculation | 10-20       | 1-10 msg/sec     |
|  | `MarketStatusData`       | Per session event     | Market-wide | 1-20 msg/hour    |

These rates are approximate and vary significantly based on:

- Trading volume during market hours
- Volatility and order activity
- Number of symbols in subscription list

- Exchange-specific update frequencies

**\*\*High-frequency periods\*\*** (market open, close) can generate peak rates 5-10x higher than average

**\*\*Sources:\*\*** Inferred from WebSocket message handling architecture and Vietnamese market clock

\* \* \*

#### #### Error Handling in Output

The WebSocket handlers do not explicitly handle exceptions during `outputConsumer.accept()`

#### #### Null Safety

Handlers perform null checks before outputting:

- In `handleStockQuote()`, if `symbolInfo` is null, an error is logged and the quote may be dropped
- Parsed numeric fields use default values (null) for unparseable data, allowing partial processing

#### #### Data Validation

No schema validation occurs at the output layer. Models are constructed with whatever data is received

- Null values for missing or unparseable fields
- Potentially inconsistent state if message format changes

Downstream consumers must implement defensive programming to handle incomplete or malformed data

**\*\*Sources:\*\***

- [

`src/main/java/com/difisoft/marketcollector/ws/WsConnection.java`188-191] (<https://github.com/difisoft/marketcollector/blob/main/src/main/java/com/difisoft/marketcollector/ws/WsConnection.java#L188-L191>)

- [

`src/main/java/com/difisoft/marketcollector/ws/WsConnection.java`314-346] (<https://github.com/difisoft/marketcollector/blob/main/src/main/java/com/difisoft/marketcollector/ws/WsConnection.java#L314-L346>)

\* \* \*

#### #### Summary

The data output and distribution system in the Market Collector Lotte application uses a clean architecture

1. **\*\*Abstraction Layer\*\***: The `Consumer<Object> outputConsumer` interface decouples WebSocket handlers from the output system
2. **\*\*Three Model Types\*\***: `QuoteUpdate`, `BidOfferUpdate`, and `MarketStatusData` represent different data types
3. **\*\*Enrichment\*\***: `WsConnection` enriches data with symbol types and session mappings; `WsHandler` handles parsing
4. **\*\*Code Mapping\*\***: Both implementations translate HTS security codes to standardized symbols
5. **\*\*Single Output Point\*\***: All handlers call `outputConsumer.accept()` exactly once per message

This architecture enables flexible downstream processing by allowing the consumer implementation to change

---

## ## Data Models

### ### Overview

Relevant source files

- [src/main/java/com/difisoft/marketcollector/constants/Constants.java] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/constants/Constants.java>)
- [src/main/java/com/difisoft/marketcollector/constants/MappingHTS.java] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/constants/MappingHTS.java>)
- [src/main/java/com/difisoft/marketcollector/model/db/Symbol.java] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/db/Symbol.java>)

This document provides a comprehensive overview of the data structures used throughout the Market Collector Lotte system. The Market Collector Lotte system uses several categories of data models to handle Vietnamese market data.

### #### Model Categories Overview

The system's data models are organized into four main categories, each serving distinct purposes:

! [Chart 1] (market-collector-lotte-images/market-collector-lotte-section14-1.svg)

**Sources:** [src/main/java/com/difisoft/marketcollector/constants/Constants.java1-37] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/constants/Constants.java#L1-37>)  
[src/main/java/com/difisoft/marketcollector/model/db/Symbol.java1-125] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/db/Symbol.java#L1-125>)  
[src/main/java/com/difisoft/marketcollector/model/lotte/api/CommonResponse.java1-14] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/lotte/api/CommonResponse.java#L1-14>)

### #### Data Flow and Relationships

The data models interact in a structured flow from external data sources through transformation and processing to provide market insights.

! [Chart 2] (market-collector-lotte-images/market-collector-lotte-section14-2.svg)

**Sources:** [src/main/java/com/difisoft/marketcollector/model/db/Symbol.java60-85] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/db/Symbol.java#L60-85>)  
[src/main/java/com/difisoft/marketcollector/constants/Constants.java12-34] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/constants/Constants.java#L12-34>)  
[src/main/java/com/difisoft/marketcollector/model/lotte/api/CommonResponse.java7-13] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/lotte/api/CommonResponse.java#L7-13>)

### #### Core Database Model: Symbol

The `Symbol` class serves as the primary database model representing market instruments across the system.

| Field | Type   | Purpose                           |
|-------|--------|-----------------------------------|
| code  | String | Primary identifier, MongoDB `@Id` |

```

| `secCode` | String | Securities code |
| `refCode` | String | Reference code for lookups |
| `exchange` | String | Exchange identifier (01, 11, 31, etc.) |
| `name` | String | Vietnamese name |
| `nameEn` | String | English name |
| `marketType` | String | Computed market type |
| `type` | SymbolTypeEnum | Instrument type (STOCK, INDEX, CW, etc.) |
| `securitiesType` | String | Securities classification |
| `isHighlight` | Integer | Highlight flag |
| `createdAt` | Date | Creation timestamp |

```

The `Symbol` model includes sophisticated logic for type resolution and market classificati

[src/main/java/com/difisoft/marketcollector/model/db/Symbol.java60-85] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/db/Symbol.java60-85>)

[src/main/java/com/difisoft/marketcollector/model/db/Symbol.java87-105] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/db/Symbol.java87-105>)

**\*\*Sources:\*\*** [<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/db/Symbol.java16-31>] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/db/Symbol.java16-31>)

[src/main/java/com/difisoft/marketcollector/model/db/Symbol.java16-31] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/db/Symbol.java16-31>)

[src/main/java/com/difisoft/marketcollector/model/db/Symbol.java60-125] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/db/Symbol.java60-125>)

#### #### Exchange and Type Constants

The system uses comprehensive constant mappings to handle Vietnamese market structure:

! [Chart 3] (market-collector-lotte-images/market-collector-lotte-section14-3.svg)

**\*\*Sources:\*\*** [<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/constants/Constants.java12-34>] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/constants/Constants.java12-34>)

[src/main/java/com/difisoft/marketcollector/constants/Constants.java12-34] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/constants/Constants.java12-34>)

[src/main/java/com/difisoft/marketcollector/model/db/Symbol.java66-84] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/db/Symbol.java66-84>)

#### #### API Response Model Hierarchy

The API models follow a hierarchical structure with `CommonResponse` as the base class prov

```

| Model | Extends | Purpose |
|---|---|---|
| `CommonResponse` | \- | Base response with `errorCode`, `errorDesc`, `success` |
| `IndexQuoteResponse` | `CommonResponse` | Index quote data |
| `EtfQuoteResponse` | `CommonResponse` | ETF quote data |
| `CwQuoteResponse` | `CommonResponse` | Covered warrant quote data |

```

The `CommonResponse` class [<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/lotte/api/CommonResponse.java7-13>] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/lotte/api/CommonResponse.java7-13>)

[src/main/java/com/difisoft/marketcollector/model/lotte/api/CommonResponse.java7-13] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/lotte/api/CommonResponse.java7-13>)

**\*\*Sources:\*\*** [<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/lotte/api/CommonResponse.java1-14>] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/lotte/api/CommonResponse.java1-14>)

[src/main/java/com/difisoft/marketcollector/model/lotte/api/CommonResponse.java1-14] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/lotte/api/CommonResponse.java1-14>)

### ### Model Validation and Business Logic

The `Symbol` model implements validation through the `isValid()` method[] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/db/Symbol.java#L33-36>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/db/Symbol.java#L33-36>)

[src/main/java/com/difisoft/marketcollector/model/db/Symbol.java33-36] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/db/Symbol.java#L33-36>)

- **Name fallback logic**: `getName()` and `getNameEn()` methods provide fallback to `get

src/main/java/com/difisoft/marketcollector/model/db/Symbol.java46-58] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/db/Symbol.java#L46-58>)

- **Type-based processing**: `getType(SymbolTypeEnum defaultType)` handles ETF classifica

src/main/java/com/difisoft/marketcollector/model/db/Symbol.java107-119] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/db/Symbol.java#L107-119>)

- **Code resolution**: `getValidCode()` prioritizes `refCode` over `code` for lookups[

src/main/java/com/difisoft/marketcollector/model/db/Symbol.java121-124] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/db/Symbol.java#L121-124>)

For detailed information about specific model categories, see:

- [Constants & Enumerations] (#4.1) for exchange mappings and type definitions
- [Database Models] (#database-models) for persistent data structures
- [API Models] (#api-models) for Lotte Securities integration models
- [Data Transformation] (#data-transformation) for HTS mapping and conversion logic

**Sources**: [] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/db/Symbol.java#L33-125>)

[src/main/java/com/difisoft/marketcollector/model/db/Symbol.java33-125] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/db/Symbol.java#L33-125>)

### ### Constants & Enumerations

Relevant source files

- [

src/main/java/com/difisoft/marketcollector/constants/Constants.java] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/constants/Constants.java#L1-10>)

- [

src/main/java/com/difisoft/marketcollector/constants/LotteExchange.java] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/constants/LotteExchange.java#L1-10>)

- [

src/main/java/com/difisoft/marketcollector/constants/MarketInfoEnum.java] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/constants/MarketInfoEnum.java#L1-10>)

- [

src/main/java/com/difisoft/marketcollector/constants/MarketTypeEnum.java] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/constants/MarketTypeEnum.java#L1-10>)

- [

src/main/java/com/difisoft/marketcollector/constants/SecurityTypeEnum.java] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/constants/SecurityTypeEnum.java#L1-10>)

### #### Purpose & Scope

This document describes the constant definitions, enumerations, and mapping systems used th

For information about how these constants are used in message processing, see [Message Proc

\* \* \*

#### #### Exchange Code Classification System

The Market Collector uses numeric exchange codes to identify different markets and asset classes.

#### ##### Exchange Code to Market Mapping

![Chart 1] (market-collector-lotte-images/market-collector-lotte-section15-1.svg)

**Sources:** <https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/constants/Constants.java#L37>

<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/constants/Constants.java#L37>

#### ##### Stock Exchange Codes

The `STOCK_EXCHANGES` array and `STOCK_EXCHANGES_MAP` define mappings for equity-like instruments.

| Code | Exchange | Asset Type | Description                                        |
|------|----------|------------|----------------------------------------------------|
| 01   | HOSE     | STOCK      | Ho Chi Minh Stock Exchange - Stocks                |
| 11   | HNX      | STOCK      | Hanoi Stock Exchange - Stocks                      |
| 31   | UPCOM    | STOCK      | Unlisted Public Company Market - Stocks            |
| 03   | HOSE     | FUND       | Ho Chi Minh Stock Exchange - Investment Funds      |
| 04   | HOSE     | ETF        | Ho Chi Minh Stock Exchange - Exchange Traded Funds |

Implementation details:

- Array definition: [src/main/java/com/difisoft/marketcollector/constants/Constants.java#L12](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/constants/Constants.java#L12)
- Map structure: [src/main/java/com/difisoft/marketcollector/constants/Constants.java#L13-19](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/constants/Constants.java#L13-19)
- Map type: `Map<String, Pair<String, String>>` where the Pair contains (exchange name, asset type)

**Sources:** <https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/constants/Constants.java#L12-19>

<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/constants/Constants.java#L12-19>

#### ##### Index Exchange Codes

The `INDEX_EXCHANGES` array and `INDEX_EXCHANGES_MAP` define mappings for market indices.

| Code | Exchange | Description                          |
|------|----------|--------------------------------------|
| 09   | HOSE     | Ho Chi Minh Stock Exchange Index     |
| 19   | HNX      | Hanoi Stock Exchange Index           |
| 39   | UPCOM    | Unlisted Public Company Market Index |

Implementation details:

- Array definition: [src/main/java/com/difisoft/marketcollector/constants/Constants.java#L21](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/constants/Constants.java#L21)

```

- Map structure:[

    src/main/java/com/difisoft/marketcollector/constants/Constants.java22-26] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/constants/Constants.java22-26) (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/constants/Constants.java22-26)

- Map type: `Map<String, String>` containing only exchange names

**Sources:**[] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/constants/Constants.java21-26) (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/constants/Constants.java21-26)

#### Other Asset Class Codes

Additional exchange code mappings are defined for specialized asset classes:

**Futures Contracts**

- Code: `15`
- Exchange: HNX
- Array: `FUTURES_EXCHANGES`[

    src/main/java/com/difisoft/marketcollector/constants/Constants.java27] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/constants/Constants.java27) (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/constants/Constants.java27)

- Map: `FUTURES_EXCHANGES_MAP`[

    src/main/java/com/difisoft/marketcollector/constants/Constants.java28-30] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/constants/Constants.java28-30) (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/constants/Constants.java28-30)

**Covered Warrants**

- Code: `07`
- Exchange: HOSE
- Array: `CW_EXCHANGES`[

    src/main/java/com/difisoft/marketcollector/constants/Constants.java31] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/constants/Constants.java31) (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/constants/Constants.java31)

- Map: `CW_EXCHANGES_MAP`[

    src/main/java/com/difisoft/marketcollector/constants/Constants.java32-34] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/constants/Constants.java32-34) (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/constants/Constants.java32-34)

**Bonds**

- Code: `12`
- Array: `BOND_EXCHANGES`[

    src/main/java/com/difisoft/marketcollector/constants/Constants.java20] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/constants/Constants.java20) (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/constants/Constants.java20)

- No associated map (bond exchange mapping not currently implemented)

**Sources:**[] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/constants/Constants.java20-34) (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/constants/Constants.java20-34)

* * *

#### Market Type Enumeration

The `MarketTypeEnum` provides a type-safe enumeration of market types with associated exchange codes.

! [Chart 2] (market-collector-lotte-images/market-collector-lotte-section15-2.svg)

```

## ##### Enum Values

| Enum Value  | Exchange Code | Description                           | Usage |
|-------------|---------------|---------------------------------------|-------|
| `ALL`       | ``            | Represents all markets (no filtering) |       |
| `HOSE`      | "01"          | Ho Chi Minh Stock Exchange stocks     |       |
| `HNX`       | "11"          | Hanoi Stock Exchange stocks           |       |
| `UPCOM`     | "31"          | Unlisted Public Company Market stocks |       |
| `ETF_HOSE`  | "04"          | HOSE Exchange Traded Funds            |       |
| `FUND_HOSE` | "03"          | HOSE Investment Funds                 |       |

**Sources:** [] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main>)

[src/main/java/com/difisoft/marketcollector/constants/MarketTypeEnum.java5] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/constants/MarketTypeEnum.java5>)

## ##### Factory Method

The enum provides a static factory method `from(String exchange)` that converts an exchange

Method signature: [] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/constants/MarketTypeEnum.java5>)

[src/main/java/com/difisoft/marketcollector/constants/MarketTypeEnum.java15-32] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/constants/MarketTypeEnum.java15-32>)

Example usage pattern:

```
MarketTypeEnum marketType = MarketTypeEnum.from("01"); // Returns HOSE
String exchangeCode = MarketTypeEnum.HOSE.getExchange(); // Returns "01"
String description = MarketTypeEnum.HOSE.getDescription(); // Returns "HOSE"
```

```

**Sources:**[] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main
[src/main/java/com/difisoft/marketcollector/constants/MarketTypeEnum.java1-41] (https://gitl

* * *

#### Security Type Enumeration

The `SecurityTypeEnum` categorizes financial instruments by their fundamental type. This en

! [Chart 3] (market-collector-lotte-images/market-collector-lotte-section15-3.svg)

##### Enum Values

| Enum Value | Asset Class | Description |
|---|---|---|
| `FUTURES` | Futures Contracts | Standardized derivative contracts |
| `STOCK` | Equity Shares | Common stocks and equity securities |
| `INDEX` | Market Indices | Calculated market performance indicators |
| `BOND` | Debt Instruments | Fixed-income securities |
| `CW` | Covered Warrants | Derivative securities issued by financial institutions |

The enum definition is minimal, containing only the enumeration constants without additiona

**Sources:**[] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main
[src/main/java/com/difisoft/marketcollector/constants/SecurityTypeEnum.java1-5] (https://git

* * *

#### Lotte Exchange Enumeration

The `LotteExchange` enum represents exchange names as used by the Lotte Securities API. Th

! [Chart 4] (market-collector-lotte-images/market-collector-lotte-section15-4.svg)

##### Enum Values

| Enum Value | Lotte API Name | Internal Market Type |
|---|---|---|
| `HNX` | HNX | `MarketTypeEnum.HNX` |
| `HSX` | HSX (Ho Chi Minh Stock Exchange) | `MarketTypeEnum.HOSE` |
| `UPCOM` | UPCOM | `MarketTypeEnum.UPCOM` |

##### Conversion Method

The `toMarketType()` method converts a `LotteExchange` value to the corresponding `com.dif:

Method implementation:[] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191
[src/main/java/com/difisoft/marketcollector/constants/LotteExchange.java9-13] (https://githu

**Sources:**[] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main

```

```
[src/main/java/com/difisoft/marketcollector/constants/LotteExchange.java1-14] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/constants/LotteExchange.java1-14)
```

\* \* \*

#### #### Market Info Type Enumeration

The `MarketInfoEnum` defines types of market information events tracked by the system. These events are categorized into different types based on their purpose and the information they provide.

| Enum Value               | Description                         | Purpose                                   |
|--------------------------|-------------------------------------|-------------------------------------------|
| `CURRENT_DIVIDEND_EVENT` | Current dividend distribution event | Tracks active dividend events             |
| `LAST_TRADING_DATE`      | Last trading date information       | Tracks final trading dates for securities |

This enum is typically used with the `MarketInfo` database model to categorize different types of market information events.

**Sources:** [1] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/constants/MarketInfoEnum.java1-5) (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/constants/MarketInfoEnum.java1-5)

\* \* \*

#### #### Default Configuration Values

The `Constants` interface defines several default configuration values used throughout the system. These values are used to configure the system's behavior and provide default settings for various operations.

| Constant                   | Value      | Type      | Purpose                                                  |
|----------------------------|------------|-----------|----------------------------------------------------------|
| `FIX_DEFAULT_CURRENCY`     | `"VND"`    | `String`  | Default currency for Vietnamese markets                  |
| `DEFAULT_TIME`             | `"120000"` | `String`  | Default time value (format: HHMMSS, represents 12:00:00) |
| `DEFAULT_HIGHLIGHT_NUMBER` | `1000`     | `Integer` | Default threshold for highlighting significant events    |
| `MAX_RETRY`                | `10`       | `Integer` | Maximum number of retry attempts for operations          |

#### #### Currency Default

The `FIX\_DEFAULT\_CURRENCY` constant [1] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/constants/Constants.java9) (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/constants/Constants.java9)

#### #### Retry Configuration

The `MAX\_RETRY` constant [1] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/constants/Constants.java36) (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/constants/Constants.java36)

**Sources:** [1] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/constants/Constants.java9-36) (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/constants/Constants.java9-36)

\* \* \*

#### #### Usage in the System

The constants and enumerations defined in this package are used throughout the Market Collector system to provide default values and categorize market information events.

#### ##### Message Routing

The WebSocket message handlers use exchange code mappings to route incoming messages to appropriate

Related components:

- Real-time service message handlers (see [Message Processing Pipeline] (#message-processing-pipeline))
- WebSocket connection implementations (see [WebSocket Architecture] (#websocket-architecture))

#### ##### Data Enrichment

Raw HTS messages contain only numeric exchange codes. The system uses these constant mappings to

#### ##### Symbol Information Processing

The symbol download and recovery services use `MarketTypeEnum` to categorize and filter symbols

Related components:

- Symbol information services (see [Database Models] (#database-models))
- Data transformation logic (see [Data Transformation] (#data-transformation))

#### ##### Configuration and Feature Flags

Default values from `Constants` are used when system configuration does not specify certain

**\*\*Sources:\*\*** [] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/constants/Constants.java#L37>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/constants/MarketTypeEnum.java#L41>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/constants/LotteExchange.java#L14>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/constants/MarketTypeEnum.java#L41>)

[src/main/java/com/difisoft/marketcollector/constants/Constants.java#L37] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/constants/MarketTypeEnum.java#L41>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/constants/LotteExchange.java#L14>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/constants/MarketTypeEnum.java#L41>)

src/main/java/com/difisoft/marketcollector/constants/MarketTypeEnum.java#L41] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/constants/LotteExchange.java#L14>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/constants/MarketTypeEnum.java#L41>)

src/main/java/com/difisoft/marketcollector/constants/LotteExchange.java#L14] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/constants/MarketTypeEnum.java#L41>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/constants/LotteExchange.java#L14>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/constants/MarketTypeEnum.java#L41>)

---

#### ### Database Models

Relevant source files

- [src/main/java/com/difisoft/marketcollector/model/db/DividendRate.java] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/db/DividendRate.java>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/db/EstimatedData.java>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/db/MappingMongo.java>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/db/MarketInfo.java>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/db/MarketInfo.java>)
- [src/main/java/com/difisoft/marketcollector/model/db/EstimatedData.java] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/db/EstimatedData.java>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/db/MappingMongo.java>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/db/MarketInfo.java>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/db/MarketInfo.java>)
- [src/main/java/com/difisoft/marketcollector/model/db/MappingMongo.java] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/db/MappingMongo.java>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/db/MarketInfo.java>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/db/MarketInfo.java>)
- [src/main/java/com/difisoft/marketcollector/model/db/MarketInfo.java] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/db/MarketInfo.java>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/db/MarketInfo.java>)

`src/main/java/com/difisoft/marketcollector/model/db/Symbol.java`] (<https://github.com/nhs>)

This page covers the persistent data models used for MongoDB storage in the Market Collector.

### #### Core Entity Models

The system uses two primary MongoDB collections to store persistent market data: symbols and market info.

#### ##### Symbol Model

The `Symbol` class represents market instruments and securities across Vietnamese exchanges.

| Field                         | Type           | Description                                            |
|-------------------------------|----------------|--------------------------------------------------------|
| ---                           | ---            | ---                                                    |
| <code>`code`</code>           | String         | Primary identifier and MongoDB <code>@Id`</code> field |
| <code>`secCode`</code>        | String         | Securities code for alternative identification         |
| <code>`refCode`</code>        | String         | Reference code used when primary code is unavailable   |
| <code>`exchange`</code>       | String         | Exchange identifier (HOSE, HNX, UPCOM, etc.)           |
| <code>`name`</code>           | String         | Vietnamese name of the instrument                      |
| <code>`nameEn`</code>         | String         | English name of the instrument                         |
| <code>`marketType`</code>     | String         | Computed market type based on symbol type and exchange |
| <code>`type`</code>           | SymbolTypeEnum | Symbol classification (STOCK, INDEX, CW, FUTURES, ETF) |
| <code>`securitiesType`</code> | String         | Computed securities type classification                |
| <code>`isHighlight`</code>    | Integer        | Flag indicating if symbol should be highlighted        |
| <code>`createdAt`</code>      | Date           | Record creation timestamp                              |

The Symbol model includes several computed properties through `@JsonIgnore`` methods that derive values from other fields.

**Sources:** [[https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/db/Symbol.java#L125)

[src/main/java/com/difisoft/marketcollector/model/db/Symbol.java#L125](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/db/Symbol.java#L125)] ([https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/db/Symbol.java#L125)

#### ##### MarketInfo Model

The `MarketInfo` class stores market-wide information and dividend events in the `c_market_info`` collection.

| Field                    | Type   | Description                        |
|--------------------------|--------|------------------------------------|
| ---                      | ---    | ---                                |
| <code>`id`</code>        | String | MongoDB <code>@Id`</code> field    |
| <code>`eventList`</code> | List   | Collection of dividend rate events |
| <code>`createdAt`</code> | Date   | Record creation timestamp          |
| <code>`updatedAt`</code> | Date   | Last modification timestamp        |

**Sources:** [[https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/db/MarketInfo.java#L18)

[src/main/java/com/difisoft/marketcollector/model/db/MarketInfo.java#L18](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/db/MarketInfo.java#L18)] ([https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/db/MarketInfo.java#L18)

### #### Supporting Data Models

#### ##### DividendRate

The `DividendRate` class represents dividend information embedded within `MarketInfo`` entities.

| Field             | Type   | Description                              |
|-------------------|--------|------------------------------------------|
| `code`            | String | Symbol code associated with the dividend |
| `basicPrice`      | double | Base price for dividend calculation      |
| `totalAdjustRate` | double | Total adjustment rate applied            |

**Sources:** <https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/db/DividendRate.java#L10>

### EstimatedData

The `EstimatedData` class contains price ceiling and floor estimates for market instruments:

| Field          | Type   | Description                     |
|----------------|--------|---------------------------------|
| `ceilingPrice` | double | Maximum allowable trading price |
| `floorPrice`   | double | Minimum allowable trading price |

**Sources:** <https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/db/EstimatedData.java#L9>

### MongoDB Integration Interface

#### MappingMongo Interface

The `MappingMongo` interface provides standardized MongoDB mapping functionality for database models.



The interface provides a default `toMapUpdate()` method that uses Java reflection to convert database models to MongoDB update documents.

[src/main/java/com/difisoft/marketcollector/model/db/MappingMongo.java#L15-26](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/db/MappingMongo.java#L15-26)

**Sources:** <https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/db/MappingMongo.java#L27>

### Data Structure and Relationships

The following diagram illustrates the relationships between database models and their MongoDB collections:



### MongoDB Collections

The system uses two primary MongoDB collections for persistent storage:



The Symbol model uses computed properties to derive market and securities type information.

**Sources:** <https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/db/Symbol.java#L10>



```

src/main/java/com/difisoft/marketcollector/model/lotte/api/CwQuoteResponse.java1-32] (https:
src/main/java/com/difisoft/marketcollector/model/lotte/api/EtfQuoteResponse.java1-35] (http:

* * *

#### Common Response Models

##### CommonResponse

The `CommonResponse` class serves as the base class for all Lotte API responses, providing

| Field | Type | JSON Property | Description |
|---|---|---|---|
| `errorCode` | `String` | `error_code` | API error code when request fails |
| `errorDesc` | `String` | `error_desc` | Human-readable error description |
| `success` | `boolean` | `success` | Indicates if the API call succeeded |

All response models extend this class to inherit error handling capabilities. Services che

**Sources:**[] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main

[src/main/java/com/difisoft/marketcollector/model/lotte/api/CommonResponse.java1-14] (https:

##### CommonListResponse<I>

The `CommonListResponse<I>` generic class provides pagination support for list-based respo

| Field | Type | Description |
|---|---|---|
| `nextKey` | `String` | Pagination token for fetching the next page |
| `hasNext` | `boolean` | Indicates if more pages are available |
| `list` | `List<I>` | The list of items of type `I` for the current page |

This model enables cursor-based pagination where clients use the `nextKey` value from one

**Sources:**[] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main

[src/main/java/com/difisoft/marketcollector/model/lotte/api/CommonListResponse.java1-12] (ht

* * *

#### Asset-Specific Quote Response Models

##### CwQuoteResponse

The `CwQuoteResponse` class models the response from the covered warrant quote history API

! [Chart 2] (market-collector-lotte-images/market-collector-lotte-section17-2.svg)

**Response Fields:**

| Field | Type | JSON Property | Description |

```

```
|--- | --- | --- | --- |
| `totalRecord` | `String` | `total_record` | Total number of records available |
| `dataList` | `List<ListResponse<Item>>` | `data_list` | List of paginated quote data |
```

**\*\*Item Fields:\*\***

```
| Field | Type | JSON Property | Description |
|--- | --- | --- | --- |
| `time` | `String` | `time` | Timestamp of the quote |
| `last` | `Double` | `last` | Last traded price |
| `change` | `Double` | `change` | Price change from reference |
| `changeRate` | `Double` | `changeRate` | Percentage change |
| `volume` | `Long` | `matchVol` | Matched volume |
```

The `CwQuoteResponse` uses a nested `ListResponse<Item>` structure to support pagination with

**\*\*Sources:\*\*** [] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main>

[src/main/java/com/difisoft/marketcollector/model/lotte/api/CwQuoteResponse.java#L32] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/lotte/api/CwQuoteResponse.java#L32>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/lotte/api/CwQuoteResponse.java#L32>)

##### EtfQuoteResponse

The `EtfQuoteResponse` class models the response from the ETF quote history API endpoint. It

**\*\*Response Fields:\*\***

```
| Field | Type | JSON Property | Description |
|--- | --- | --- | --- |
| `totalRecord` | `String` | `total_record` | Total number of records available |
| `dataList` | `List<ListResponse<Item>>` | `data_list` | List of paginated quote data |
```

**\*\*Item Fields:\*\***

```
| Field | Type | JSON Property | Description |
|--- | --- | --- | --- |
| `time` | `String` | `time` | Timestamp of the quote |
| `last` | `Double` | `last` | Last traded price (close) |
| `change` | `Double` | `change` | Price change from reference |
| `changeRate` | `Double` | `changeRate` | Percentage change |
| `open` | `Double` | `open` | Opening price for the period |
| `high` | `Double` | `high` | Highest price for the period |
| `low` | `Double` | `low` | Lowest price for the period |
| `volume` | `Long` | `matchVol` | Matched volume |
```

The key difference from `CwQuoteResponse` is the inclusion of `open`, `high`, and `low` fields

**\*\*Sources:\*\*** [] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main>

[src/main/java/com/difisoft/marketcollector/model/lotte/api/EtfQuoteResponse.java#L35] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/lotte/api/EtfQuoteResponse.java#L35>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/lotte/api/EtfQuoteResponse.java#L35>)

\* \* \*

#### Request Models

## #### IndexDailyRequest

The `IndexDailyRequest` class defines the request payload for retrieving historical daily :

![[Chart 3](market-collector-lotte-images/market-collector-lotte-section17-3.svg)]

## \*\*Request Fields:\*\*

| Field          | Type      | JSON Property | Required | Description                                               |
|----------------|-----------|---------------|----------|-----------------------------------------------------------|
| ---            | ---       | ---           | ---      | ---                                                       |
| `indexRefCode` | `String`  | `idx`         | Yes      | The reference code of the index (e.g., "VN-INDEX")        |
| `toDate`       | `String`  | `dt`          | Yes      | End date for the historical data query (format: YYYYMMDD) |
| `nextData`     | `String`  | `next_data`   | No       | Pagination token for fetching the next page               |
| `rowCount`     | `Integer` | `row_count`   | No       | Number of records to return per page                      |

This model uses the builder pattern (indicated by the `@Builder` annotation) to construct :

```
IndexDailyRequest request = IndexDailyRequest.builder()
    .indexRefCode("VN-INDEX")
    .toDate("20231215")
    .rowCount(100)
    .build();
```

The `nextData` field enables pagination by passing the continuation token from a previous

**\*\*Sources:\*\*** [<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/lotte/api/IndexDailyRequest.java#L18>] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/lotte/api/CommonResponse.java#L8-11>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/lotte/api/CwQuoteResponse.java#L10-22>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/lotte/api/EtfQuoteResponse.java#L10-25>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/lotte/api/IndexDailyRequest.java#L10-17>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/lotte/api/MatchVolRequest.java#L10-17>)

#### #### JSON Serialization Mapping

All API models use Jackson annotations to control JSON serialization and deserialization, e

#### ##### Field Name Mapping Pattern

| Java Convention | JSON Property  | Annotation                      |
|-----------------|----------------|---------------------------------|
| `errorCode`     | `error_code`   | `@JsonProperty("error_code")`   |
| `errorDesc`     | `error_desc`   | `@JsonProperty("error_desc")`   |
| `totalRecord`   | `total_record` | `@JsonProperty("total_record")` |
| `dataList`      | `data_list`    | `@JsonProperty("data_list")`    |
| `indexRefCode`  | `idx`          | `@JsonProperty("idx")`          |
| `toDate`        | `dt`           | `@JsonProperty("dt")`           |
| `nextData`      | `next_data`    | `@JsonProperty("next_data")`    |
| `rowCount`      | `row_count`    | `@JsonProperty("row_count")`    |
| `volume`        | `matchVol`     | `@JsonProperty("matchVol")`     |

The `@JsonProperty` annotation ensures bidirectional mapping during:

- **\*\*Serialization:\*\*** When Java objects are converted to JSON for API requests
- **\*\*Deserialization:\*\*** When JSON responses are parsed into Java objects

**\*\*Sources:\*\*** [<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/lotte/api/CommonResponse.java#L8-11>] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/lotte/api/CwQuoteResponse.java#L10-22>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/lotte/api/EtfQuoteResponse.java#L10-25>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/lotte/api/IndexDailyRequest.java#L10-17>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/lotte/api/MatchVolRequest.java#L10-17>)

#### #### Pagination Support

The API models implement cursor-based pagination to handle large result sets efficiently. 5

#### ##### Pattern 1: Top-Level Pagination (CommonListResponse)

! [Chart 4] (market-collector-lotte-images/market-collector-lotte-section17-4.svg)

The `CommonListResponse<I>` class provides:

- `nextKey`: Opaque pagination token
- `hasNext`: Boolean flag indicating more pages exist
- `list`: Current page of items

#### #### Pattern 2: Nested Pagination (Quote Responses)

The `CwQuoteResponse` and `EtfQuoteResponse` use nested `ListResponse<Item>` structures where

! [Chart 5] (market-collector-lotte-images/market-collector-lotte-section17-5.svg)

In this pattern:

- Each `ListResponse` in the `dataList` can be independently paginated
- The `seq` field (mapped from JSON) serves as the pagination token
- Clients must track pagination state per data list entry

**Sources:** [] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main>

[src/main/java/com/difisoft/marketcollector/model/lotte/api/CommonListResponse.java1-12] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/lotte/api/CommonListResponse.java1-12>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/lotte/api/CwQuoteResponse.java26-31>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/lotte/api/EtfQuoteResponse.java29-34>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/lotte/api/EtfQuoteResponse.java29-34>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/lotte/api/EtfQuoteResponse.java29-34>)

[src/main/java/com/difisoft/marketcollector/model/lotte/api/CwQuoteResponse.java26-31] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/lotte/api/CwQuoteResponse.java26-31>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/lotte/api/EtfQuoteResponse.java29-34>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/lotte/api/EtfQuoteResponse.java29-34>)

[src/main/java/com/difisoft/marketcollector/model/lotte/api/EtfQuoteResponse.java29-34] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/lotte/api/EtfQuoteResponse.java29-34>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/model/lotte/api/EtfQuoteResponse.java29-34>)

---

#### ### Data Transformation

Relevant source files

- [src/main/java/com/difisoft/marketcollector/constants/MappingHTS.java] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/constants/MappingHTS.java>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/constants/MappingHTS.java>)

#### #### Purpose and Scope

This document describes the data transformation layer that converts raw HTS (HTS Trading System) data into a format suitable for the database.

For information about the `SymbolInfo` model structure and other database models, see [Data Models] ([Data Models](#)).

\* \* \*

#### #### Overview

The transformation system consists of stateless mapping functions defined in the `MappingHTS` class.

#### ##### Transformation Function Registry

| Function Name               | Input Type                      | Output Type  | Asset Type | Purpose                        |
|-----------------------------|---------------------------------|--------------|------------|--------------------------------|
| ---   ---   ---   ---   --- |                                 |              |            |                                |
| `QUERY_STOCK_INFO`          | `MarketStockCurrentPriceRcv`    | `SymbolInfo` | STOCK      | Transform stock price data     |
| `QUERY_INDEX_INFO`          | `MarketIndustryCurrentIndexRcv` | `SymbolInfo` | INDEX      | Transform index data           |
| `QUERY_CW_INFO`             | `MarketCWCurrentPriceRcv`       | `SymbolInfo` | CW         | Transform covered warrant data |
| `QUERY_FUTURES_INFO`        | `MarketFuturesCurrentPriceRcv`  | `SymbolInfo` | FUTURES    | Transform futures data         |

```

**Sources:**[] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main
[src/main/java/com/difisoft/marketcollector/constants/MappingHTS.java18-344] (https://github
* * *

#### Transformation Architecture

![[Chart 1]](market-collector-lotte-images/market-collector-lotte-section18-1.svg)

**Sources:**[] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main
[src/main/java/com/difisoft/marketcollector/constants/MappingHTS.java1-349] (https://github
* * *

#### Common Transformation Patterns

All transformation functions apply consistent patterns to ensure data standardization across

##### Price and Value Normalization

Price values are normalized using `NumberUtil.round2Decimal()` to ensure consistent decimal

```

```

// Price normalization example open: NumberUtil.round2Decimal(res.getOpen().getValue()) ceilingPrice:
NumberUtil.round2Decimal(res.getCeilingPrice().getValue()) floorPrice:
NumberUtil.round2Decimal(res.getFloorPrice().getValue())

```

```

// Value scaling example tradingValue: NumberUtil.round2Decimal(res.getTradingValue().getValue() * 1000000)

```

```

**Sources:**[] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main
[src/main/java/com/difisoft/marketcollector/constants/MappingHTS.java21-37] (https://github
src/main/java/com/difisoft/marketcollector/constants/MappingHTS.java131-141] (https://github
##### Market Name Mapping

The transformation layer maps HTS market identifiers to standardized market names. The pri

```

```

if (res.getMarketName().getValue().equals("HSX")) { symbolInfo.setMarketType("HOSE"); } else {
symbolInfo.setMarketType(res.getMarketName().getValue()); }

```



```

**Sources:**[] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/
[src/main/java/com/difisoft/marketcollector/constants/MappingHTS.java79-83] (https://github
src/main/java/com/difisoft/marketcollector/constants/MappingHTS.java241-245] (https://github

##### Bid/Offer Data Processing

Market depth data (bid/offer lists) is processed by iterating through `bidOfferData` array:

! [Chart 2] (market-collector-lotte-images/market-collector-lotte-section18-2.svg)

Each `BidOfferItem` contains:

- `bidPrice`, `bidVolume`, `bidVolumeChange`
- `offerPrice`, `offerVolume`, `offerVolumeChange`

**Sources:**[] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/
[src/main/java/com/difisoft/marketcollector/constants/MappingHTS.java99-118] (https://github
src/main/java/com/difisoft/marketcollector/constants/MappingHTS.java184-203] (https://github
src/main/java/com/difisoft/marketcollector/constants/MappingHTS.java285-304] (https://github

##### High/Low Year Data Processing

Historical price range data is processed in two modes:

1. Detailed Mode: When `highLowYearData` array is available, each item contains high/low
2. Simple Mode: When only 52-week high/low values are available, a single `HighLowYearData`

Items are added to the `highLowYearData` list in reverse order (index 0) to maintain chronological order.

**Sources:**[] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/
[src/main/java/com/difisoft/marketcollector/constants/MappingHTS.java84-98] (https://github
src/main/java/com/difisoft/marketcollector/constants/MappingHTS.java169-183] (https://github
src/main/java/com/difisoft/marketcollector/constants/MappingHTS.java266-280] (https://github

* * *

#### Asset-Specific Transformations

##### Stock Transformation (QUERY\_STOCK\_INFO)

Transforms `MarketStockCurrentPriceRcv` messages into `SymbolInfo` objects with `type` set to `STOCK`.

Key Fields Transformed:

| Source Field | Target Field | Transformation |

```

```

|--- | --- | --- |
| `open`, `high`, `low`, `last` | `open`, `high`, `low`, `last` | `round2Decimal` |
| `tradingValue` | `tradingValue` | `round2Decimal` * 1,000,000 |
| `stockCode` | `code` | Direct mapping |
| `industry` | `industry` | Trimmed via `trim()` helper |
| `listedQuantity` | `listedQuantity` | Cast to `Long` |
| `foreignerBuyVolume`, `foreignerSellVolume` | `foreignerBuyVolume`, `foreignerSellVolume`
| `foreignerTotalRoom`, `foreignerCurrentRoom` | `foreignerTotalRoom`, `foreignerCurrentRoom`

**Special Logic:**

- If `last` value is 0, it is replaced with `referencePrice`
- Includes foreigner trading data (buy/sell volumes and room capacity)
- Processes bid/offer data and high/low year data

**Sources:**[] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main
[src/main/java/com/difisoft/marketcollector/constants/MappingHTS.java128-205] (https://github

##### Index Transformation (QUERY\_INDEX\_INFO)

Transforms `MarketIndustryCurrentIndexRcv` messages into `SymbolInfo` objects with `type` set to

**Key Fields Transformed:**

| Source Field | Target Field | Transformation |
|--- | --- | --- |
| `openIndex`, `highIndex`, `lowIndex` | `open`, `high`, `low` | `round2Decimal` |
| `last`, `change`, `rate` | `last`, `change`, `rate` | `round2Decimal` |
| `upCount`, `downCount` | `upCount`, `downCount` | Direct mapping |
| `ceilingCount`, `floorCount` | `ceilingCount`, `floorCount` | Direct mapping |
| `unchangedCount` | `unchangedCount` | Direct mapping |

**Special Logic:**

- `referencePrice` is computed as `last - change` rather than received directly
- Includes market breadth indicators (up/down/ceiling/floor/unchanged counts)
- Does not process bid/offer data (indices don't have market depth)

**Sources:**[] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main
[src/main/java/com/difisoft/marketcollector/constants/MappingHTS.java207-226] (https://github

##### Covered Warrant Transformation (QUERY\_CW\_INFO)

Transforms `MarketCWCurrentPriceRcv` messages into `SymbolInfo` objects with `type` set to

**Key Fields Transformed:**

| Source Field | Target Field | Transformation |
|--- | --- | --- |
| `exercisePrice` | `exercisePrice` | `round2Decimal` ÷ 10 |
| `exerciseRatio` | `exerciseRatio` | Direct mapping (string) |
| `breakEven` | `breakEven` | Computed or `round4Decimal` |

```

```

| `lastTradingDate`, `maturityDate` | `lastTradingDate`, `maturityDate` | `DATE_FORMAT().parse()` |
| `underlyingSymbol`, `underlyingPrice` | `underlyingSymbol`, `underlyingPrice` | Direct mapping |
| `delta`, `gearingRt` | `delta`, `gearingRt` | `round2Decimal` |
| `impliedVolatility`, `parity` | `impliedVolatility`, `parity` | Direct mapping |

**Special Logic:**

1. Exercise Ratio Parsing: The `exerciseRatio` string (format "a:b") is parsed to calculate the exercise ratio.
2. Break-Even Calculation: Computed as `last * exerciseRatioValue + exercisePrice`
3. Date Parsing: Attempts to parse `lastTradingDate` and `maturityDate` with error suppression.
4. Underlying Asset Data: Includes underlying symbol price, change, and rate

**Sources:**[] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/constants/MappingHTS.java#L18-126) (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/constants/MappingHTS.java#L18-126)

##### Futures Transformation (QUERY\_FUTURES\_INFO)

Transforms `MarketFuturesCurrentPriceRcv` messages into `SymbolInfo` objects with `type` set to `FUTURES`.

**Key Fields Transformed:**

| Source Field | Target Field | Transformation |
|---|---|---|
| `stockCode`, `stockName` | `code`, `name` | Direct mapping |
| `baseCode` | `baseCode` | Direct mapping |
| `openInterest` | `openInterest` | `round2Decimal` |
| `openInterestChange` | `openInterestChange` | `round2Decimal`, `max(value, 0)` |
| `theoryPrice`, `basis` | `theoryPrice`, `basis` | `round2Decimal` |
| `theoryBasis`, `marketBasis` | `theoryBasis`, `marketBasis` | `round2Decimal` |
| `disparate`, `disparateRate` | `disparate`, `disparateRate` | `round2Decimal` |
| `remainDate` | `remainDate` | Cast to `Long` |
| `startDate`, `endDate` | `firstTradingDate`, `maturityDate` | `DATE_FORMAT().parse()` |

**Special Logic:**

1. Open Interest Change: Ensured to be non-negative via `Math.max(value, 0)`
2. Foreigner Trading Data: Includes both normal and put-through (PT) trading metrics
   - `normalForeignerBuyValue`, `normalForeignerBuyVolume`
   - `normalForeignerSellValue`, `normalForeignerSellVolume`
   - `ptForeignerTotalBuyValue`, `ptForeignerTotalBuyVolume`
   - `ptForeignerTotalSellValue`, `ptForeignerTotalSellVolume`
3. Basis Calculations: Multiple basis measures (theory, market, disparate) for pricing
4. Contract Lifecycle: Includes `firstTradingDate`, `maturityDate`, and `remainDate`

**Sources:**[] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/constants/MappingHTS.java#L228-344) (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/constants/MappingHTS.java#L228-344)

* * *

##### Transformation Function Usage

! [Chart 3] (market-collector-lotte-images/market-collector-lotte-section18-3.svg)

```

The transformation functions are invoked by service layer components when converting HTS me

**\*\*Sources:\*\*** [] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main>

[src/main/java/com/difisoft/marketcollector/constants/MappingHTS.java#L17-349] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/constants/MappingHTS.java#L17-349>)

\* \* \*

#### Utility Functions

##### trim() Helper

The `trim()` helper function safely trims string values, handling null inputs.

```
private static String trim(String s) {
    return s == null ? s : s.trim();
}
```

This is used for processing industry names and other string fields that may contain leading

**Sources:** [ (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main> [src/main/java/com/difisoft/marketcollector/constants/MappingHTS.java346-348] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/constants/MappingHTS.java346-348>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/constants/MappingHTS.java346-348>)

\* \* \*

#### #### Data Quality Considerations

The transformation layer implements several data quality measures:

1. **Null Safety:** All field access uses `getValue()` methods that handle null gracefully.
2. **Numeric Precision:** Consistent rounding via `NumberUtil.round2Decimal()` and `round2Decimal()`.
3. **Scale Normalization:** Trading values consistently multiplied by 1,000,000.
4. **Zero Value Handling:** Stock last price defaults to reference price if zero.
5. **Negative Value Constraints:** Open interest change constrained to non-negative values.
6. **Date Parsing Resilience:** Try-catch blocks suppress parse exceptions to prevent trading data loss.
7. **Volume Filtering:** Bid/offer items with zero volumes are excluded from output.

These patterns ensure consistent, high-quality data regardless of source data variations.

**Sources:** [ (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main> [src/main/java/com/difisoft/marketcollector/constants/MappingHTS.java18-344] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/constants/MappingHTS.java18-344>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/constants/MappingHTS.java18-344>)

---

## ## Services & Processing

### ### Overview

Relevant source files

- [src/main/java/com/difisoft/marketcollector/consumers/RequestHandler.java] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/consumers/RequestHandler.java>)
- [src/main/java/com/difisoft/marketcollector/job/JobService.java] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/job/JobService.java>)

This document covers the orchestration layer of the Market Collector Lotte application, specifically detailing the services and processing components that manage real-time data collection and processing.

1. **Scheduled Jobs** - Automated tasks that run on defined schedules to manage real-time data collection.
2. **Request Processing** - Job-based API endpoints that allow external systems to trigger data collection.

For details on the real-time data collection services orchestrated by these components, see the [Real-time Data Collection Services](#) section.

#### #### Architecture Overview

The services and processing layer acts as the orchestration component of the Market Collector Lotte application, managing the flow of data from source systems to the data warehouse.

1. **Time-based Execution** - The `JobService` uses Spring's `@Scheduled` annotations to trigger data collection jobs at specific intervals.

2. **\*\*Event-based Execution\*\*** - The `RequestHandler` extends the job framework to provide t

**\*\*Service Orchestration Architecture\*\***

![[Chart 1](market-collector-lotte-images/market-collector-lotte-section19-1.svg)]

**\*\*Sources:\*\*** [] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main>

[src/main/java/com/difisoft/marketcollector/job/JobService.java#L76] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/consumers/RequestHandler.java#L60>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/consumers/RequestHandler.java#L60>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/consumers/RequestHandler.java#L60>)

#### JobService - Scheduled Task Orchestration

The `JobService` class implements scheduled task orchestration using Spring's scheduling fr

##### Implementation Structure

The service is defined as a Spring `@Service` component with `@Scheduled` methods that execu

**\*\*Class Dependencies:\*\***

| Dependency                               | Type                                     | Purpose                                |
|------------------------------------------|------------------------------------------|----------------------------------------|
| <code>cacheService</code>                | <code>CacheService</code>                | Cache management operations            |
| <code>realTimeDataListenerService</code> | <code>RealTimeDataListenerService</code> | Real-time WebSocket conn               |
| <code>appConf</code>                     | <code>AppConf</code>                     | Configuration access for feature flags |
| <code>symbolInfoService</code>           | <code>ISymbolInfoService</code>          | Symbol data download and management    |
| <code>riseFallStockRankService</code>    | <code>RiseFallStockRankService</code>    | Stock ranking analytics                |
| <code>rightInfoService</code>            | <code>RightInfoService</code>            | Rights information processing          |

**\*\*Sources:\*\*** [] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main>

[src/main/java/com/difisoft/marketcollector/job/JobService.java#L15-36] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/job/JobService.java#L15-36>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/job/JobService.java#L15-36>)

##### Scheduled Operations

The `JobService` implements six scheduled operations, each triggered by CRON expressions co

**\*\*Real-Time Connection Management\*\***

The service provides four scheduled methods to control real-time data collection, supportin

![[Chart 2](market-collector-lotte-images/market-collector-lotte-section19-2.svg)]

| Method                          | CRON Property                                | Purpose                         |
|---------------------------------|----------------------------------------------|---------------------------------|
| <code>startRealtime1st()</code> | <code>app.schedulers.startRealtime1st</code> | Start WebSocket connections     |
| <code>stopRealtime1st()</code>  | <code>app.schedulers.stopRealtime1st</code>  | Stop connections during midday  |
| <code>startRealtime2nd()</code> | <code>app.schedulers.startRealtime2nd</code> | Resume connections for aftern   |
| <code>stopRealtime2nd()</code>  | <code>app.schedulers.stopRealtime2nd</code>  | Stop connections at market clos |

**\*\*Sources:\*\*** [] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main>

```
[src/main/java/com/difisoft/marketcollector/job/JobService.java38-56] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/job/JobService.java#L38-L56)

**Symbol Download Schedule**

The `downloadSymbol()` method triggers periodic symbol data downloads from the configured sources.
```

```
@Scheduled(cron = "${app.schedulers.downloadSymbol}")
public void downloadSymbol() {
    symbolInfoService.downloadSymbol("JobScheduler");
}
```

```
This typically runs once daily before market open to ensure the system has current symbol information.

**Sources:**[] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/job/JobService.java#L38-L56)

[src/main/java/com/difisoft/marketcollector/job/JobService.java58-61] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/job/JobService.java#L58-L61)

**Analytics and Ranking Updates**

Two conditional scheduled tasks update analytics data, executing only when the `enableQuery` configuration property is set to true.

| Method | CRON Property | Condition | Purpose |
|---|---|---|---|
| `riseFallStockRankSave()` | `app.schedulers.riseFallStockRankSave` | `appConf.isEnableQuery` | Update stock ranking data.
| `rightInfoSave()` | `app.schedulers.rightInfoSave` | `appConf.isEnableQuery` | Update right information data.

The conditional execution pattern allows the system to disable resource-intensive query operations when not needed.
```

```
@Scheduled(cron = "${app.schedulers.riseFallStockRankSave}")
public void riseFallStockRankSave() {
    if (appConf.isEnableQuery()) {
        riseFallStockRankService.getAndSaveStockRanking();
    }
}
```

```

**Sources:**[] (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/job/JobService.java#L63-75) (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/job/JobService.java#L63-75)

#### RequestHandler - Job-Based API Endpoints

The `RequestHandler` class extends the `JobHandler` framework to provide URI-based endpoints.

#### Job Handler Framework Integration

The `RequestHandler` extends `com.difisoft.job.JobHandler`, a base class that provides:

- Kafka consumer integration for job message processing
- URI routing with type-safe request/response handling
- Automatic deserialization of job payloads
- Error handling and response publishing

The constructor configures the handler with Kafka connection details and builds a routing table.

```

```

public RequestHandler(/* dependencies */) {
    super(objectMapper, appConf.getKafkaBootstraps(), appConf.getClusterId(), "1", 5);
    this.uriBuilder()
        .add("job:/api/v1/collector/downloadSymbol", Object.class, symbolInfoService::downloadSymbolFromRequest)
        // ... additional routes
        .end();
}

```

**Sources:** [\[https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/consumers/RequestHandler.java#L24-58\]](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/consumers/RequestHandler.java#L24-58) <https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/consumers/RequestHandler.java#L24-58>

##### Available API Endpoints

The `RequestHandler` exposes eleven job-based API endpoints, organized by functional category.

**Symbol Management Endpoints**

! [Chart 3] (market-collector-lotte-images/market-collector-lotte-section19-3.svg)

| Endpoint URI                                            | Request Type              | Handler Method                                      | Purpose                           |
|---------------------------------------------------------|---------------------------|-----------------------------------------------------|-----------------------------------|
| <code>/job:/api/v1/collector/downloadSymbol</code>      | <code>Object.class</code> | <code>symbolInfoService::downloadSymbol</code>      | Download symbol information       |
| <code>/job:/api/v1/collector/forceDownloadSymbol</code> | <code>Object.class</code> | <code>symbolInfoService::forceDownloadSymbol</code> | Force download symbol information |

**Sources:** [\[https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/consumers/RequestHandler.java#L38-44\]](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/consumers/RequestHandler.java#L38-44) <https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/consumers/RequestHandler.java#L38-44>

**Data Recovery Endpoints**

The recovery endpoints allow external systems to request historical data backfill for specific symbols.

! [Chart 4] (market-collector-lotte-images/market-collector-lotte-section19-4.svg)

| Endpoint URI                                                | Request Type                        | Handler Service                                            | Purpose                           |
|-------------------------------------------------------------|-------------------------------------|------------------------------------------------------------|-----------------------------------|
| <code>/job:/api/v1/collector/quoteDataRecover</code>        | <code>String.class</code>           | <code>QuoteRecoverService::downloadQuoteData</code>        | Download quote data               |
| <code>/job:/api/v1/collector/dailyDataRecover</code>        | <code>Object.class</code>           | <code>DailyRecoverService::downloadDailyData</code>        | Download daily data               |
| <code>/job:/api/v1/collector/dailyDataRecoverByApi</code>   | <code>DailyApiRecoverRequest</code> | <code>DailyRecoverService::downloadDailyDataByApi</code>   | Download daily data by API        |
| <code>/job:/api/v1/collector/dailyDataRecoverNoCache</code> | <code>Object.class</code>           | <code>DailyRecoverService::downloadDailyDataNoCache</code> | Download daily data without cache |

**Sources:** [\[https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/consumers/RequestHandler.java#L39-42\]](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/consumers/RequestHandler.java#L39-42) <https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/consumers/RequestHandler.java#L39-42>

**Statistics and Analytics Endpoints**

| Endpoint URI                                       | Request Type              | Handler Service                                       | Purpose             |
|----------------------------------------------------|---------------------------|-------------------------------------------------------|---------------------|
| <code>/job:/api/v1/collector/quoteStatistic</code> | <code>Object.class</code> | <code>QuoteStatisticService::getQuoteStatistic</code> | Get quote statistic |

**Sources:** [\[https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/consumers/RequestHandler.java#L43\]](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/consumers/RequestHandler.java#L43) <https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/consumers/RequestHandler.java#L43>

**Real-Time Control Endpoints**

The request handler provides manual control endpoints for the real-time data collection system.

! [Chart 5] (market-collector-lotte-images/market-collector-lotte-section19-5.svg)

| Endpoint | URI                                    | Implementation                                    | Purpose |
|----------|----------------------------------------|---------------------------------------------------|---------|
|          | ---                                    | ---                                               | ---     |
|          | `job:/api/v1/collector/realtime/start` | Lambda calling `realTimeDataListenerService.run`  |         |
|          | `job:/api/v1/collector/realtime/stop`  | Lambda calling `realTimeDataListenerService.stop` |         |

These endpoints use inline lambda expressions rather than delegating to named service methods.

```
.add("job:/api/v1/collector/realtime/start", Object.class, (r, c) -> {
    realTimeDataListenerService.run();
    return null;
})
```

**Sources:** [(https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/consumers/RequestHandler.java45-52) (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/consumers/RequestHandler.java53-56) (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/consumers/RequestHandler.java57-60)]

### System Control Endpoints

| Endpoint URI           | Implementation                            | Purpose                        |
|------------------------|-------------------------------------------|--------------------------------|
| ---   ---   ---        |                                           |                                |
| /api/v1/collector/exit | Lambda calling `systemService.shutdown()` | Gracefully shutdown the system |

This endpoint provides a mechanism for remote system shutdown, useful for orchestrated deployments.

**Sources:** [(https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/consumers/RequestHandler.java45-52) (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/consumers/RequestHandler.java53-56) (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/consumers/RequestHandler.java57-60)]

### Request Handler Architecture

#### Complete Request Handler Flow

![[Chart 6](market-collector-lotte-images/market-collector-lotte-section19-6.svg)]

The `RequestHandler` acts as a bridge between the Kafka-based job system and the internal service layer.

- Consumes the message from Kafka
- Routes to the appropriate handler based on the URI
- Deserializes the request payload into the specified type
- Invokes the handler method (service method or lambda)
- Serializes the response
- Publishes the result back to Kafka for the requesting client

This pattern enables asynchronous, distributed control of the market collector system through Kafka.

**Sources:** [(https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/consumers/RequestHandler.java1-60) (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/consumers/RequestHandler.java61-65) (https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/consumers/RequestHandler.java66-70)]

### Service Coordination Patterns

The orchestration layer implements several patterns to coordinate activities across the application.

#### Scheduled vs. On-Demand Execution

| Pattern         | Implementation                           | Use Cases                                |
|-----------------|------------------------------------------|------------------------------------------|
| ---   ---   --- |                                          |                                          |
| Time-Based      | `JobService` with `@Scheduled`           | Market-aligned operations (start/stop)   |
| Event-Based     | `RequestHandler` with Kafka jobs         | On-demand data recovery, manual override |
| Conditional     | Feature flag checks in scheduled methods | Optional analytics that can be disabled  |

### Dual Control Mechanism

Several operations can be triggered through both mechanisms, providing flexibility in system management.

- **\*\*Real-time Start/Stop\*\***: Both ``JobService`` scheduled methods and ``RequestHandler`` manage
- **\*\*Symbol Download\*\***: Both ``JobService.downloadSymbol()`` and ``RequestHandler`` endpoints

This dual control pattern allows normal automated operation via schedules while supporting

#### ##### Service Lifecycle Management

The orchestration layer manages service lifecycles through dependency injection and method

![[Chart 7](market-collector-lotte-images/market-collector-lotte-section19-7.svg)]

Neither ``JobService`` nor ``RequestHandler`` maintain state about the services they orchestrate

**\*\*Sources:\*\*** [] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/job/JobService.java#L15-76>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/consumers/RequestHandler.java#L24-58>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/consumers/RequestHandler.java#L24-58>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/consumers/RequestHandler.java#L24-58>)

[src/main/java/com/difisoft/marketcollector/job/JobService.java#L15-76] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/job/JobService.java#L15-76>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/consumers/RequestHandler.java#L24-58>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/consumers/RequestHandler.java#L24-58>)

[src/main/java/com/difisoft/marketcollector/consumers/RequestHandler.java#L24-58] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/consumers/RequestHandler.java#L24-58>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/consumers/RequestHandler.java#L24-58>)

#### ### Scheduled Jobs

Relevant source files

- [src/main/java/com/difisoft/marketcollector/job/JobService.java] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/job/JobService.java>)

#### #### Purpose and Scope

This page documents the automated scheduled jobs implemented in the Market Collector Lotte

For information about request-based job processing and dispatching, see page 6.2. For details

#### #### Job Scheduling Architecture

The ``JobService`` class orchestrates all scheduled tasks in the Market Collector Lotte system

#### ##### JobService Class Structure

![[Chart 1](market-collector-lotte-images/market-collector-lotte-section20-1.svg)]

**\*\*Diagram: JobService Class Structure and Spring Integration\*\***

Sources: [] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/job/JobService.java#L15-76>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/consumers/RequestHandler.java#L24-58>)

[src/main/java/com/difisoft/marketcollector/job/JobService.java#L15-76] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/job/JobService.java#L15-76>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/consumers/RequestHandler.java#L24-58>)

#### #### Scheduled Jobs Summary

The following table provides a complete overview of all scheduled jobs managed by ``JobService``

| Method Name                       | Cron Expression | Property | Target Service                                 | Operation | Feature Flag                               |  |
|-----------------------------------|-----------------|----------|------------------------------------------------|-----------|--------------------------------------------|--|
| ---                               | ---             | ---      | ---                                            | ---       |                                            |  |
| <code>`startRealtime1st()`</code> |                 |          | <code>`app.schedulers.startRealtime1st`</code> |           | <code>`RealTimeDataListenerService`</code> |  |

```

| `stopRealtime1st()` | `app.schedulers.stopRealtime1st` | `RealTimeDataListenerService`
| `startRealtime2nd()` | `app.schedulers.startRealtime2nd` | `RealTimeDataListenerService`
| `stopRealtime2nd()` | `app.schedulers.stopRealtime2nd` | `RealTimeDataListenerService`
| `downloadSymbol()` | `app.schedulers.downloadSymbol` | `ISymbolInfoService` | `downloadSymbol`
| `riseFallStockRankSave()` | `app.schedulers.riseFallStockRankSave` | `RiseFallStockRankSave`
| `rightInfoSave()` | `app.schedulers.rightInfoSave` | `RightInfoService` | `getAndSaveRightInfo`

```

Sources:[] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/job/JobService.java#L38-75>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/job/JobService.java#L38-75>)

[src/main/java/com/difisoft/marketcollector/job/JobService.java38-75] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/job/JobService.java#L38-75>)

#### #### Real-Time Data Session Management

The system implements two separate real-time data collection sessions, each with start and stop methods.

##### ##### Session 1 Jobs

| Job Method           | Purpose                                            | Configuration                         |
|----------------------|----------------------------------------------------|---------------------------------------|
| `startRealtime1st()` | Starts the first real-time data collection session | `\${app.schedulers.startRealtime1st}` |
| `stopRealtime1st()`  | Stops the first real-time data collection session  | `\${app.schedulers.stopRealtime1st}`  |

##### ##### Session 2 Jobs

| Job Method           | Purpose                                             | Configuration                         |
|----------------------|-----------------------------------------------------|---------------------------------------|
| `startRealtime2nd()` | Starts the second real-time data collection session | `\${app.schedulers.startRealtime2nd}` |
| `stopRealtime2nd()`  | Stops the second real-time data collection session  | `\${app.schedulers.stopRealtime2nd}`  |

Both session types use the same `RealTimeDataListenerService` for actual data collection and processing.

Sources:[] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/job/JobService.java#L38-56>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/job/JobService.java#L38-56>)

[src/main/java/com/difisoft/marketcollector/job/JobService.java38-56] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/job/JobService.java#L38-56>)

#### #### Symbol Information Jobs

##### ##### Symbol Download Job

The `downloadSymbol()` job periodically downloads and updates symbol information from external data sources.

**\*\*Method Signature\*\*:**

```
@Scheduled(cron = "${app.schedulers.downloadSymbol}")
```

```
public void downloadSymbol()
```

**\*\*Execution Flow\*\*:**

![[Chart 2](market-collector-lotte-images/market-collector-lotte-section20-2.svg)]

**\*\*Diagram: Symbol Download Job Execution Flow\*\***

- **\*\*Trigger\*\*:** Scheduled via `\${app.schedulers.downloadSymbol}` cron expression
- **\*\*Implementation\*\*:** Calls `symbolInfoService.downloadSymbol("JobScheduler")`
- **\*\*Purpose\*\*:** Maintains up-to-date symbol metadata for market instruments
- **\*\*Request Identifier\*\*:** The string `"JobScheduler"` is passed as the `requestBy` parameter

Sources:[] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/job/JobService.java#L58-61>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/job/JobService.java#L58-61>)

[src/main/java/com/difisoft/marketcollector/job/JobService.java#L58-61] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/job/JobService.java#L58-61>)

**#### Market Data Processing Jobs****##### Rise/Fall Stock Ranking Job**

The `riseFallStockRankSave()` job processes and saves stock ranking data based on price movements

**\*\*Method Signature\*\*:**

```
@Scheduled(cron = "${app.schedulers.riseFallStockRankSave}")
```

```
public void riseFallStockRankSave()
```

**\*\*Conditional Execution Logic\*\*:**

![[Chart 3](market-collector-lotte-images/market-collector-lotte-section20-3.svg)]

**\*\*Diagram: Rise/Fall Stock Ranking Job with Feature Flag Check\*\***

- **\*\*Trigger\*\*:** Scheduled via `\${app.schedulers.riseFallStockRankSave}` cron expression
- **\*\*Feature Flag\*\*:** Only executes when `appConf.isEnableQuery()` returns `true`
- **\*\*Implementation\*\*:** Calls `riseFallStockRankService.getAndSaveStockRanking()`
- **\*\*Purpose\*\*:** Computes and persists stock ranking data based on price movements for queried instruments

Sources:[] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/job/JobService.java#L63-68>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/job/JobService.java#L63-68>)

[src/main/java/com/difisoft/marketcollector/job/JobService.java#L63-68] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/job/JobService.java#L63-68>)

**##### Right Information Job**

The `rightInfoSave()` job processes and saves rights information for financial instruments

**\*\*Method Signature\*\*:**

```
@Scheduled(cron = "${app.schedulers.rightInfoSave}")
```

```
public void rightInfoSave()
```

**\*\*Conditional Execution Logic\*\*:**

![Chart 4] (market-collector-lotte-images/market-collector-lotte-section20-4.svg)

**\*\*Diagram: Right Information Job with Feature Flag Check\*\***

- **\*\*Trigger\*\***: Scheduled via `\${app.schedulers.rightInfoSave}` cron expression
- **\*\*Feature Flag\*\***: Only executes when `appConf.isEnableQuery()` returns `true`
- **\*\*Implementation\*\***: Calls `rightInfoService.getAndSaveRightInfo()`
- **\*\*Purpose\*\***: Retrieves and persists corporate action data such as dividends, stock splits

Sources:[] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/job/JobService.java70-75>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/job/JobService.java70-75>)

[src/main/java/com/difisoft/marketcollector/job/JobService.java70-75] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/job/JobService.java70-75>)

**#### Service Dependencies and Configuration****##### JobService Dependencies**

The `JobService` class requires the following service dependencies, injected through constructor:

| Service                     | Purpose                                     |
|-----------------------------|---------------------------------------------|
| CacheService                | Cache management operations                 |
| RealTimeDataListenerService | Real-time data collection control           |
| AppConf                     | Application configuration and feature flags |
| ISymbolInfoService          | Symbol information management               |
| RiseFallStockRankService    | Stock ranking data processing               |
| RightInfoService            | Rights information processing               |

**##### Configuration Properties**

All scheduled jobs use externalized cron expressions defined in application configuration:

![Chart 5] (market-collector-lotte-images/market-collector-lotte-section20-5.svg)

This configuration approach allows for flexible scheduling adjustments without code changes:

Sources:[] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/job/JobService.java22-36>) (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/job/JobService.java22-36>)

[src/main/java/com/difisoft/marketcollector/job/JobService.java22-36] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/job/JobService.java22-36>)

src/main/java/com/difisoft/marketcollector/job/JobService.java38-75] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/job/JobService.java38-75>)

---

**### Request Processing****Relevant source files**

- [src/main/java/com/difisoft/marketcollector/consumers/RequestHandler.java] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/consumers/RequestHandler.java>)

#### #### Purpose and Scope

The Request Processing system provides on-demand, job-based API endpoints for administrative

The `RequestHandler` class implements a job-based request/response pattern, allowing external

- Symbol data downloads and refreshes
- Historical data recovery (quotes and daily data)
- Real-time data collection start/stop control
- System diagnostics and shutdown

This mechanism enables manual intervention, ad-hoc data recovery, and operational control of

**Sources:** <https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main>

[\[src/main/java/com/difisoft/marketcollector/consumers/RequestHandler.java#L1-60\]](#) [https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/consumers/RequestHandler.java#L1-60)

\* \* \*

#### #### Job Handler Architecture

The `RequestHandler` extends the `JobHandler` base class, which implements a Kafka-based job

#### ##### Core Components

![[Chart 1](market-collector-lotte-images/market-collector-lotte-section21-1.svg)]

The `JobHandler` constructor is invoked with configuration parameters that define the Kafka

| Parameter         | Value                | Purpose                                           |
|-------------------|----------------------|---------------------------------------------------|
| ---               | ---                  | ---                                               |
| `objectMapper`    | Jackson ObjectMapper | JSON serialization/deserialization                |
| `kafkaBootstraps` | From AppConfig       | Kafka cluster connection string                   |
| `clusterId`       | From AppConfig       | Cluster identifier for multi-cluster environments |
| `workerId`        | "1"                  | Worker instance identifier                        |
| `concurrency`     | 5                    | Number of concurrent job processing threads       |

**Sources:** <https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main>

[\[src/main/java/com/difisoft/marketcollector/consumers/RequestHandler.java#L21-36\]](#) [https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main](https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/consumers/RequestHandler.java#L21-36)

\* \* \*

#### #### Endpoint Registration

The `RequestHandler` registers endpoints using a fluent URI builder pattern during construc

#### ##### Endpoint Catalog

![[Chart 2](market-collector-lotte-images/market-collector-lotte-section21-2.svg)]

#### ##### Detailed Endpoint Specifications

| Endpoint | URI                                             | Request Type                   | Handler Method                               | Purpose           |
|----------|-------------------------------------------------|--------------------------------|----------------------------------------------|-------------------|
| ---      | ---                                             | ---                            | ---                                          | ---               |
|          | `job:/api/v1/collector/downloadSymbol`          | `Object.class`                 | `symbolInfoService::downloadSymbol`          |                   |
|          | `job:/api/v1/collector/forceDownloadSymbol`     | `Object.class`                 | `symbolInfoService::forceDownloadSymbol`     |                   |
|          | `job:/api/v1/collector/quoteDataRecover`        | `String.class`                 | `quoteRecoverService::downloadSymbol`        |                   |
|          | `job:/api/v1/collector/dailyDataRecover`        | `Object.class`                 | `dailyRecoverService::downloadSymbol`        |                   |
|          | `job:/api/v1/collector/dailyDataRecoverByApi`   | `DailyApiRecoverRequest.class` | `dailyRecoverService::downloadSymbol`        |                   |
|          | `job:/api/v1/collector/dailyDataRecoverNoCache` | `Object.class`                 | `dailyRecoverService::downloadSymbol`        |                   |
|          | `job:/api/v1/collector/quoteStatistic`          | `Object.class`                 | `quoteStatisticService::getStatistic`        |                   |
|          | `job:/api/v1/collector/realtime/start`          | `Object.class`                 | Lambda: `realTimeDataListenerService::start` |                   |
|          | `job:/api/v1/collector/realtime/stop`           | `Object.class`                 | Lambda: `realTimeDataListenerService::stop`  |                   |
|          | `job:/api/v1/collector/exit`                    | `Object.class`                 | Lambda: `systemService.shutdown()`           | Graceful shutdown |

**Sources:** <https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/consumers/RequestHandler.java#L37-57> (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/consumers/RequestHandler.java#L37-57>)

[src/main/java/com/difisoft/marketcollector/consumers/RequestHandler.java37-57] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/consumers/RequestHandler.java#L37-57>)

\* \* \*

#### #### Request Processing Flow

The following diagram illustrates how a job request flows through the system from submission to processing and result publishing.

! [Chart 3] (market-collector-lotte-images/market-collector-lotte-section21-3.svg)

#### ##### Processing Steps

- Job Submission:** External clients publish job requests to the configured Kafka job topic. The request contains:
  - URI pattern identifying the target endpoint
  - Payload containing request parameters
  - Correlation ID for result tracking
- Message Consumption:** The `RequestHandler` consumes messages from the job topic using a `KafkaConsumer`.
- Request Routing:** The URI builder pattern matches the request URI to the appropriate handler method.
- Type Conversion:** The framework deserializes the payload JSON into the expected request object.
- Handler Execution:** The target service method executes the requested operation, accessing the database or external services as needed.
- Result Publishing:** The handler's return value is serialized and published to the result topic.

**Sources:** <https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/consumers/RequestHandler.java#L1-60> (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/consumers/RequestHandler.java#L1-60>)

[src/main/java/com/difisoft/marketcollector/consumers/RequestHandler.java1-60] (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/consumers/RequestHandler.java#L1-60>)

\* \* \*

#### #### Service Integration Patterns

The `RequestHandler` integrates with seven service classes injected via Spring dependency injection.

! [Chart 4] (market-collector-lotte-images/market-collector-lotte-section21-4.svg)

#### ##### Service Dependencies

The constructor receives the following autowired services:

| Service                       | Interface/Class               | Primary Responsibility                         |
|-------------------------------|-------------------------------|------------------------------------------------|
| ---                           | ---                           | ---                                            |
| `symbolInfoService`           | `ISymbolInfoService`          | Symbol metadata management (download, storage) |
| `realTimeDataListenerService` | `RealTimeDataListenerService` | WebSocket connection listener                  |
| `dailyRecoverService`         | `DailyRecoverService`         | Historical daily data recovery from HTS        |
| `dailyRecoverByApiService`    | `DailyRecoverByApiService`    | Historical daily data recovery from API        |
| `quoteRecoverService`         | `QuoteRecoverService`         | Historical quote data recovery                 |
| `quoteStatisticService`       | `QuoteStatisticService`       | Quote data quality and coverage statistics     |
| `systemService`               | `SystemService`               | Application lifecycle management               |

The `ISymbolInfoService` interface enables conditional implementation selection (Lotte API or HTS).

**Sources:** <https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/consumers/RequestHandler.java#L24-35> (<https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main/java/com/difisoft/marketcollector/consumers/RequestHandler.java#L24-35>)

\* \* \*

#### #### Operational Use Cases

##### ##### Symbol Data Management

**Endpoint:** `job:/api/v1/collector/downloadSymbol`

Triggers symbol information download from the configured source (Lotte API or HTS). This operation performs the following tasks:

- Downloads the complete symbol list for all exchanges (HOSE, HNX, UPCOM)
- Updates MongoDB collections with current symbol metadata
- Uploads symbol list files to MinIO/S3 storage
- Publishes symbol update events to Kafka

**Forced Download:** The `forceDownloadSymbol` endpoint bypasses any caching mechanisms and triggers a fresh download.

##### ##### Historical Data Recovery

**Quote Recovery** (`job:/api/v1/collector/quoteDataRecover`):

- Recovers intraday quote data for specified date ranges
- Useful for filling gaps in real-time collection
- Request payload specifies symbols and date range

**Daily Recovery** (`job:/api/v1/collector/dailyDataRecover`):

- Recovers end-of-day summary data (OHLCV)
- Supports cache-enabled mode (default) and cache-bypass mode (`dailyDataRecoverNoCache`)
- Can process single symbols or bulk recovery

**API-based Recovery** (`job:/api/v1/collector/dailyDataRecoverByApi`):

- Uses Lotte Securities API for historical data retrieval
- Accepts typed request: `DailyRecoverByApiService.DailyApiRecoverRequest`
- Preferred method when HTS system is unavailable

##### Real-time Control

**\*\*Start Real-time Collection\*\*** (`job:/api/v1/collector/realtime/start`):

```
(r, c) -> {
  realTimeDataListenerService.run();
  return null;
}
```

Initiates WebSocket connections to HTS system and begins streaming market data. See [WebSoc

**\*\*Stop Real-time Collection\*\*** (`job:/api/v1/collector/realtime/stop`):

```
(r, c) -> {
  realTimeDataListenerService.stop();
  return null;
}
```

Gracefully closes WebSocket connections and suspends real-time data collection.

##### System Diagnostics

**\*\*Quote Statistics\*\*** (`job:/api/v1/collector/quoteStatistic`):

- Generates comprehensive statistics on collected quote data
- Reports coverage, gaps, and data quality metrics
- Useful for monitoring data collection health

##### System Shutdown

**\*\*Exit\*\*** (`job:/api/v1/collector/exit`):

```
(r, c) -> {
  systemService.shutdown();
  return null;
}
```

Initiates graceful application shutdown, allowing in-flight operations to complete before t

**Sources:** <https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main>

[\[src/main/java/com/difisoft/marketcollector/consumers/RequestHandler.java38-56\]](#) <https://git>

\* \* \*

### Error Handling and Response Format

While the specific error handling and response format are defined in the `JobHandler` base

- Type Safety:** Each endpoint specifies the expected request type, enabling automatic
- Null Returns:** Lambda handlers explicitly return `null` for operations that don't p
- Service Exceptions:** Any exceptions thrown by service methods are caught by the `Job

The correlation ID mechanism ensures that clients can reliably match responses to their or

**Sources:** <https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main>

[\[src/main/java/com/difisoft/marketcollector/consumers/RequestHandler.java36\]](#) <https://github>

\* \* \*

### Configuration Dependencies

The `RequestHandler` initialization requires three configuration values from `AppConf`:

![\[Chart 5\] \(market-collector-lotte-images/market-collector-lotte-section21-5.svg\)](#)

These values determine:

- Kafka Bootstraps:** The Kafka cluster address for consuming job requests and publish:
- Cluster ID:** Identifies the deployment cluster, enabling multi-cluster job routing :

See [\[Configuration Management\] \(#configuration-management\)](#) for details on `AppConf` structu

**Sources:** <https://github.com/nhsv-tradex/market-collector-lotte/blob/1df4d191/src/main>

[\[src/main/java/com/difisoft/marketcollector/consumers/RequestHandler.java36\]](#) <https://github>

---